Deep Learning with Neural Networks

Autoregressive models

Alejandro Lancho Serrano, alancho@ing.uc3m.es

A quick recap on probability theory

- Two main rules that we need to remember

Marginalization rule: $p(x) = \sum_y p(x, y)$

Product rule: $p(x, y) = p(y \mid x)p(x) = p(x \mid y)p(y)$ ← Bayes Theorem!

- How to model **any** joint distribution over observed data?

Product rule!

$$p(\mathbf{x}) = p(x_1) \prod_{d=2}^{D} p(x_d \mid \mathbf{x}_{<d})$$

where $\mathbf{x}_{<d} = [x_1, \ldots, x_{d-1}]^T$

Deep belief networks (a.k.a. deep autoregressive models)

- Explicit model conditional probabilities

$$p_{\text{model}}(\boldsymbol{x}) = p_{\text{model}}(x_1) \prod_{i=2}^{n} p_{\text{model}}(x_i | x_1, \ldots, x_{i-1})$$

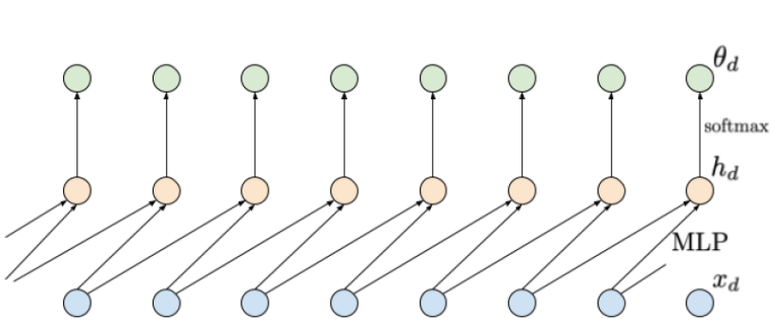
Each conditional can be a neural network

- Maximum likelihood training

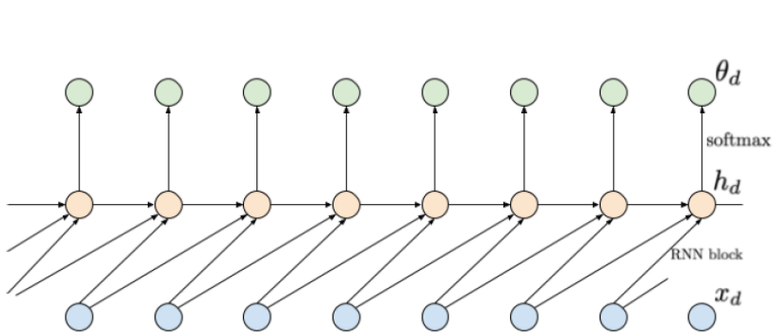
$$\mathcal{L} = \frac{1}{N} \sum_{n=1}^{N} \log p_{\text{model}}(\mathbf{x}_n)$$

Modeling all conditional probabilities separately can be problematic!

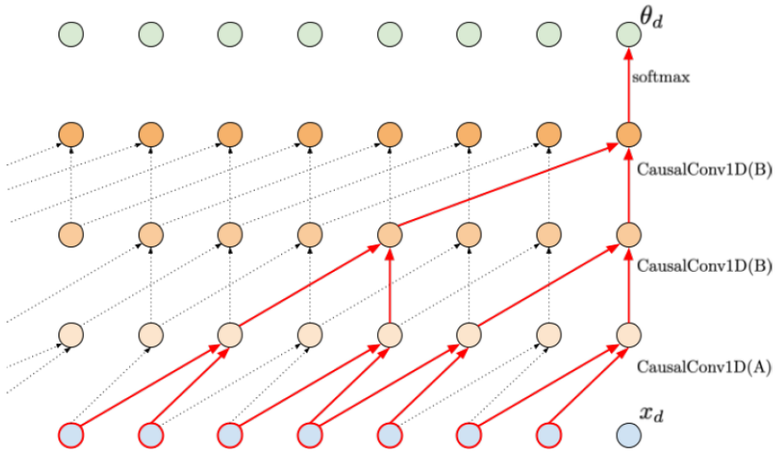
Possible ways to model conditional probabilities efficiently



Finite memory



Infinite memory (RNNs)



Infinite memory (CNNs)

More up to date, **Transformers**!

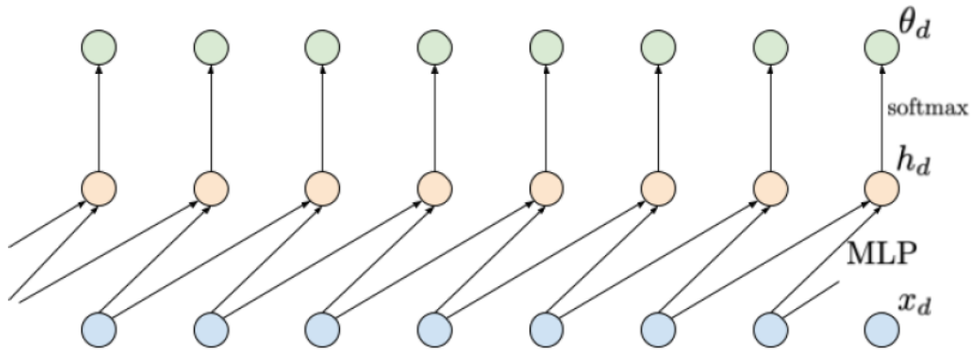
Finite memory

- We can express conditionals assuming **finite memory**

  Finite memory: e.g., 2 variables

  A share parameterization: e.g., an single MLP

  Why one?
  - **Weight sharing** across time
  - **Generalization** and **efficiency**
  - **Sequential dependency** modeling



Then, an example would be as follows:

$$p(x_d \mid x_{d-1}, x_{d-2}) = \text{Cat}(\text{softmax}(\text{NN}(x_{d-1}, x_{d-2})))$$

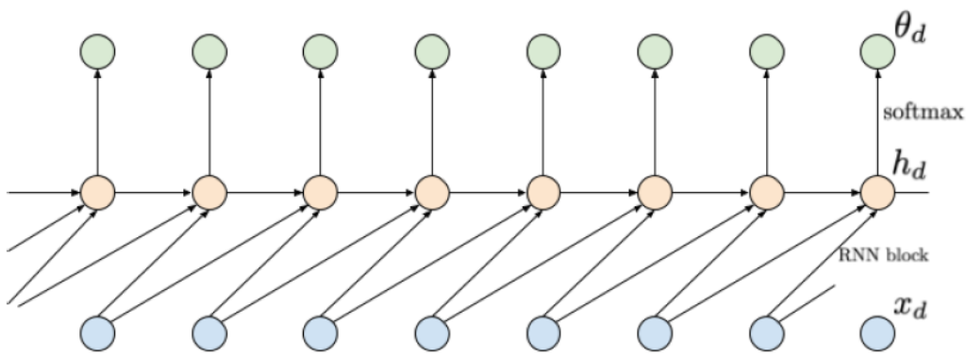
But, limiting as we need to decide for the length of the memory

Infinite memory: RNNs

- We can **model infinite memory** by using **RNNs**

  Infinite memory by **sharing context**

  A shared parameterization through an RNN



For example:

$$p(x_d | x_{d-1}, x_{d-2}) = \text{Cat}(\text{softmax}(\text{RNN}(x_{d-1}, h_{d-1})))$$

No need to define the memory length

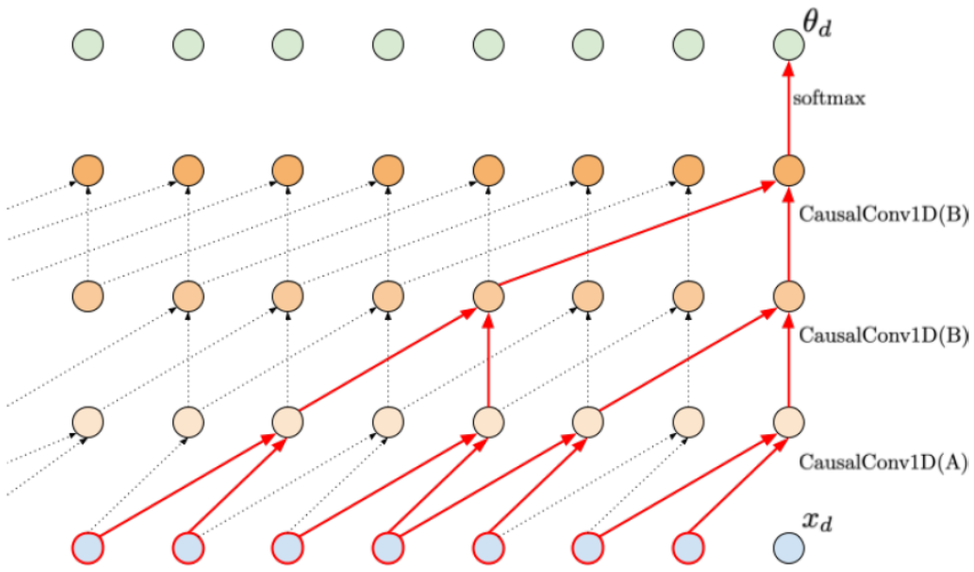
Training RNNs is sometimes problematic

Infinite memory: CNNs

- We can **model infinite memory** by using **CNNs**

  Infinite memory by **applying causal convolution**

  A shared parameterization through a CNN



For example:

$$p(x_d \mid x_{d-1}, x_{d-2}) = \text{Cat}(\text{softmax}(\text{CNN}(\mathbf{x}_{<d})))$$

No need to define the memory length

Easy training!

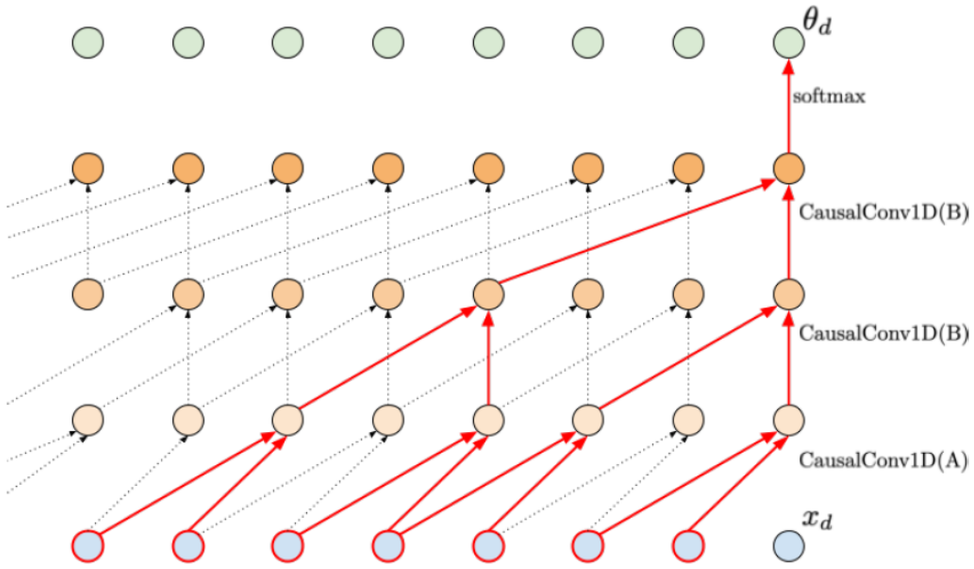
Infinite memory: CNNs

- **How to build** an autoregressive model using CNNs?

Consider $\mathbf{x} = [x_1, \ldots, x_D]^T$

We use causal 1D convolutions:

- **Option A**: Do not use x_d
- **Option B**: Use all inputs
- **Never look into the future!**



Additional tip: Use **dilation** if you want to enlarge dependencies

Infinite memory: CNNs

- **Causal convolutions**

```
class CausalConv1d(nn.Module):
    def __init__(self, in_channels, out_channels, kernel_size, dilation, A=False):
        super(CausalConv1d, self).__init__()

        self.kernel_size = kernel_size
        self.dilation = dilation  # important to get larger dependencies
        self.A = A  # option A (A=True) or option B (B=False)
        self.padding = (kernel_size - 1) * dilation + A * 1

        self.conv1d = nn.Conv1d(in_channels, out_channels,
                                kernel_size, stride=1, padding=0,
                                dilation=dilation)  # we pad later

    def forward(self, x):
        x = nn.functional.pad(x, (self.padding, 0))  # padding only on the left
        conv1d_out = self.conv1d(x)
        if self.A:
            return conv1d_out[:, :, : -1]  # NO dependency on the current element!
        else:
            return conv1d_out
```

Infinite memory: CNNs

- **Causal convolutions**

```
class CausalConv1d(nn.Module):
    def __init__(self, in_channels, out_channels, kernel_size, dilation, A=False):
        super(CausalConv1d, self).__init__()

        self.kernel_size = kernel_size
        self.dilation = dilation  # important to get larger dependencies
        self.A = A  # option A (A=True) or option B (B=False)
        self.padding = (kernel_size - 1) * dilation + A * 1

        self.conv1d = nn.Conv1d(in_channels, out_channels,
                                kernel_size, stride=1, padding=0,
                                dilation=dilation)  # we pad later

def forward(self, x):
    x = nn.functional.pad(x, (self.padding, 0))  # padding only on the left
    conv1d_out = self.conv1d(x)
    if self.A:
        return conv1d_out[:, :, : -1]  # NO dependency on the current element!
    else:
        return conv1d_out
```

Option A:
kernel size = 2
dilation = 1
A = True

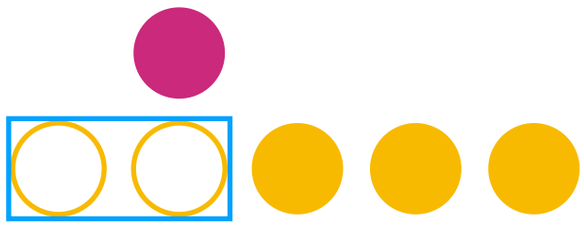
Pad first

Infinite memory: CNNs

- **Causal convolutions**

```
class CausalConv1d(nn.Module):
    def __init__(self, in_channels, out_channels, kernel_size, dilation, A=False):
        super(CausalConv1d, self).__init__()

        self.kernel_size = kernel_size
        self.dilation = dilation  # important to get larger dependencies
        self.A = A  # option A (A=True) or option B (B=False)
        self.padding = (kernel_size - 1) * dilation + A * 1

        self.conv1d = nn.Conv1d(in_channels, out_channels,
                                kernel_size, stride=1, padding=0,
                                dilation=dilation)  # we pad later

def forward(self, x):
    x = nn.functional.pad(x, (self.padding, 0))  # padding only on the left
    conv1d_out = self.conv1d(x)
    if self.A:
        return conv1d_out[:, :, : -1]  # NO dependency on the current element!
    else:
        return conv1d_out
```

Option A:
kernel size = 2
dilation = 1
A = True



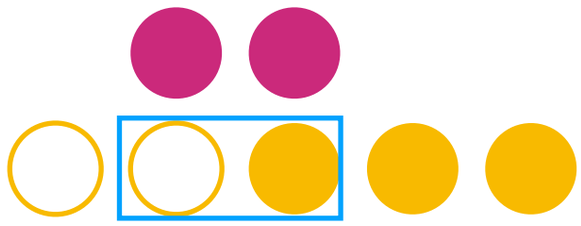
Pad first

Infinite memory: CNNs

- **Causal convolutions**

```
class CausalConv1d(nn.Module):
    def __init__(self, in_channels, out_channels, kernel_size, dilation, A=False):
        super(CausalConv1d, self).__init__()

        self.kernel_size = kernel_size
        self.dilation = dilation  # important to get larger dependencies
        self.A = A  # option A (A=True) or option B (B=False)
        self.padding = (kernel_size - 1) * dilation + A * 1

        self.conv1d = nn.Conv1d(in_channels, out_channels,
                                kernel_size, stride=1, padding=0,
                                dilation=dilation)  # we pad later

def forward(self, x):
    x = nn.functional.pad(x, (self.padding, 0))  # padding only on the left
    conv1d_out = self.conv1d(x)
    if self.A:
        return conv1d_out[:, :, : -1]  # NO dependency on the current element!
    else:
        return conv1d_out
```

Option A:
kernel size = 2
dilation = 1
A = True

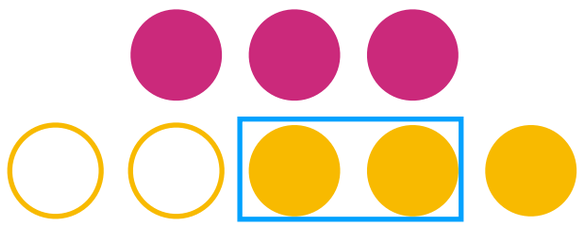


Infinite memory: CNNs

- **Causal convolutions**

```
class CausalConv1d(nn.Module):
    def __init__(self, in_channels, out_channels, kernel_size, dilation, A=False):
        super(CausalConv1d, self).__init__()

        self.kernel_size = kernel_size
        self.dilation = dilation  # important to get larger dependencies
        self.A = A  # option A (A=True) or option B (B=False)
        self.padding = (kernel_size - 1) * dilation + A * 1

        self.conv1d = nn.Conv1d(in_channels, out_channels,
                                kernel_size, stride=1, padding=0,
                                dilation=dilation)  # we pad later

def forward(self, x):
    x = nn.functional.pad(x, (self.padding, 0))  # padding only on the left
    conv1d_out = self.conv1d(x)
    if self.A:
        return conv1d_out[:, :, : -1]  # NO dependency on the current element!
    else:
        return conv1d_out
```

Option A:
kernel size = 2
dilation = 1
A = True

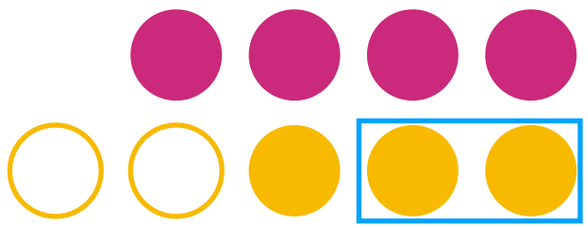


Infinite memory: CNNs

- **Causal convolutions**

```
class CausalConv1d(nn.Module):
    def __init__(self, in_channels, out_channels, kernel_size, dilation, A=False):
        super(CausalConv1d, self).__init__()

        self.kernel_size = kernel_size
        self.dilation = dilation  # important to get larger dependencies
        self.A = A  # option A (A=True) or option B (B=False)
        self.padding = (kernel_size - 1) * dilation + A * 1

        self.conv1d = nn.Conv1d(in_channels, out_channels,
                                kernel_size, stride=1, padding=0,
                                dilation=dilation)  # we pad later

    def forward(self, x):
        x = nn.functional.pad(x, (self.padding, 0))  # padding only on the left
        conv1d_out = self.conv1d(x)
        if self.A:
            return conv1d_out[:, :, : -1]  # NO dependency on the current element!
        else:
            return conv1d_out
```

Option A:
kernel size = 2
dilation = 1
A = True

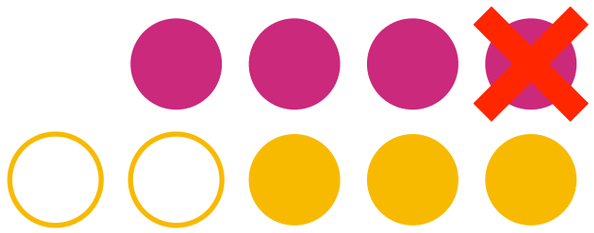


Infinite memory: CNNs

- **Causal convolutions**

```
class CausalConv1d(nn.Module):
    def __init__(self, in_channels, out_channels, kernel_size, dilation, A=False):
        super(CausalConv1d, self).__init__()

        self.kernel_size = kernel_size
        self.dilation = dilation  # important to get larger dependencies
        self.A = A  # option A (A=True) or option B (B=False)
        self.padding = (kernel_size - 1) * dilation + A * 1

        self.conv1d = nn.Conv1d(in_channels, out_channels,
                                kernel_size, stride=1, padding=0,
                                dilation=dilation)  # we pad later

    def forward(self, x):
        x = nn.functional.pad(x, (self.padding, 0))  # padding only on the left
        conv1d_out = self.conv1d(x)
        if self.A:
            return conv1d_out[:, :, : -1]  # NO dependency on the current element!
        else:
            return conv1d_out
```

Option A:
kernel size = 2
dilation = 1
A = True



Since it's **Option A**,
we skip the last element from the output

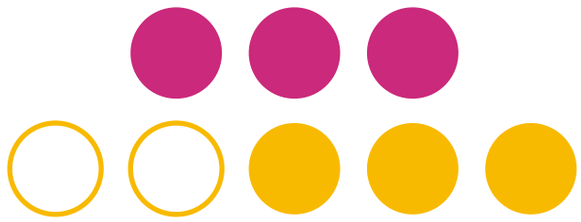
Infinite memory: CNNs

- **Causal convolutions**

```
class CausalConv1d(nn.Module):
    def __init__(self, in_channels, out_channels, kernel_size, dilation, A=False):
        super(CausalConv1d, self).__init__()

        self.kernel_size = kernel_size
        self.dilation = dilation  # important to get larger dependencies
        self.A = A  # option A (A=True) or option B (B=False)
        self.padding = (kernel_size - 1) * dilation + A * 1

        self.conv1d = nn.Conv1d(in_channels, out_channels,
                                kernel_size, stride=1, padding=0,
                                dilation=dilation)  # we pad later

def forward(self, x):
    x = nn.functional.pad(x, (self.padding, 0))  # padding only on the left
    conv1d_out = self.conv1d(x)
    if self.A:
        return conv1d_out[:, :, : -1]  # NO dependency on the current element!
    else:
        return conv1d_out
```

Option A:
kernel size = 2
dilation = 1
A = True

Option B:
kernel size = 2
dilation = 1
A = True



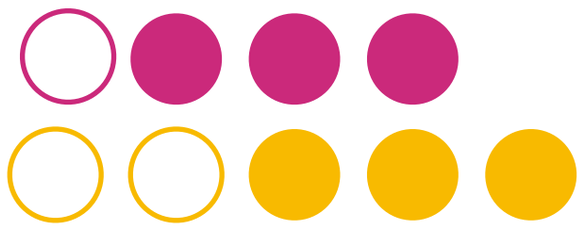
Infinite memory: CNNs

- **Causal convolutions**

```
class CausalConv1d(nn.Module):
    def __init__(self, in_channels, out_channels, kernel_size, dilation, A=False):
        super(CausalConv1d, self).__init__()

        self.kernel_size = kernel_size
        self.dilation = dilation  # important to get larger dependencies
        self.A = A  # option A (A=True) or option B (B=False)
        self.padding = (kernel_size - 1) * dilation + A * 1

        self.conv1d = nn.Conv1d(in_channels, out_channels,
                                kernel_size, stride=1, padding=0,
                                dilation=dilation)  # we pad later

def forward(self, x):
    x = nn.functional.pad(x, (self.padding, 0))  # padding only on the left
    conv1d_out = self.conv1d(x)
    if self.A:
        return conv1d_out[:, :, : -1]  # NO dependency on the current element!
    else:
        return conv1d_out
```

Option A:
kernel size = 2
dilation = 1
A = True

Option B:
kernel size = 2
dilation = 1
A = True



It's **Option B** now
We pad again from the left

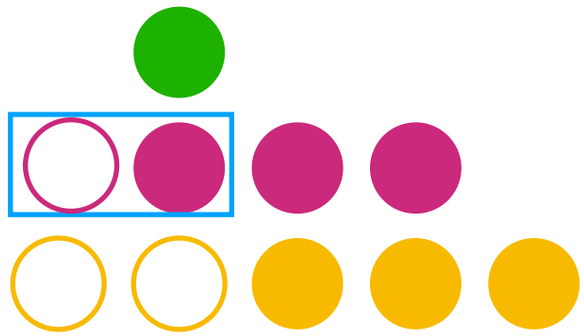
Infinite memory: CNNs

- **Causal convolutions**

```
class CausalConv1d(nn.Module):
    def __init__(self, in_channels, out_channels, kernel_size, dilation, A=False):
        super(CausalConv1d, self).__init__()

        self.kernel_size = kernel_size
        self.dilation = dilation  # important to get larger dependencies
        self.A = A  # option A (A=True) or option B (B=False)
        self.padding = (kernel_size - 1) * dilation + A * 1

        self.conv1d = nn.Conv1d(in_channels, out_channels,
                                kernel_size, stride=1, padding=0,
                                dilation=dilation)  # we pad later

def forward(self, x):
    x = nn.functional.pad(x, (self.padding, 0))  # padding only on the left
    conv1d_out = self.conv1d(x)
    if self.A:
        return conv1d_out[:, :, : -1]  # NO dependency on the current element!
    else:
        return conv1d_out
```

Option A:
kernel size = 2
dilation = 1
A = True

Option B:
kernel size = 2
dilation = 1
A = True



It's **Option B** now

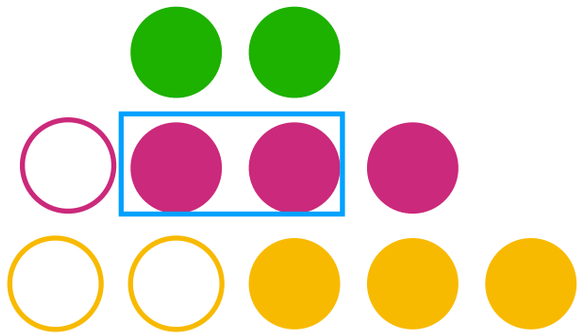
Infinite memory: CNNs

- **Causal convolutions**

```
class CausalConv1d(nn.Module):
    def __init__(self, in_channels, out_channels, kernel_size, dilation, A=False):
        super(CausalConv1d, self).__init__()

        self.kernel_size = kernel_size
        self.dilation = dilation  # important to get larger dependencies
        self.A = A  # option A (A=True) or option B (B=False)
        self.padding = (kernel_size - 1) * dilation + A * 1

        self.conv1d = nn.Conv1d(in_channels, out_channels,
                                kernel_size, stride=1, padding=0,
                                dilation=dilation)  # we pad later

def forward(self, x):
    x = nn.functional.pad(x, (self.padding, 0))  # padding only on the left
    conv1d_out = self.conv1d(x)
    if self.A:
        return conv1d_out[:, :, : -1]  # NO dependency on the current element!
    else:
        return conv1d_out
```

Option A:
kernel size = 2
dilation = 1
A = True

Option B:
kernel size = 2
dilation = 1
A = True



It's **Option B** now

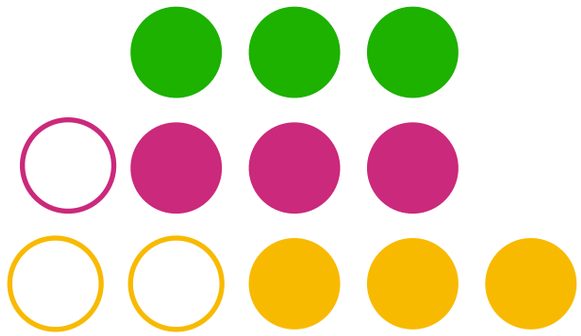
Infinite memory: CNNs

- **Causal convolutions**

```
class CausalConv1d(nn.Module):
    def __init__(self, in_channels, out_channels, kernel_size, dilation, A=False):
        super(CausalConv1d, self).__init__()

        self.kernel_size = kernel_size
        self.dilation = dilation  # important to get larger dependencies
        self.A = A  # option A (A=True) or option B (B=False)
        self.padding = (kernel_size - 1) * dilation + A * 1

        self.conv1d = nn.Conv1d(in_channels, out_channels,
                                kernel_size, stride=1, padding=0,
                                dilation=dilation)  # we pad later

def forward(self, x):
    x = nn.functional.pad(x, (self.padding, 0))  # padding only on the left
    conv1d_out = self.conv1d(x)
    if self.A:
        return conv1d_out[:, :, : -1]  # NO dependency on the current element!
    else:
        return conv1d_out
```

Option A:
kernel size = 2
dilation = 1
A = True

Option B:
kernel size = 2
dilation = 1
A = True



It's **Option B** now

Infinite memory: CNNs

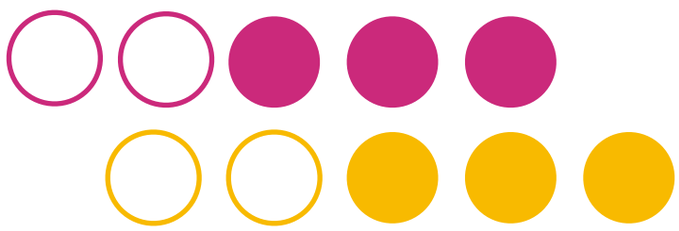
- **Causal convolutions**

```
class CausalConv1d(nn.Module):
    def __init__(self, in_channels, out_channels, kernel_size, dilation, A=False):
        super(CausalConv1d, self).__init__()

        self.kernel_size = kernel_size
        self.dilation = dilation  # important to get larger dependencies
        self.A = A  # option A (A=True) or option B (B=False)
        self.padding = (kernel_size - 1) * dilation + A * 1

        self.conv1d = nn.Conv1d(in_channels, out_channels,
                                kernel_size, stride=1, padding=0,
                                dilation=dilation)  # we pad later

def forward(self, x):
    x = nn.functional.pad(x, (self.padding, 0))  # padding only on the left
    conv1d_out = self.conv1d(x)
    if self.A:
        return conv1d_out[:, :, : -1]  # NO dependency on the current element!
    else:
        return conv1d_out
```

Option A:
kernel size = 2
dilation = 1
A = True

Option B:
kernel size = 2
dilation = 2
A = True

Pad again



It's **Option B** now

with dilation 2!

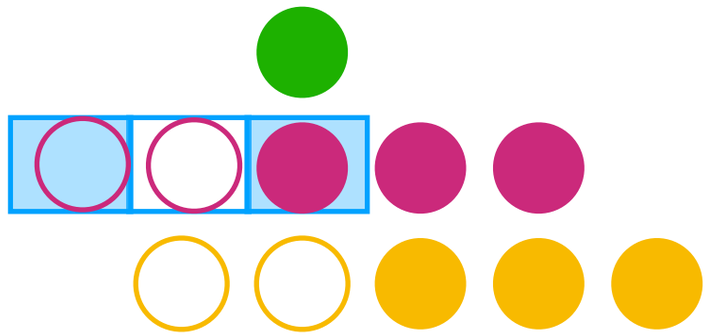
Infinite memory: CNNs

- **Causal convolutions**

```
class CausalConv1d(nn.Module):
    def __init__(self, in_channels, out_channels, kernel_size, dilation, A=False):
        super(CausalConv1d, self).__init__()

        self.kernel_size = kernel_size
        self.dilation = dilation  # important to get larger dependencies
        self.A = A  # option A (A=True) or option B (B=False)
        self.padding = (kernel_size - 1) * dilation + A * 1

        self.conv1d = nn.Conv1d(in_channels, out_channels,
                                kernel_size, stride=1, padding=0,
                                dilation=dilation)  # we pad later

def forward(self, x):
    x = nn.functional.pad(x, (self.padding, 0))  # padding only on the left
    conv1d_out = self.conv1d(x)
    if self.A:
        return conv1d_out[:, :, : -1]  # NO dependency on the current element!
    else:
        return conv1d_out
```

Option A:
kernel size = 2
dilation = 1
A = True

Option B:
kernel size = 2
dilation = 2
A = True



It's **Option B** now

with dilation 2!

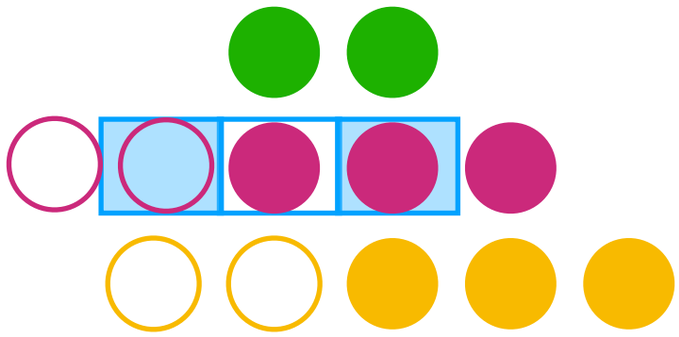
Infinite memory: CNNs

- **Causal convolutions**

```
class CausalConv1d(nn.Module):
    def __init__(self, in_channels, out_channels, kernel_size, dilation, A=False):
        super(CausalConv1d, self).__init__()

        self.kernel_size = kernel_size
        self.dilation = dilation  # important to get larger dependencies
        self.A = A  # option A (A=True) or option B (B=False)
        self.padding = (kernel_size - 1) * dilation + A * 1

        self.conv1d = nn.Conv1d(in_channels, out_channels,
                                kernel_size, stride=1, padding=0,
                                dilation=dilation)  # we pad later

def forward(self, x):
    x = nn.functional.pad(x, (self.padding, 0))  # padding only on the left
    conv1d_out = self.conv1d(x)
    if self.A:
        return conv1d_out[:, :, : -1]  # NO dependency on the current element!
    else:
        return conv1d_out
```

Option A:
kernel size = 2
dilation = 1
A = True

Option B:
kernel size = 2
dilation = 2
A = True



It's **Option B** now

with dilation 2!

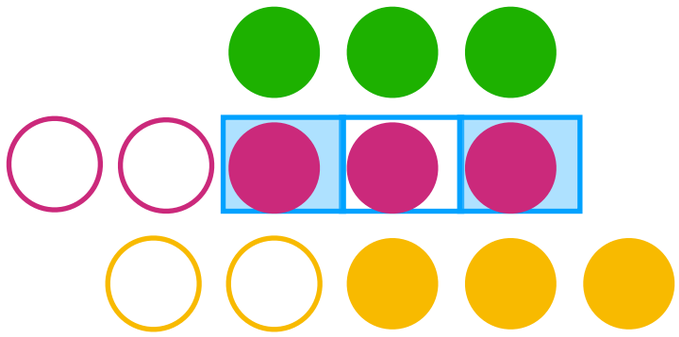
Infinite memory: CNNs

- **Causal convolutions**

```
class CausalConv1d(nn.Module):
    def __init__(self, in_channels, out_channels, kernel_size, dilation, A=False):
        super(CausalConv1d, self).__init__()

        self.kernel_size = kernel_size
        self.dilation = dilation  # important to get larger dependencies
        self.A = A  # option A (A=True) or option B (B=False)
        self.padding = (kernel_size - 1) * dilation + A * 1

        self.conv1d = nn.Conv1d(in_channels, out_channels,
                                kernel_size, stride=1, padding=0,
                                dilation=dilation)  # we pad later

def forward(self, x):
    x = nn.functional.pad(x, (self.padding, 0))  # padding only on the left
    conv1d_out = self.conv1d(x)
    if self.A:
        return conv1d_out[:, :, : -1]  # NO dependency on the current element!
    else:
        return conv1d_out
```

Option A:
kernel size = 2
dilation = 1
A = True

Option B:
kernel size = 2
dilation = 2
A = True

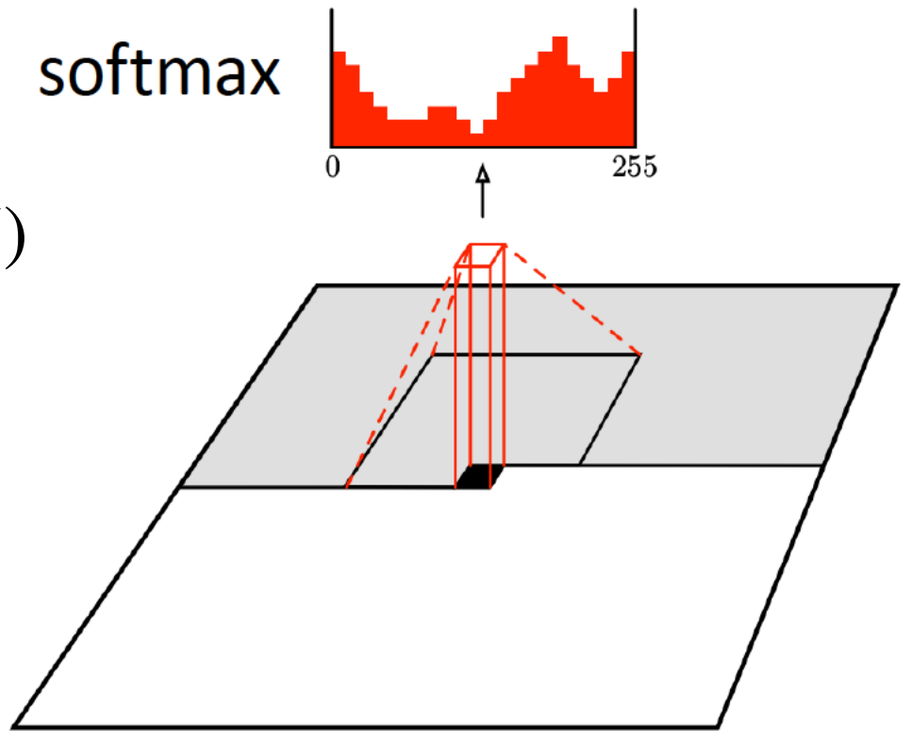


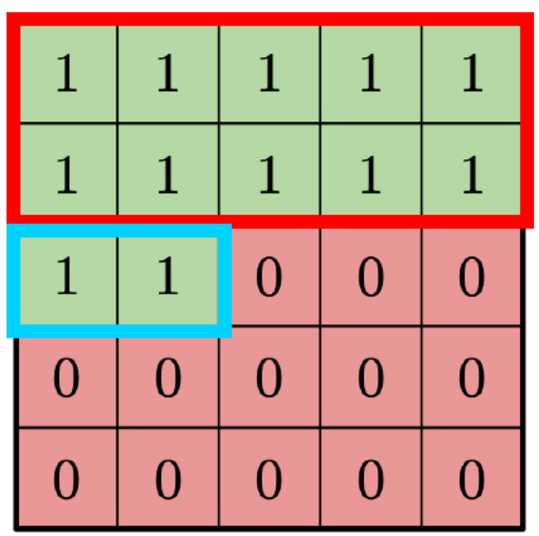
It's **Option B** now

with dilation 2!

We can use causal convolutions in 2D: PixelCNN

- We can use the very same idea with images and Conv2D
- Need to remember about **Causal Convolutions**
- We should remember about 3 dimensions $(C \times H \times W)$

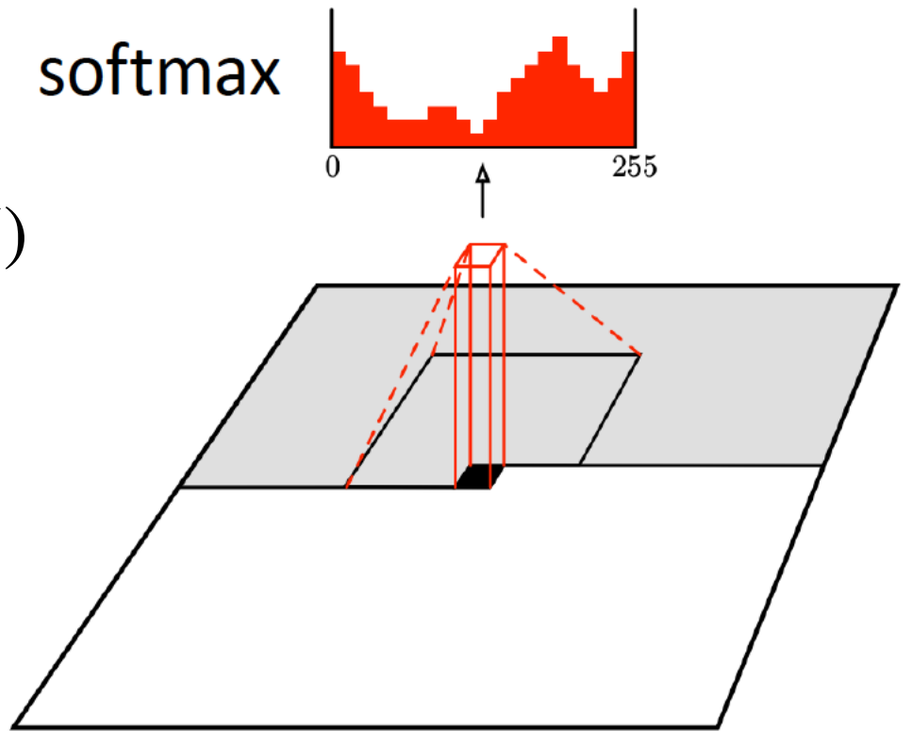




Van den Oord et al.,ICML2016
"Conditional image generation with PixelCNNdecoders"

We can use causal convolutions in 2D: PixelCNN

- We can use the very same idea with images and Conv2D
- Need to remember about **Causal Convolutions**
- We should remember about 3 dimensions $(C \times H \times W)$
- Accomplish this with two Conv2D layers:
  - The first Conv2D layer covers the **upper part**
  - The second Conv2D layer covers the **left part**

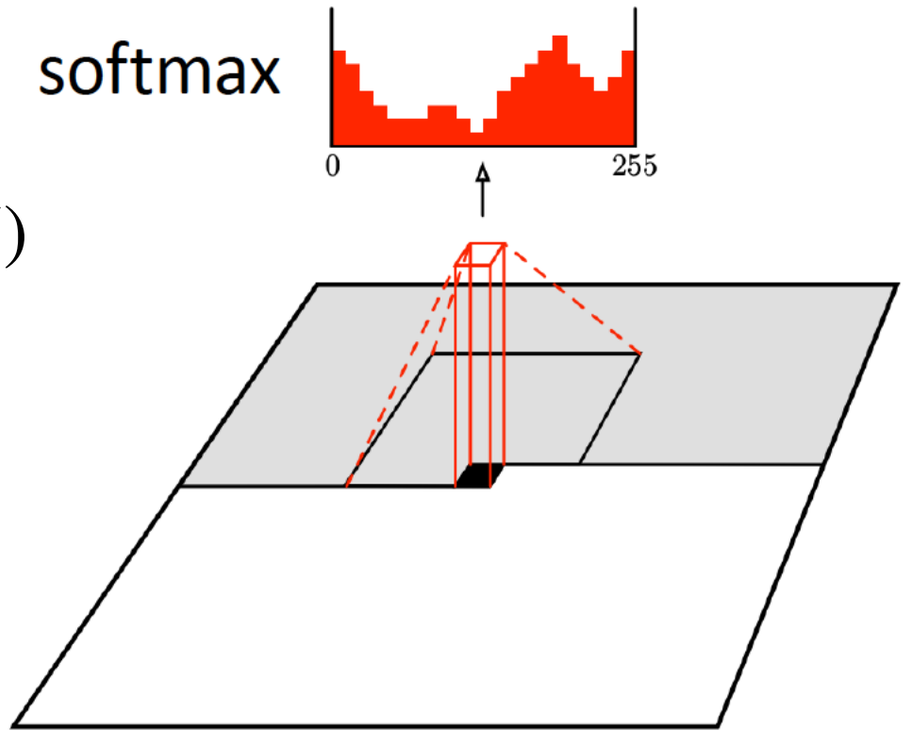


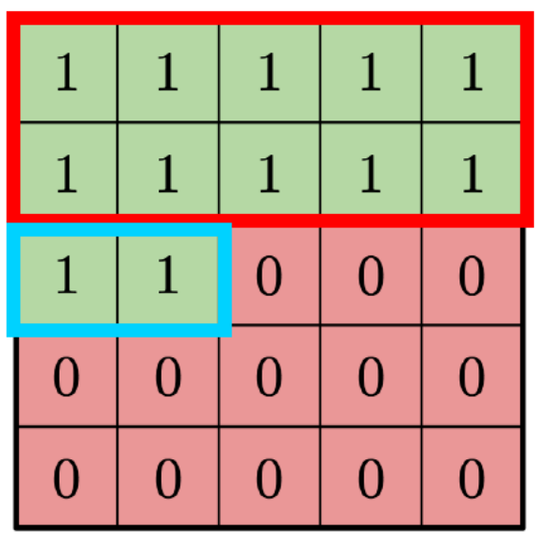
1	1	1	1	1
1	1	1	1	1
1	1	0	0	0
0	0	0	0	0
0	0	0	0	0

Van den Oord et al.,ICML2016
"Conditional image generation with PixelCNNdecoders"

We can use causal convolutions in 2D: PixelCNN

- We can use the very same idea with images and Conv2D
- Need to remember about **Causal Convolutions**
- We should remember about 3 dimensions $(C \times H \times W)$
- Accomplish this with clever masking

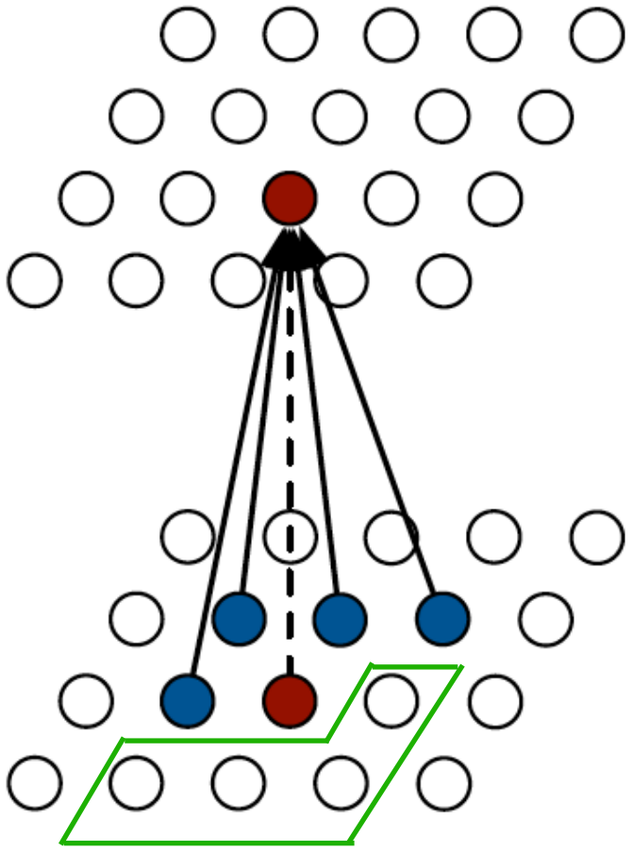




Van den Oord et al.,ICML2016
"Conditional image generation with PixelCNNdecoders"

Limitations of pixel CNN

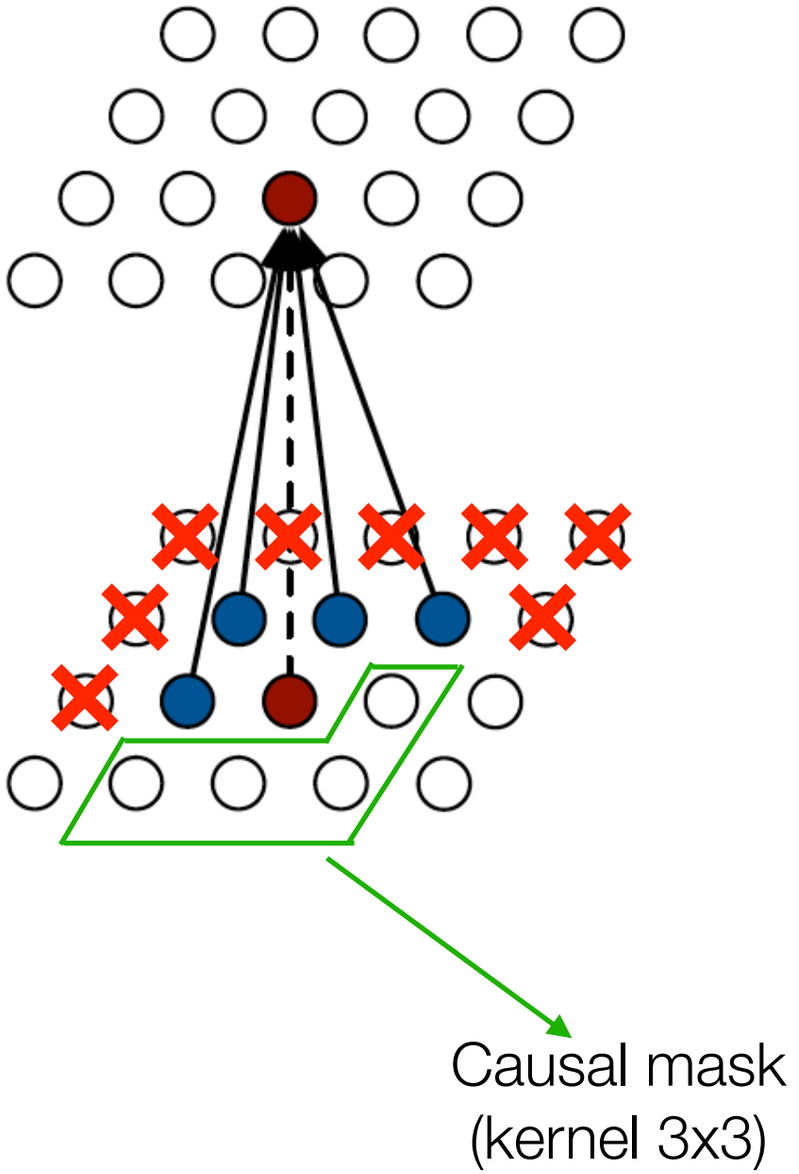
- Although it can be trained quite fast
- The **receptive field is limited**
- Does not use all the available context



Van den Oord et al.,ICML2016
"Conditional image generation with PixelCNNdecoders"

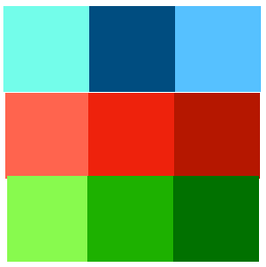
Limitations of pixel CNN

- Although it can be trained quite fast
- The **receptive field is limited**
- Does not use all the available context

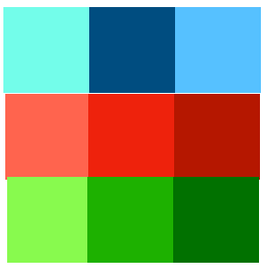


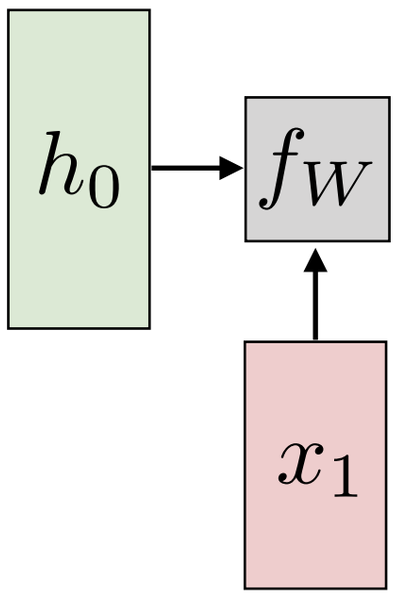
Van den Oord et al.,ICML2016
"Conditional image generation with PixelCNNdecoders"

Recurrent Neural Language Model (Training)

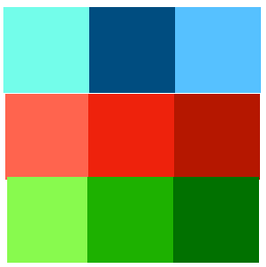


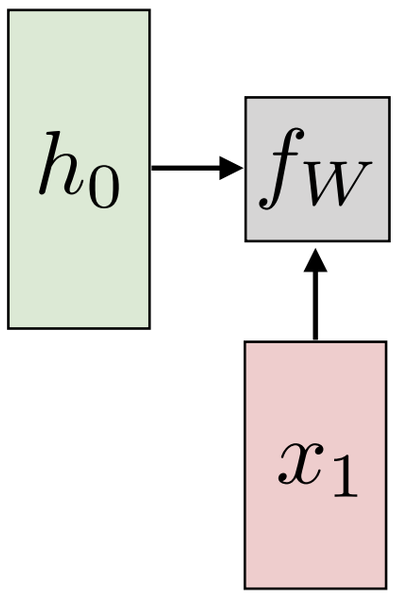
Recurrent Neural Language Model (Training)



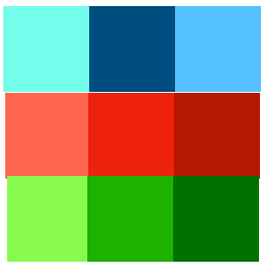


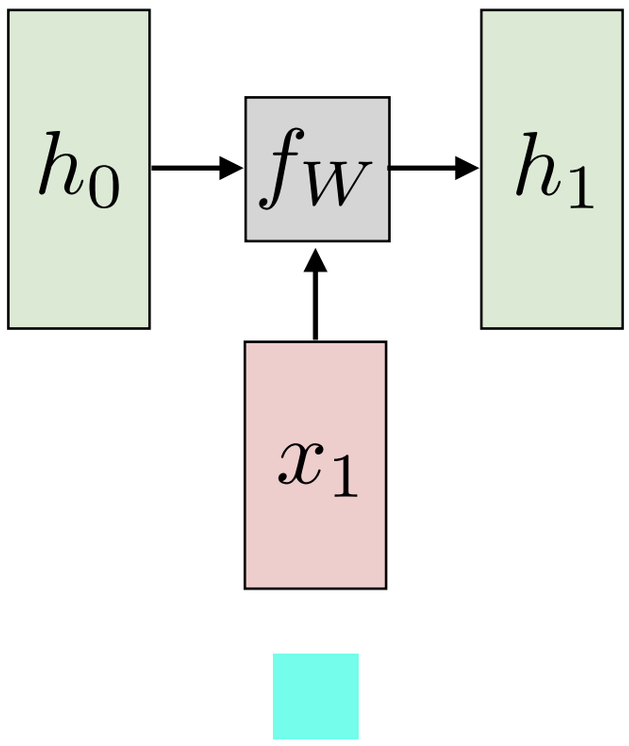
Recurrent Neural Language Model (Training)



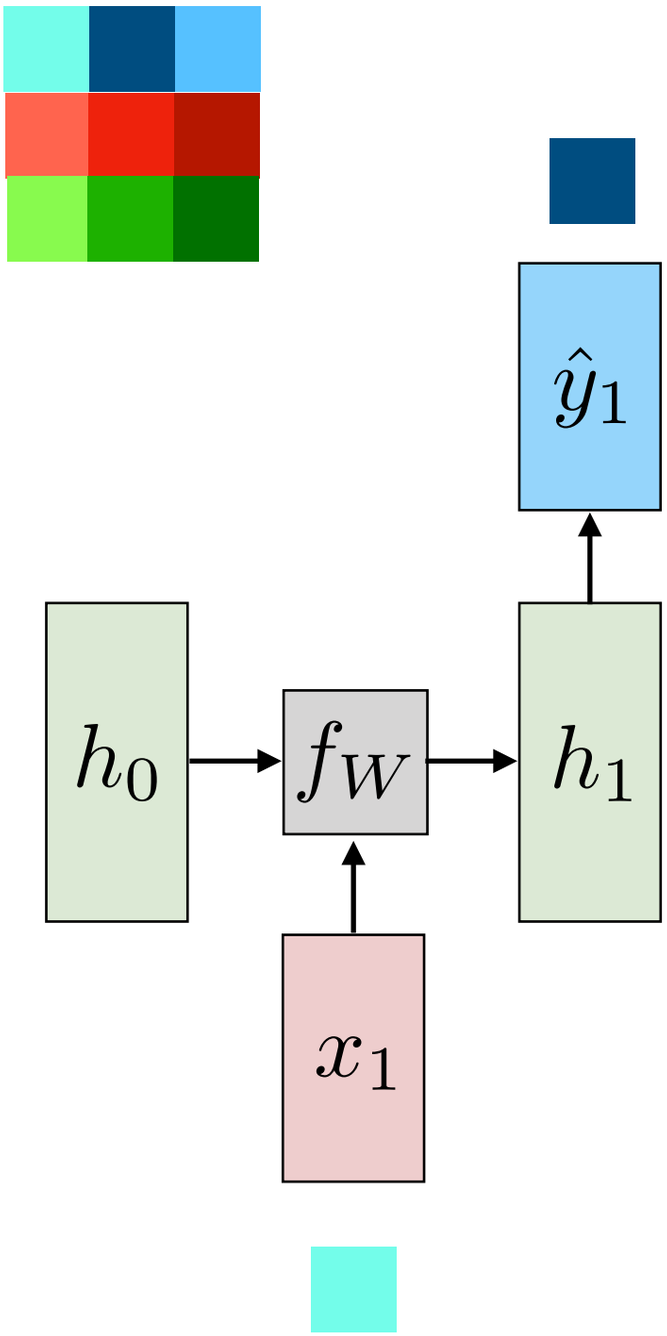


Recurrent Neural Language Model (Training)

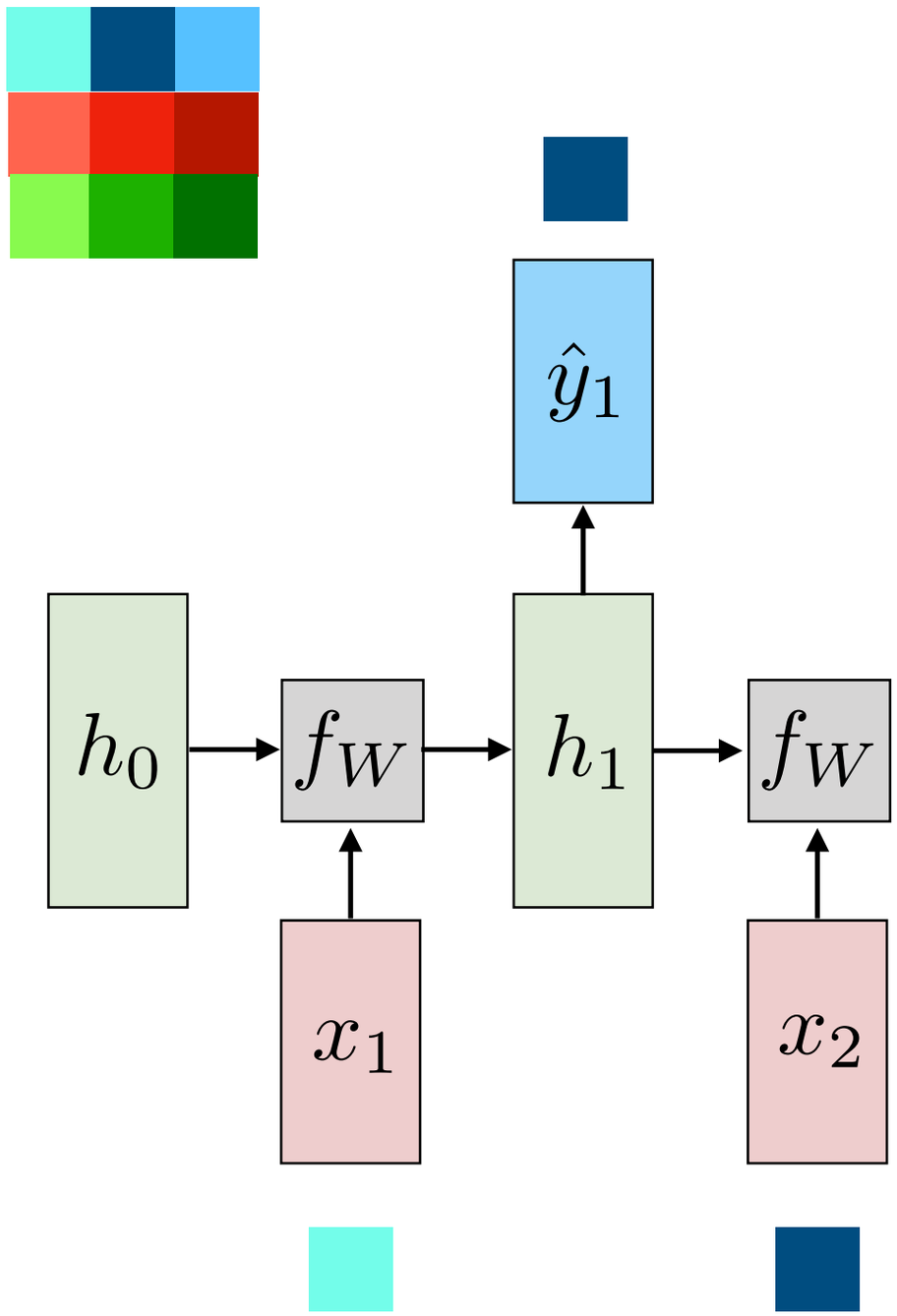




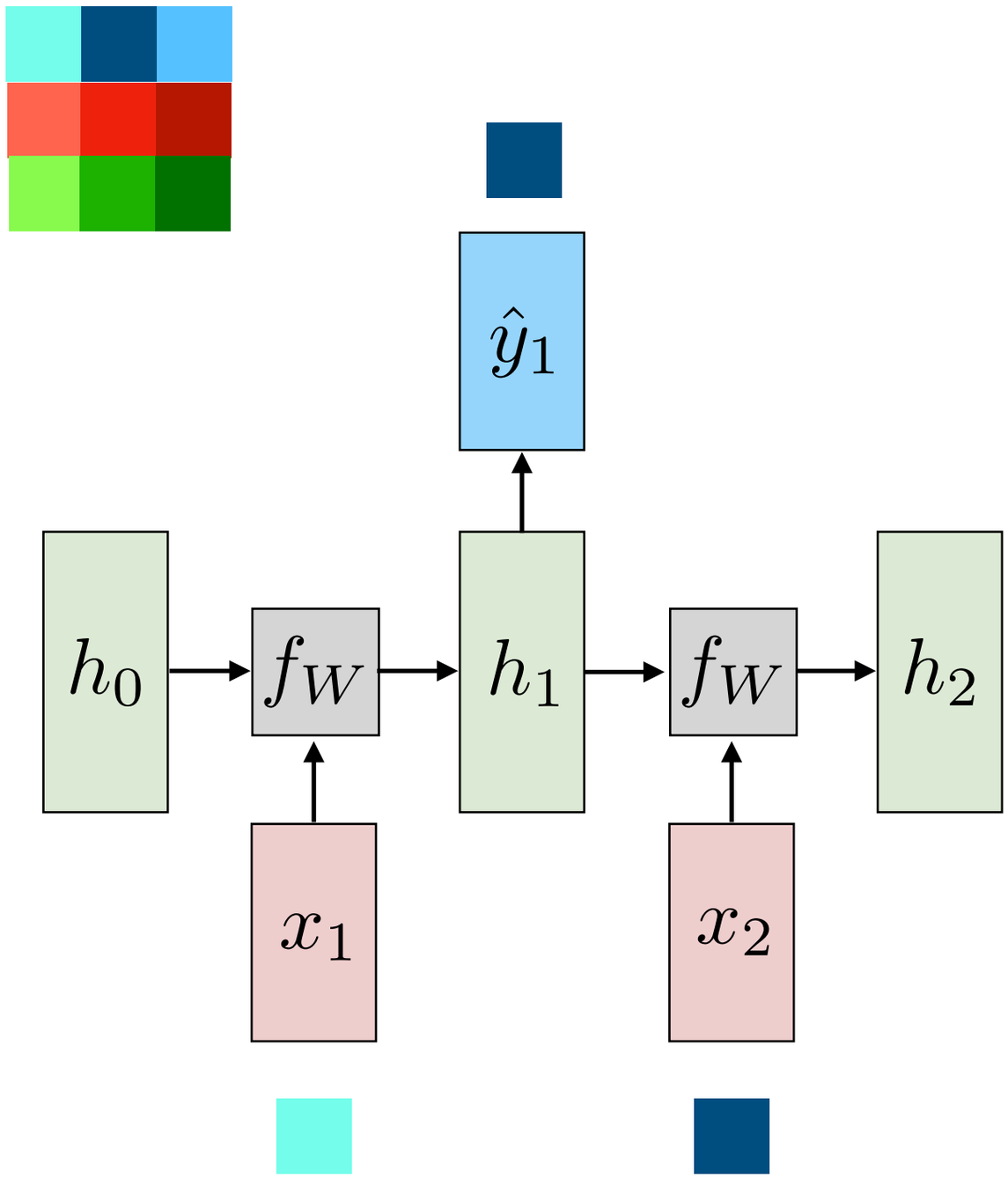
Recurrent Neural Language Model (Training)



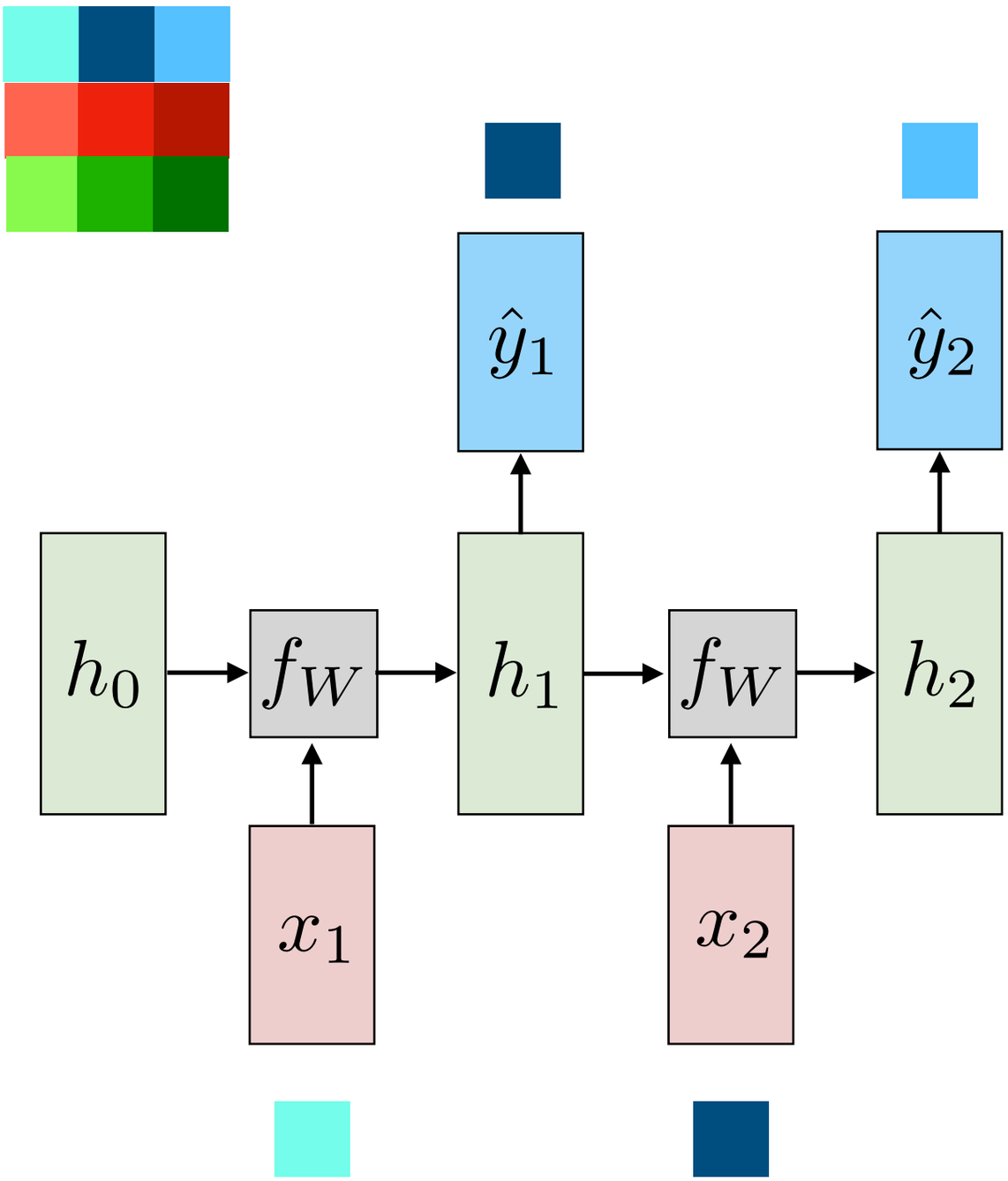
Recurrent Neural Language Model (Training)



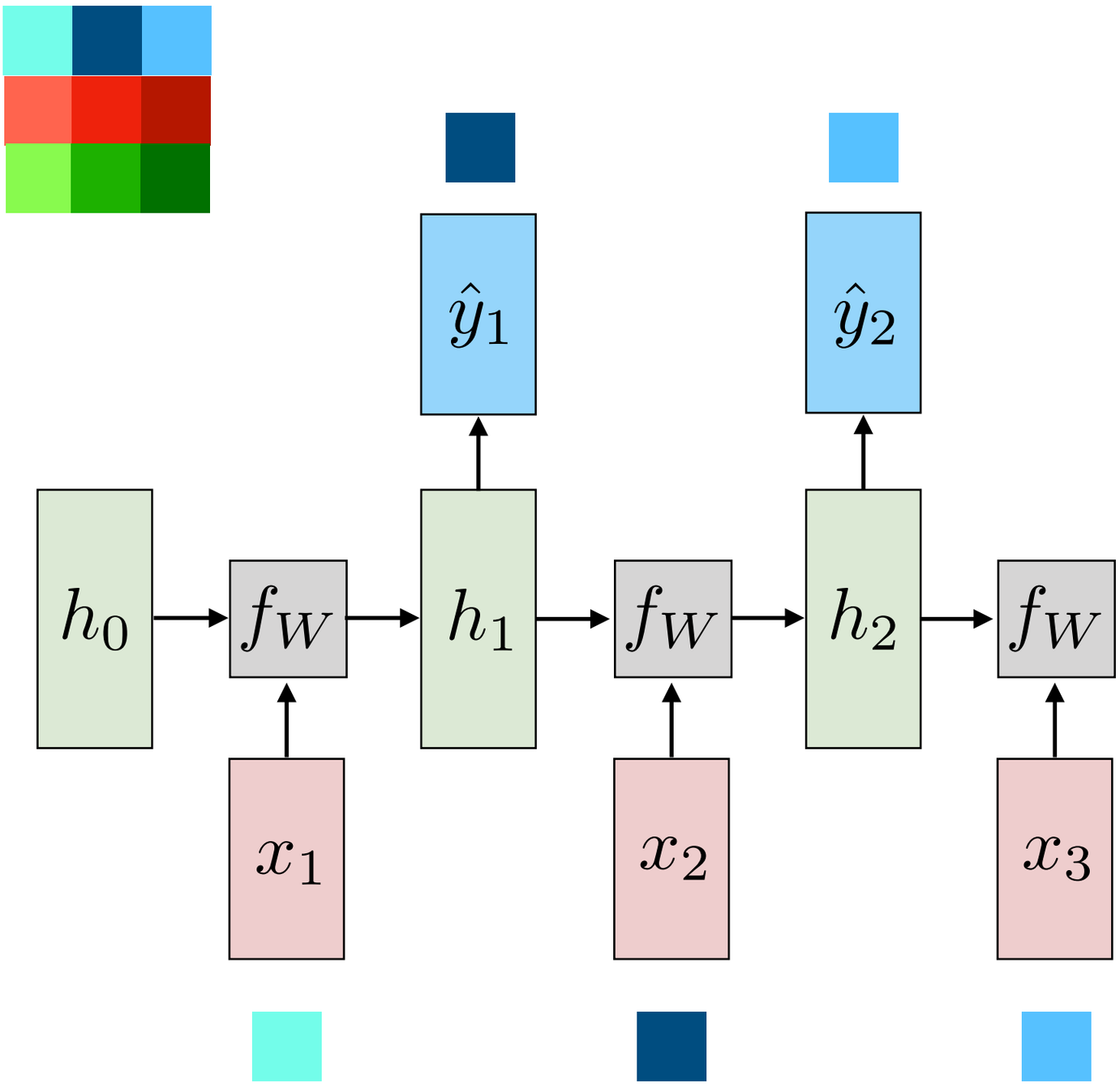
Recurrent Neural Language Model (Training)



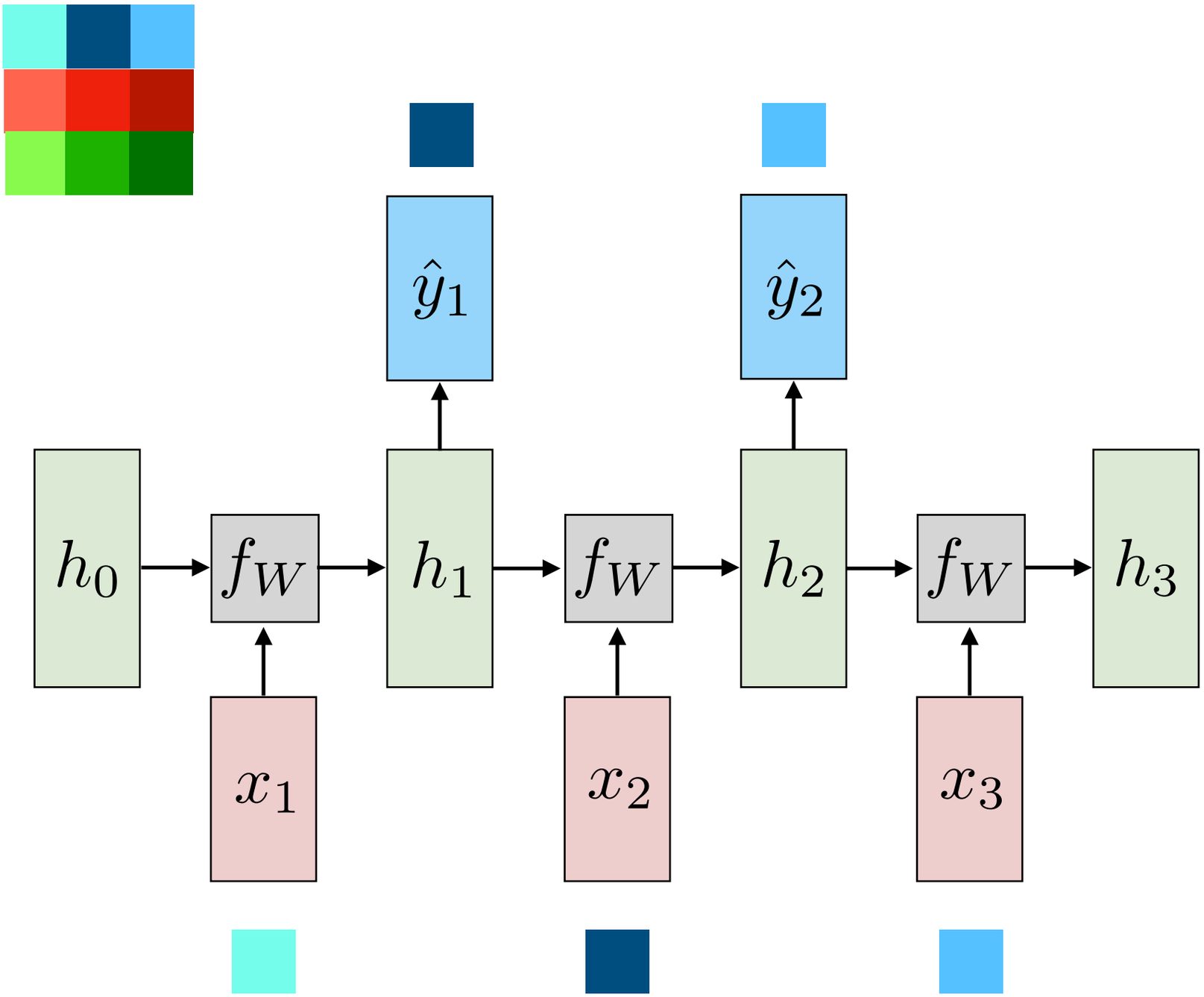
Recurrent Neural Language Model (Training)



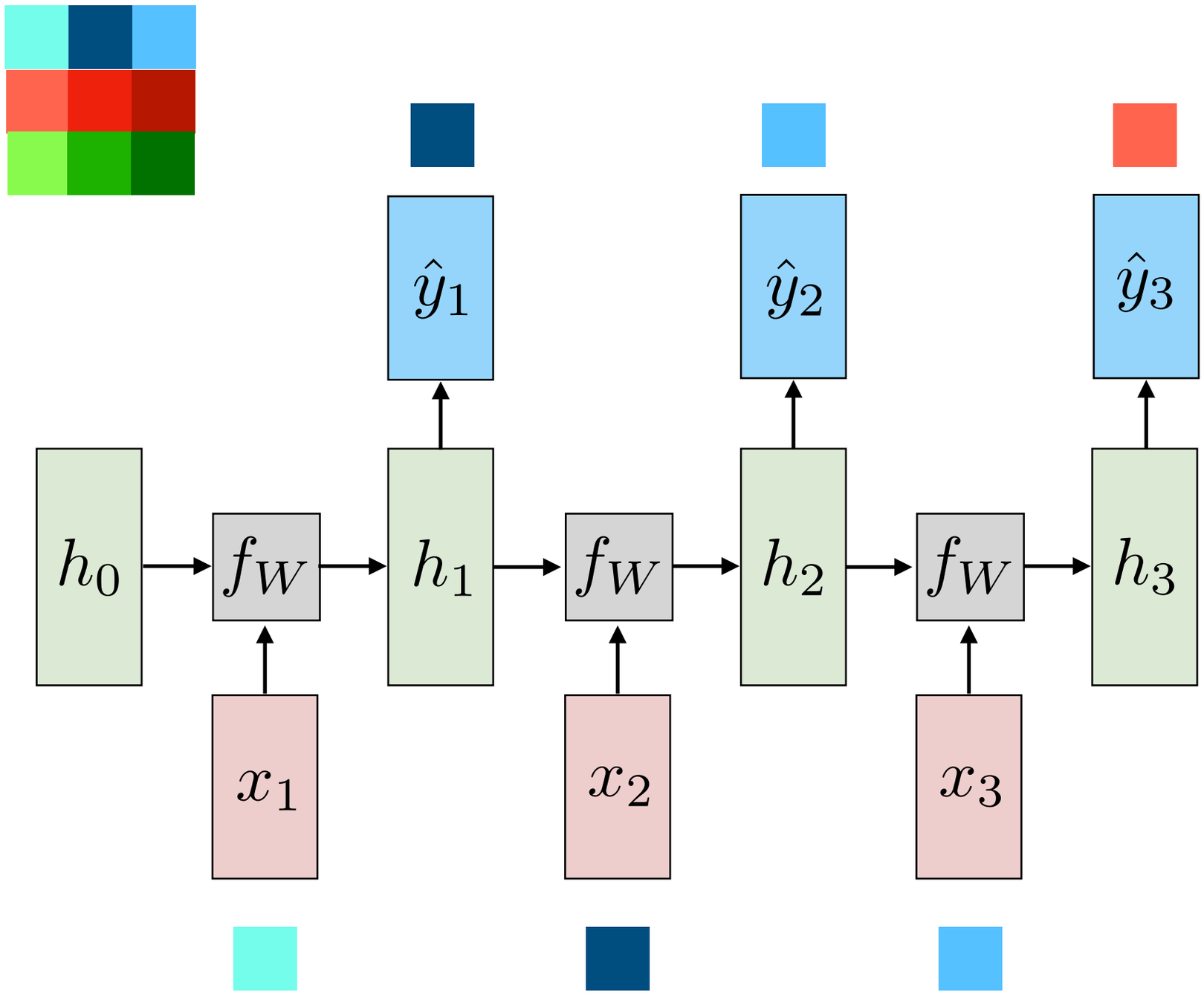
Recurrent Neural Language Model (Training)



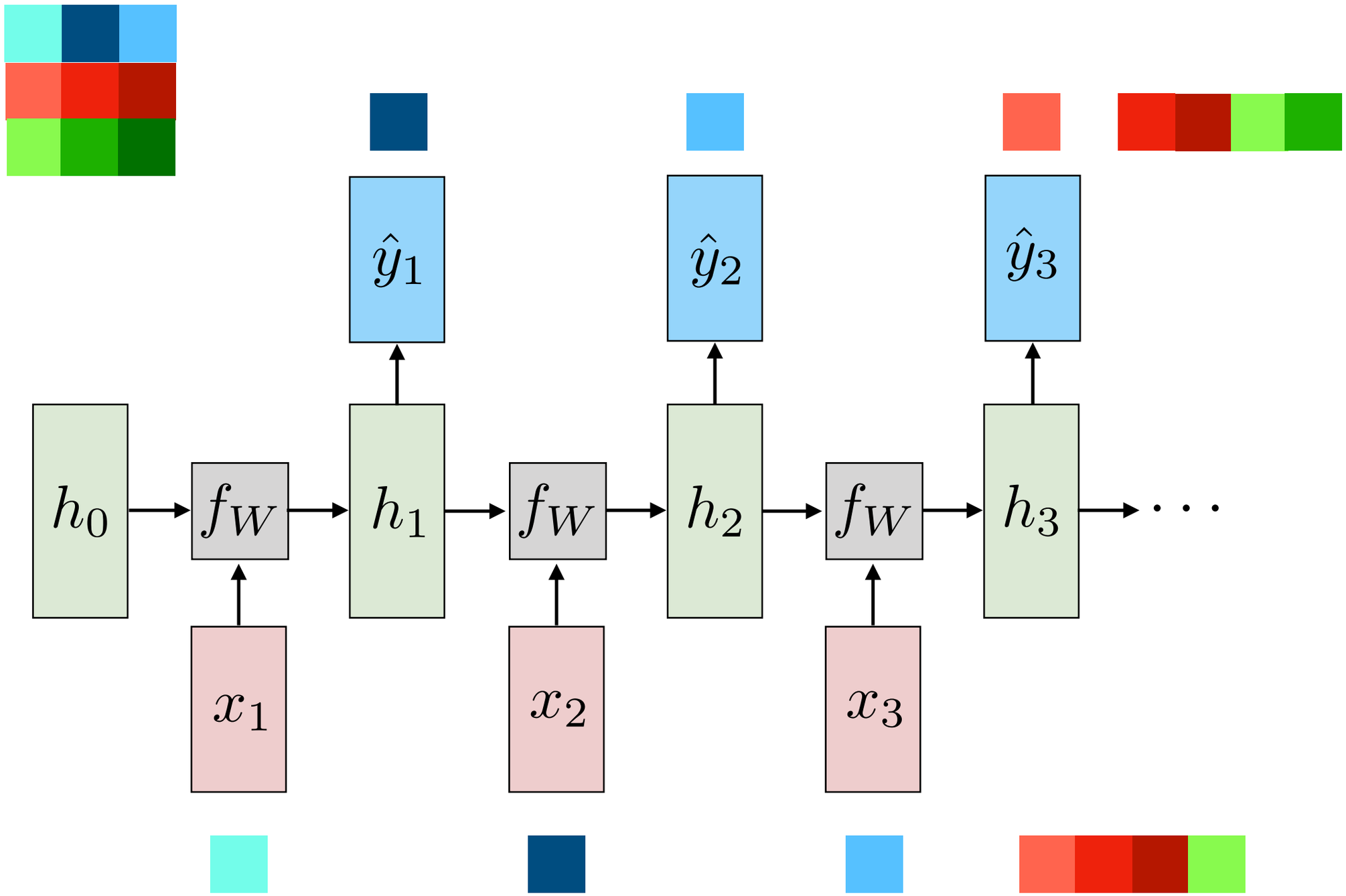
Recurrent Neural Language Model (Training)



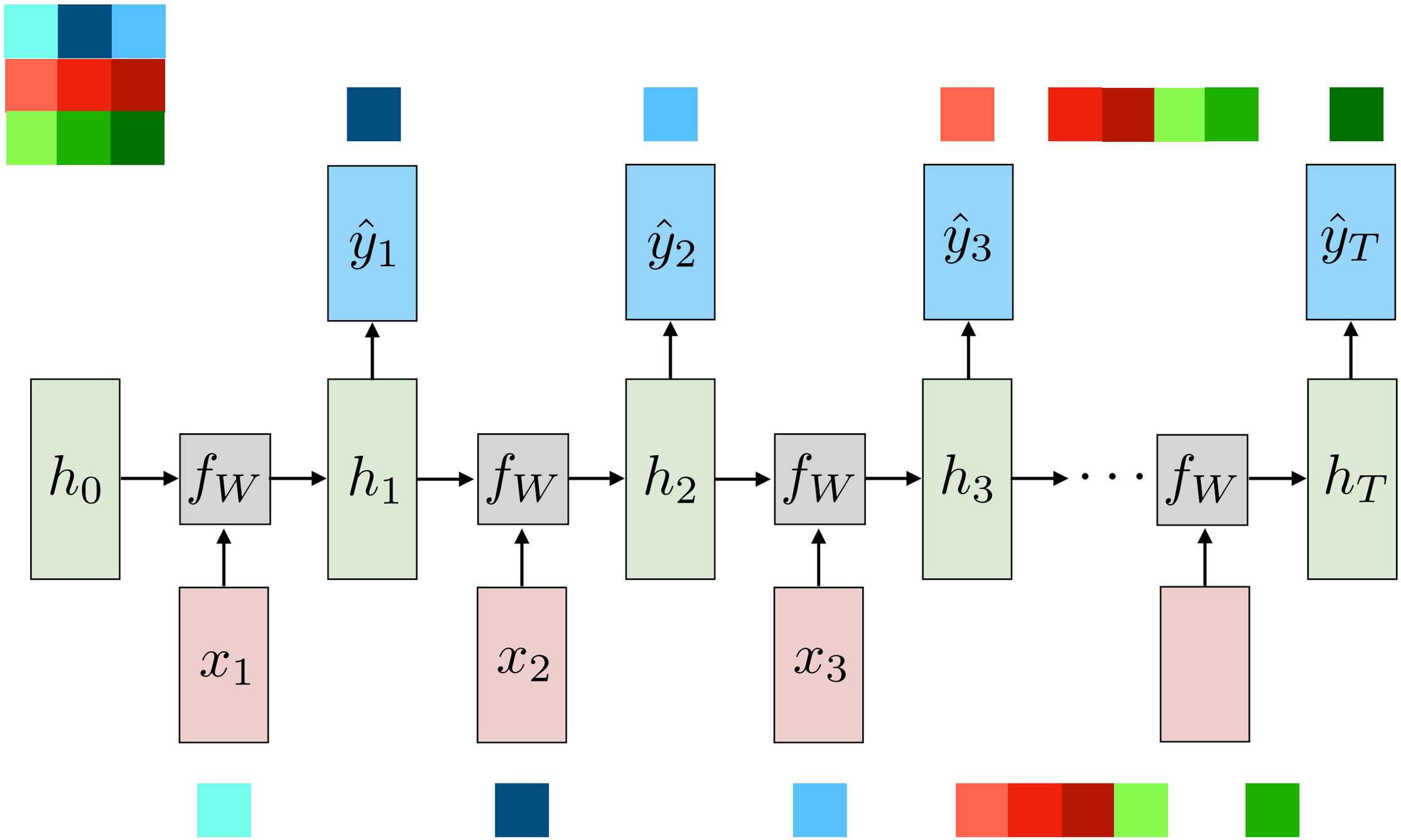
Recurrent Neural Language Model (Training)



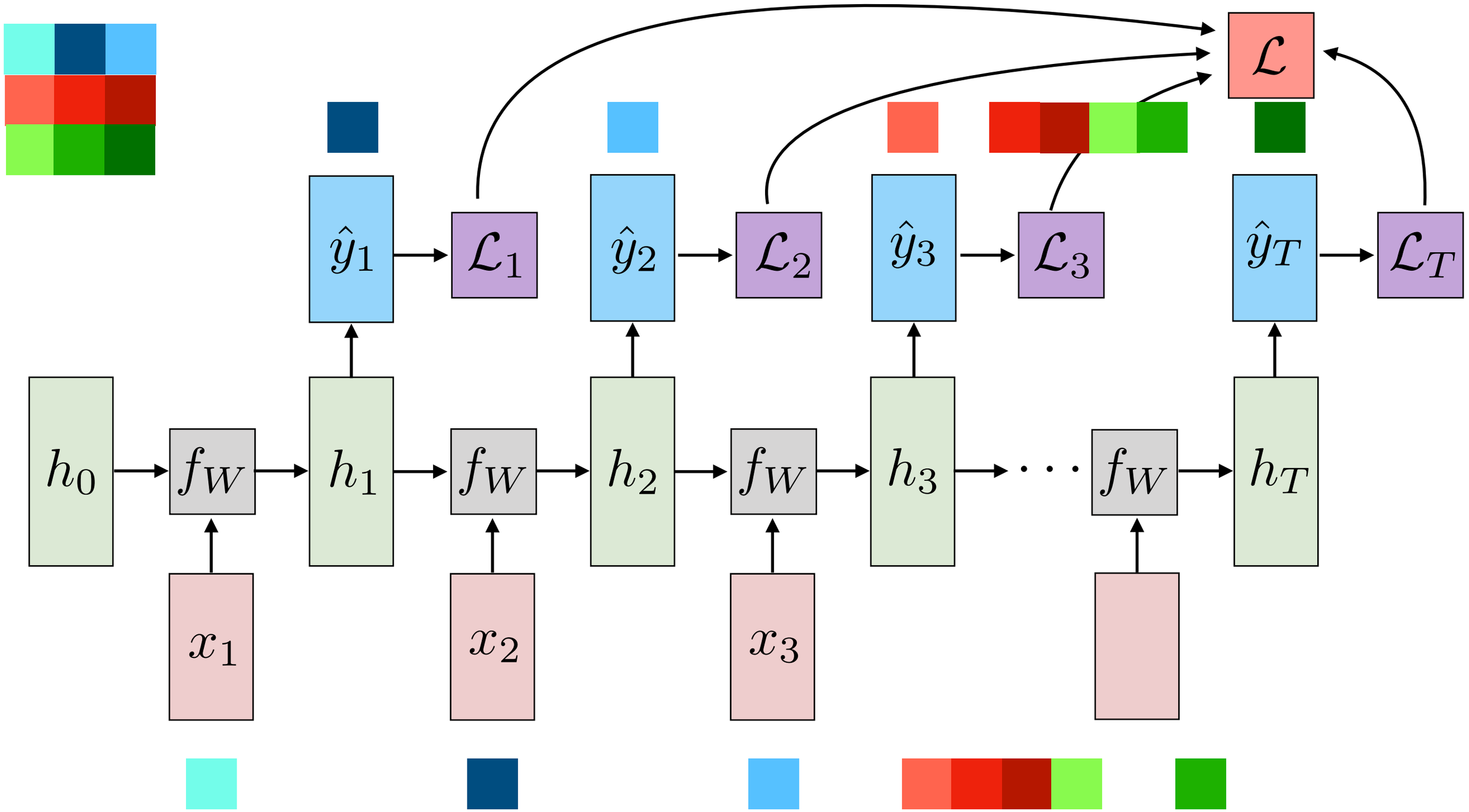
Recurrent Neural Language Model (Training)



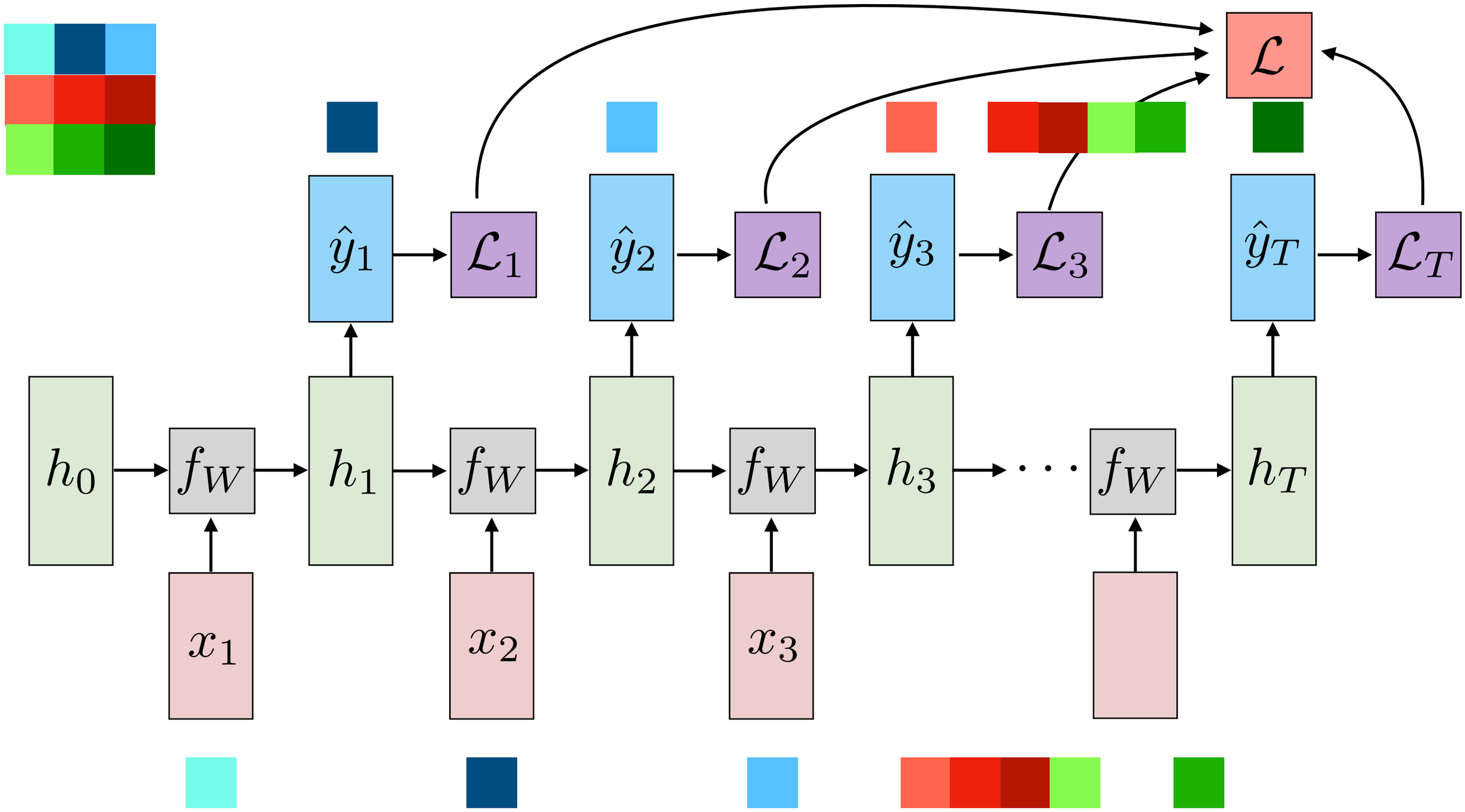
Recurrent Neural Language Model (Training)



Recurrent Neural Language Model (Training)



Recurrent Neural Language Model (Training)

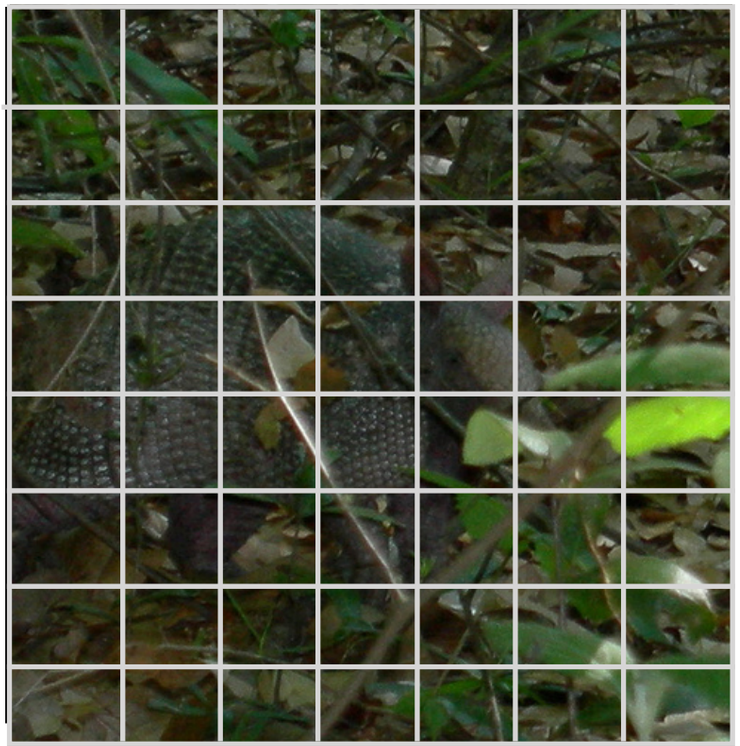


$$p(x_t) = \text{softmax}(\hat{y}_{t-1})$$

Maximum Likelihood Loss

Pixel RNN with row LSTM

$$p(\mathbf{x}) = \prod_{i=1}^{n^2} p\left(x_i | x_1, \ldots, x_{i-1}\right)$$



- Sequential model

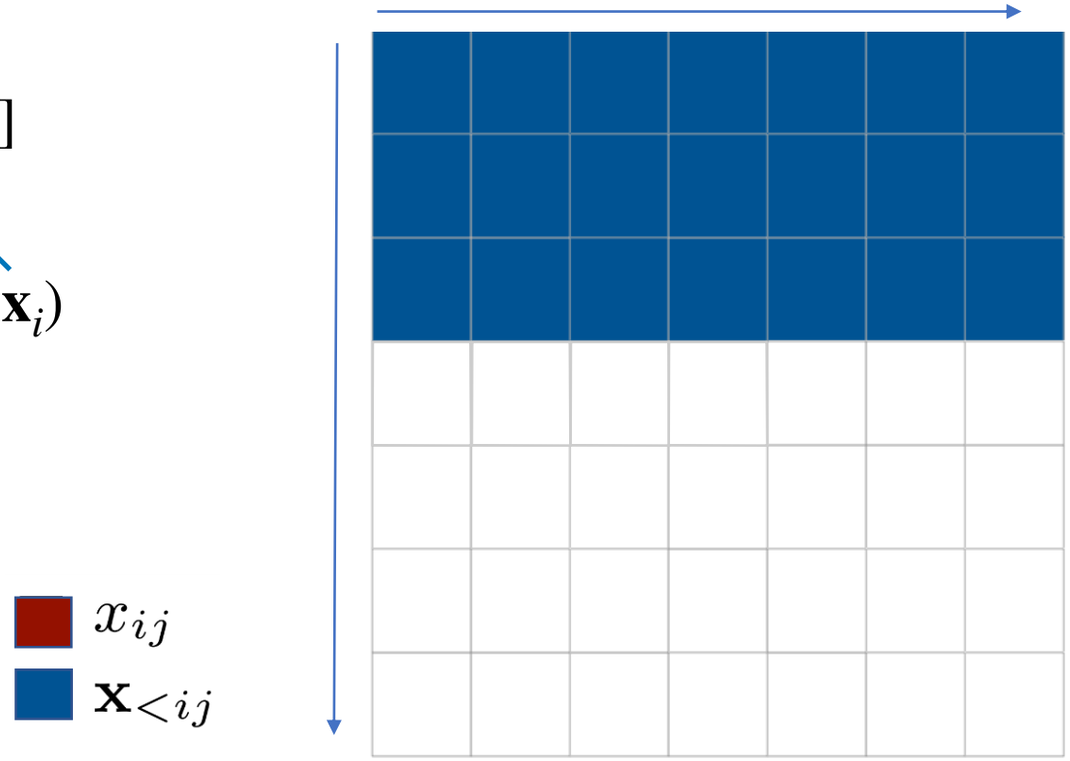
- Spatial Correlation ⟶ Spatial LSTMs

- Each row represents one state

$$[\mathbf{o}_i, \mathbf{f}_i, \mathbf{i}_i, \mathbf{g}_i] = \sigma(\mathbf{K}^{ss} \circledast \mathbf{h}_{i-1} + \mathbf{K}^{is} \circledast \mathbf{x}_i)$$

$[h \times n \times 1]$

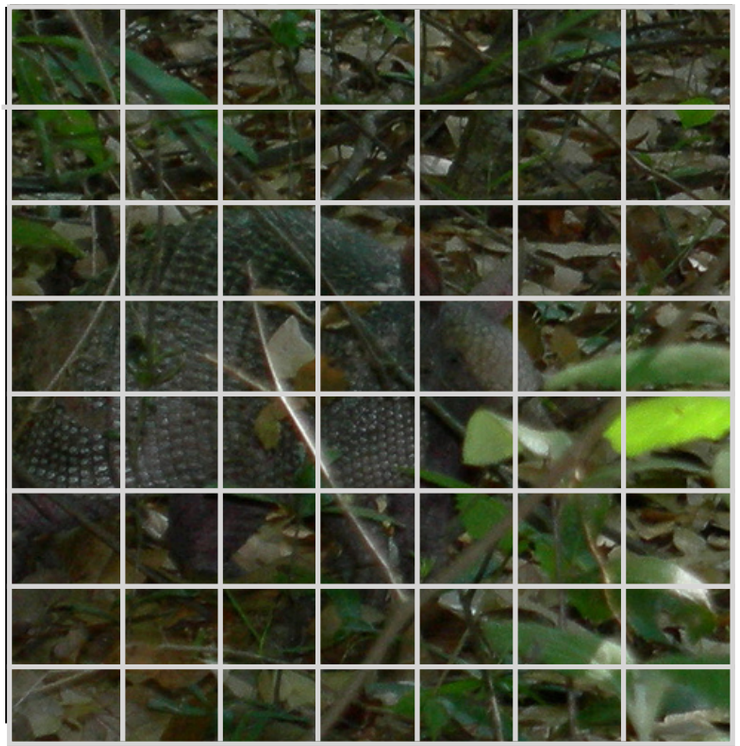
$[4h \times n \times n]$



Van den Oord et al.,ICML2016
"Conditional image generation with PixelCNNdecoders"

Pixel RNN with row LSTM

$$p(\mathbf{x}) = \prod_{i=1}^{n^2} p\left(x_i | x_1, \ldots, x_{i-1}\right)$$



- Sequential model

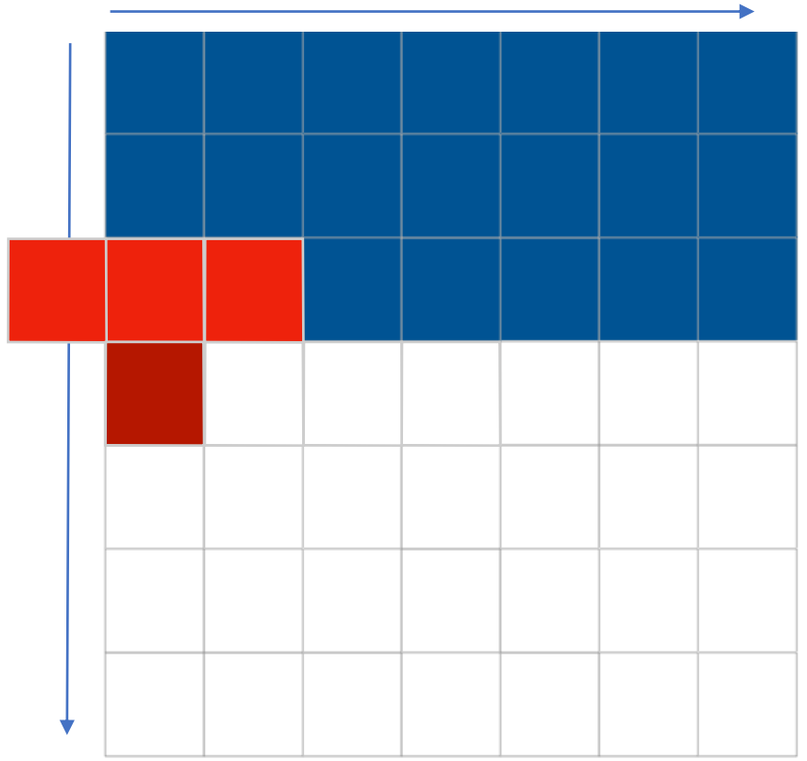
- Spatial Correlation ⟶ Spatial LSTMs

- Each row represents one state

$$[\mathbf{o}_i, \mathbf{f}_i, \mathbf{i}_i, \mathbf{g}_i] = \sigma(\mathbf{K}^{ss} \circledast \mathbf{h}_{i-1} + \mathbf{K}^{is} \circledast \mathbf{x}_i)$$

$[h \times n \times 1]$

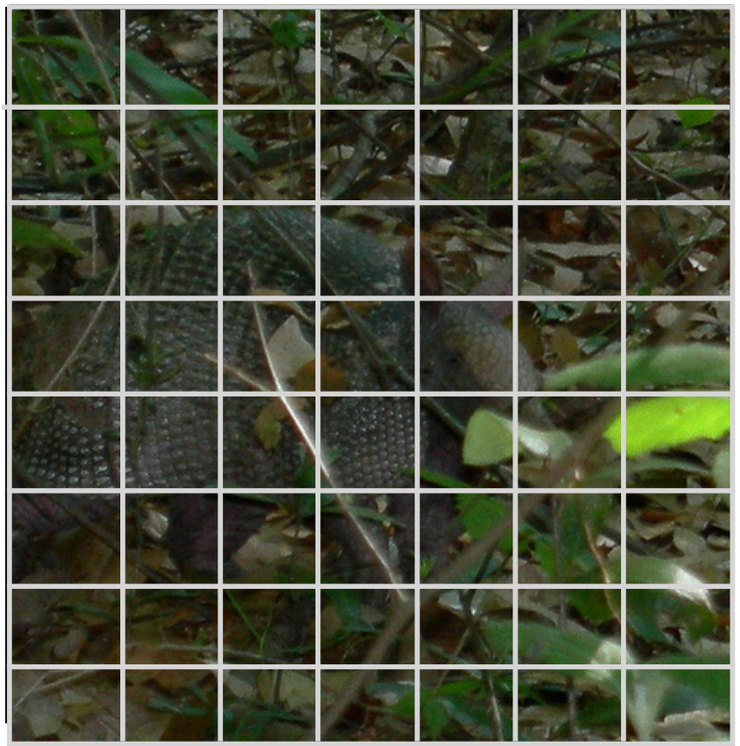
$[4h \times n \times n]$



Van den Oord et al.,ICML2016
"Conditional image generation with PixelCNNdecoders"

Pixel RNN with row LSTM

$$p(\mathbf{x}) = \prod_{i=1}^{n^2} p\left(x_i | x_1, \ldots, x_{i-1}\right)$$



- Sequential model

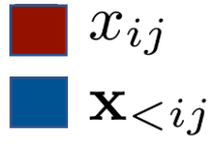
- Spatial Correlation ⟶ Spatial LSTMs

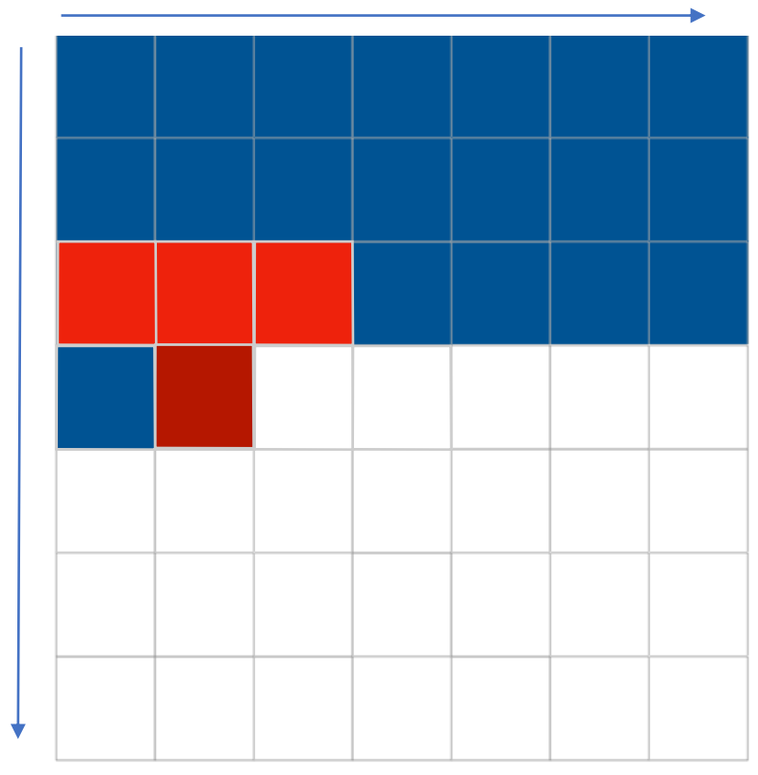
- Each row represents one state

$$[\mathbf{o}_i, \mathbf{f}_i, \mathbf{i}_i, \mathbf{g}_i] = \sigma(\mathbf{K}^{ss} \circledast \mathbf{h}_{i-1} + \mathbf{K}^{is} \circledast \mathbf{x}_i)$$

$[h \times n \times 1]$

$[4h \times n \times n]$

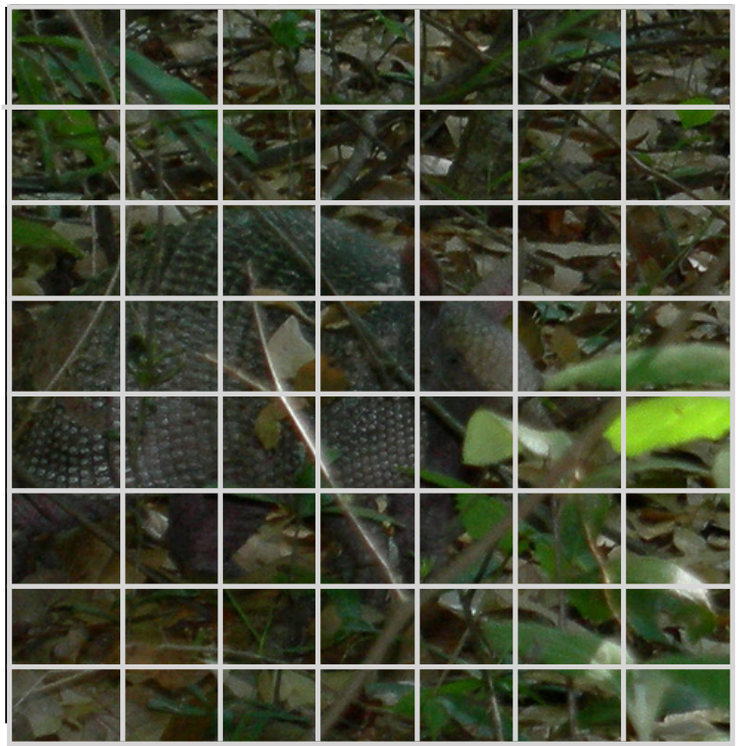




Van den Oord et al.,ICML2016
"Conditional image generation with PixelCNNdecoders"

Pixel RNN with row LSTM

$$p(\mathbf{x}) = \prod_{i=1}^{n^2} p\left(x_i | x_1, \ldots, x_{i-1}\right)$$



- Sequential model

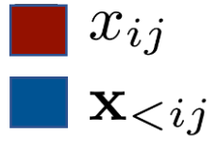
- Spatial Correlation ⟶ Spatial LSTMs

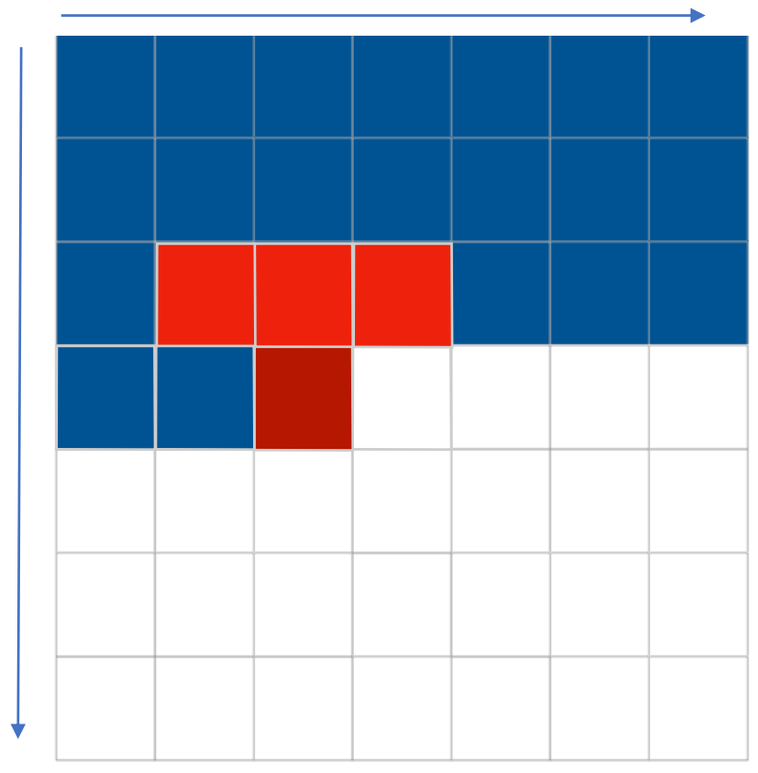
- Each row represents one state

$$[\mathbf{o}_i, \mathbf{f}_i, \mathbf{i}_i, \mathbf{g}_i] = \sigma(\mathbf{K}^{ss} \circledast \mathbf{h}_{i-1} + \mathbf{K}^{is} \circledast \mathbf{x}_i)$$

$[h \times n \times 1]$

$[4h \times n \times n]$

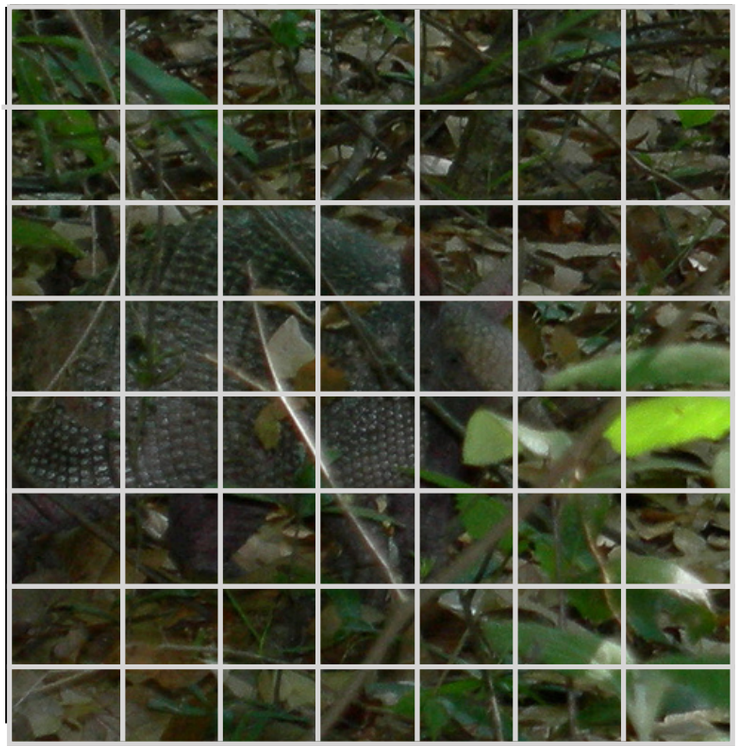




Van den Oord et al.,ICML2016
"Conditional image generation with PixelCNNdecoders"

Pixel RNN with row LSTM

$$p(\mathbf{x}) = \prod_{i=1}^{n^2} p\left(x_i | x_1, \ldots, x_{i-1}\right)$$



- Sequential model

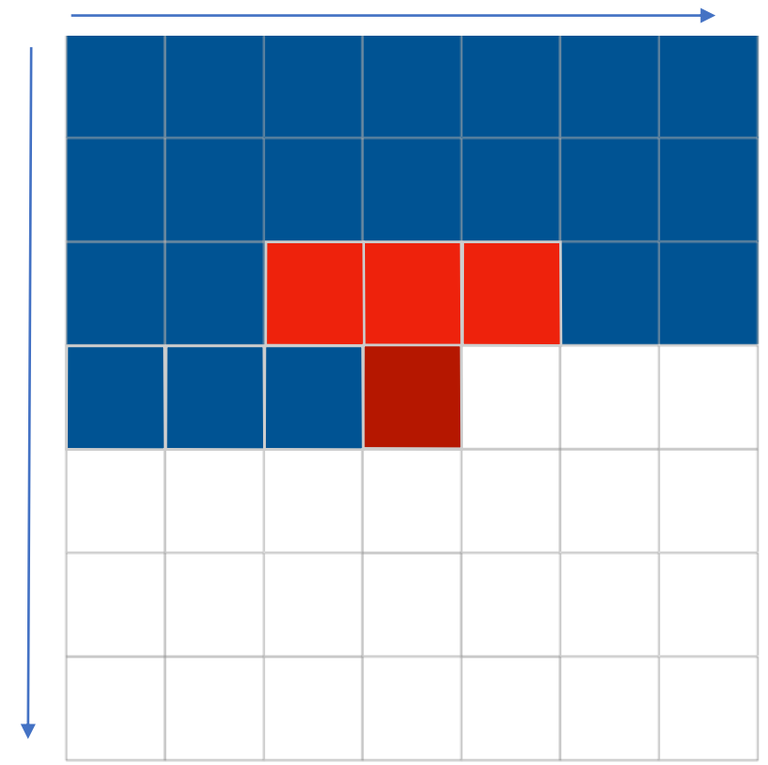
- Spatial Correlation ⟶ Spatial LSTMs

- Each row represents one state

$$[\mathbf{o}_i, \mathbf{f}_i, \mathbf{i}_i, \mathbf{g}_i] = \sigma(\mathbf{K}^{ss} \circledast \mathbf{h}_{i-1} + \mathbf{K}^{is} \circledast \mathbf{x}_i)$$

$[h \times n \times 1]$

$[4h \times n \times n]$



Van den Oord et al.,ICML2016
"Conditional image generation with PixelCNNdecoders"

Pixel RNN with row LSTM

$$p(\mathbf{x}) = \prod_{i=1}^{n^2} p\left(x_i | x_1, \ldots, x_{i-1}\right)$$

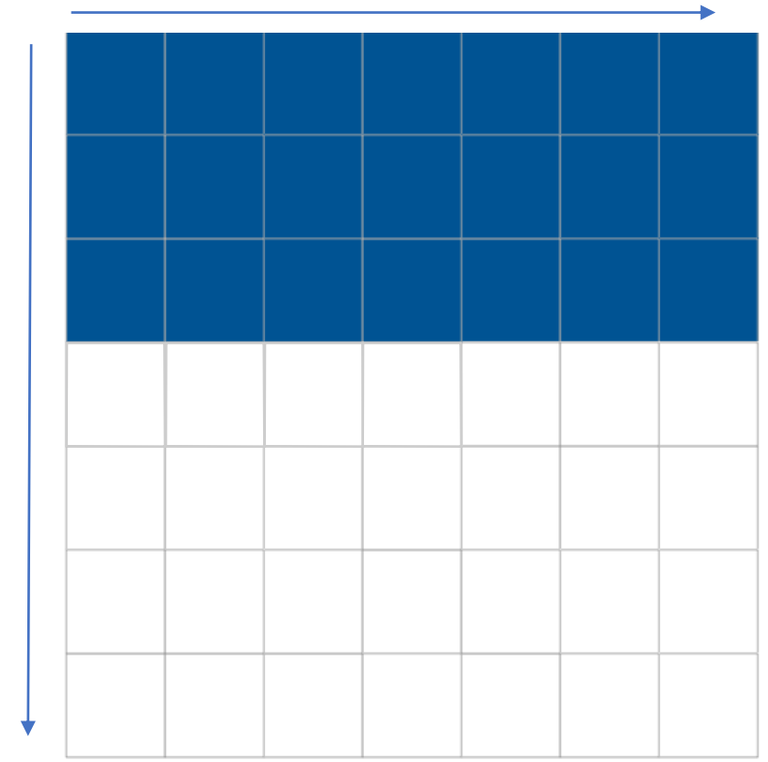
masked convolution

- Sequential model
- Spatial Correlation ⟶ Spatial LSTMs
- Each row represents one state

$$[\mathbf{o}_i, \mathbf{f}_i, \mathbf{i}_i, \mathbf{g}_i] = \sigma(\mathbf{K}^{ss} \circledast \mathbf{h}_{i-1} + \mathbf{K}^{is} \circledast \mathbf{x}_i)$$

$[h \times n \times 1]$

$[4h \times n \times n]$



Van den Oord et al.,ICML2016
"Conditional image generation with PixelCNNdecoders"

Pixel RNN with row LSTM

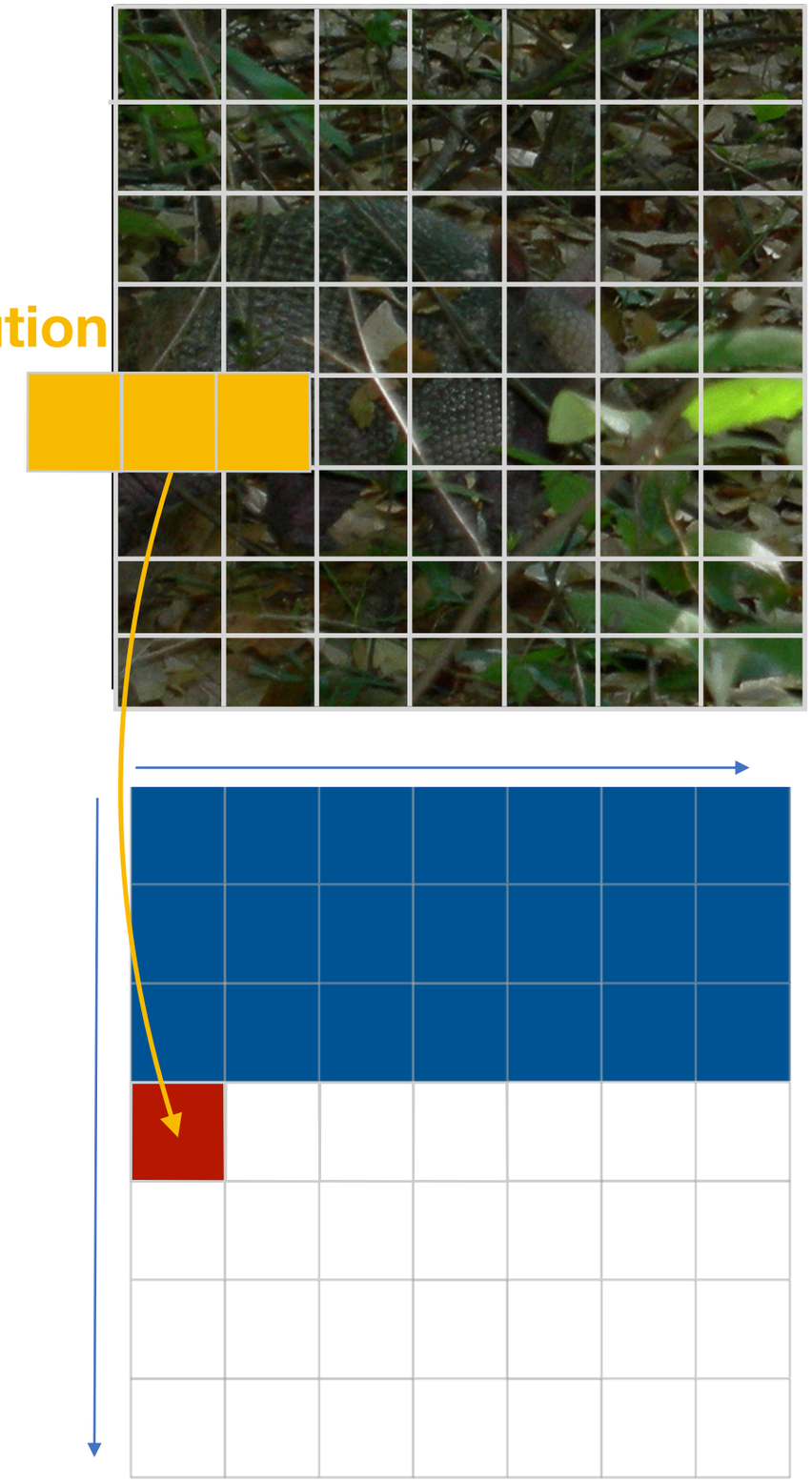
$$p(\mathbf{x}) = \prod_{i=1}^{n^2} p\left(x_i | x_1, \ldots, x_{i-1}\right)$$

- Sequential model
- Spatial Correlation ⟶ Spatial LSTMs
- Each row represents one state

$$[\mathbf{o}_i, \mathbf{f}_i, \mathbf{i}_i, \mathbf{g}_i] = \sigma(\mathbf{K}^{ss} \circledast \mathbf{h}_{i-1} + \mathbf{K}^{is} \circledast \mathbf{x}_i)$$

$[h \times n \times 1]$

$[4h \times n \times n]$



Van den Oord et al.,ICML2016
"Conditional image generation with PixelCNNdecoders"

Pixel RNN with row LSTM

$$p(\mathbf{x}) = \prod_{i=1}^{n^2} p\left(x_i | x_1, \ldots, x_{i-1}\right)$$

- Sequential model

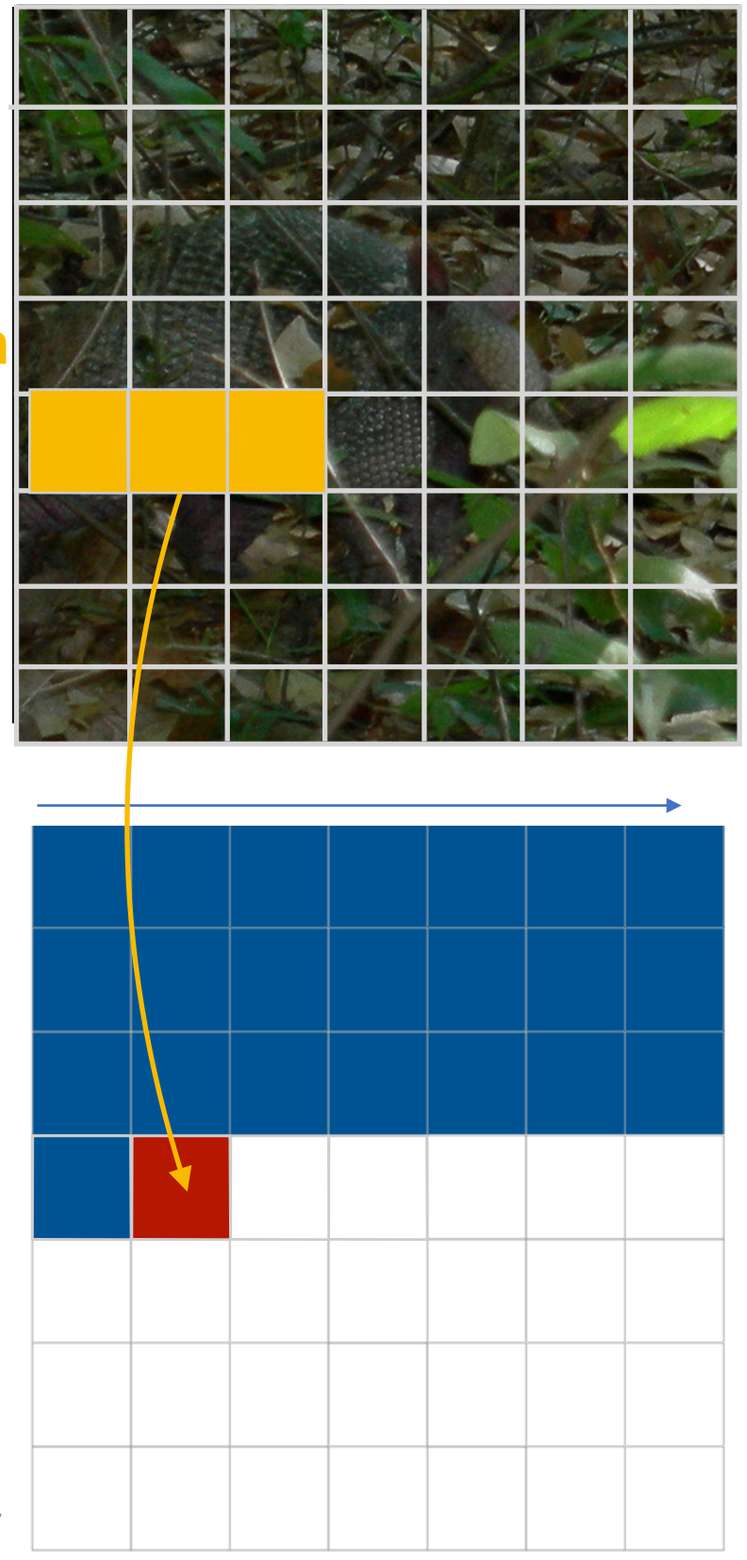
- Spatial Correlation ⟶ Spatial LSTMs

- Each row represents one state

$$[\mathbf{o}_i, \mathbf{f}_i, \mathbf{i}_i, \mathbf{g}_i] = \sigma(\mathbf{K}^{ss} \circledast \mathbf{h}_{i-1} + \mathbf{K}^{is} \circledast \mathbf{x}_i)$$

$[h \times n \times 1]$

$[4h \times n \times n]$



Van den Oord et al.,ICML2016
"Conditional image generation with PixelCNNdecoders"

Pixel RNN with row LSTM

$$p(\mathbf{x}) = \prod_{i=1}^{n^2} p\left(x_i | x_1, \ldots, x_{i-1}\right)$$

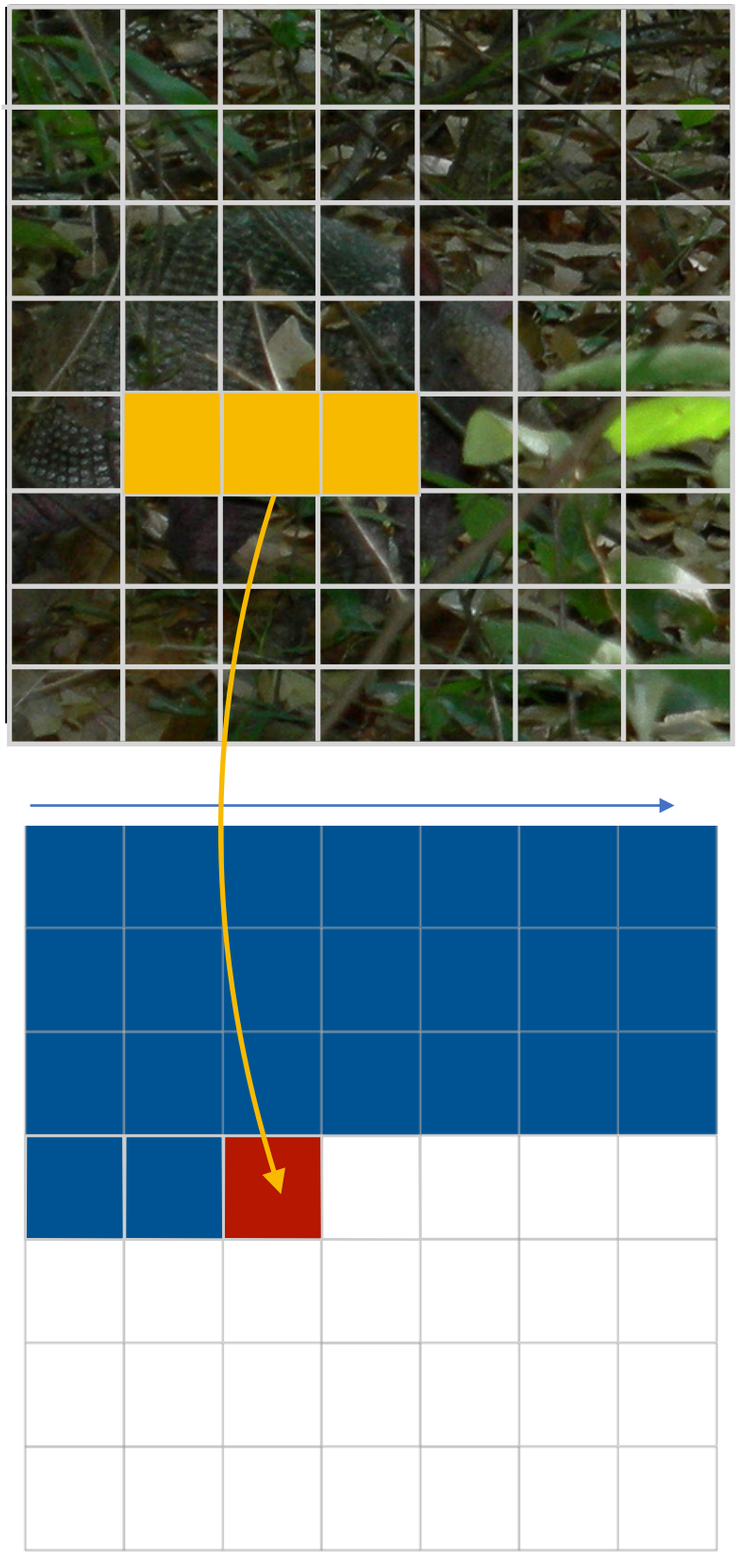
- Sequential model
- Spatial Correlation ⟶ Spatial LSTMs
- Each row represents one state

$$[\mathbf{o}_i, \mathbf{f}_i, \mathbf{i}_i, \mathbf{g}_i] = \sigma(\mathbf{K}^{ss} \circledast \mathbf{h}_{i-1} + \mathbf{K}^{is} \circledast \mathbf{x}_i)$$

$[h \times n \times 1]$

$[4h \times n \times n]$

masked convolution



Van den Oord et al.,ICML2016
"Conditional image generation with PixelCNNdecoders"

Pixel RNN with row LSTM

$$p(\mathbf{x}) = \prod_{i=1}^{n^2} p\left(x_i | x_1, \ldots, x_{i-1}\right)$$

- Sequential model

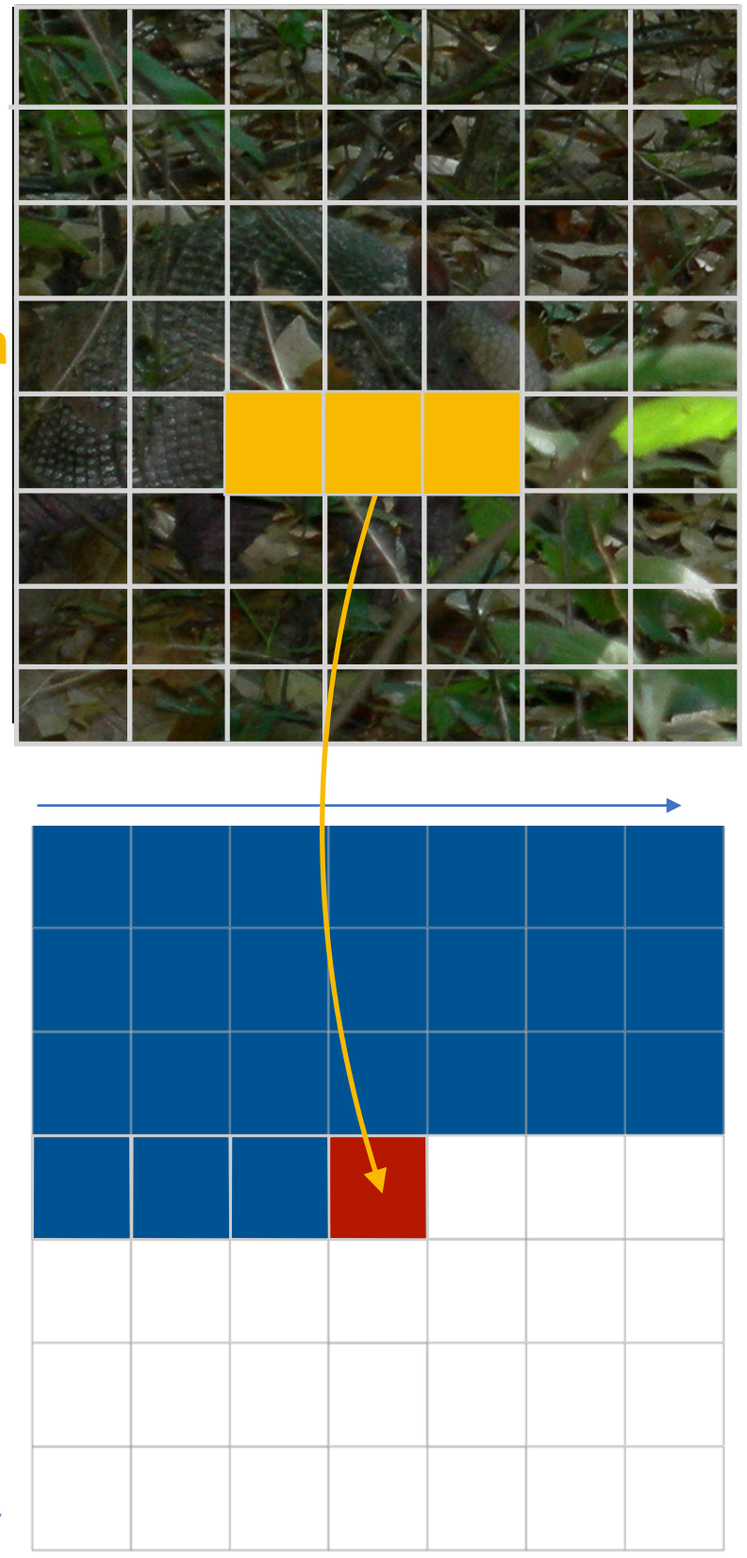
- Spatial Correlation ⟶ Spatial LSTMs

- Each row represents one state

$$[\mathbf{o}_i, \mathbf{f}_i, \mathbf{i}_i, \mathbf{g}_i] = \sigma(\mathbf{K}^{ss} \circledast \mathbf{h}_{i-1} + \mathbf{K}^{is} \circledast \mathbf{x}_i)$$

$[h \times n \times 1]$

$[4h \times n \times n]$

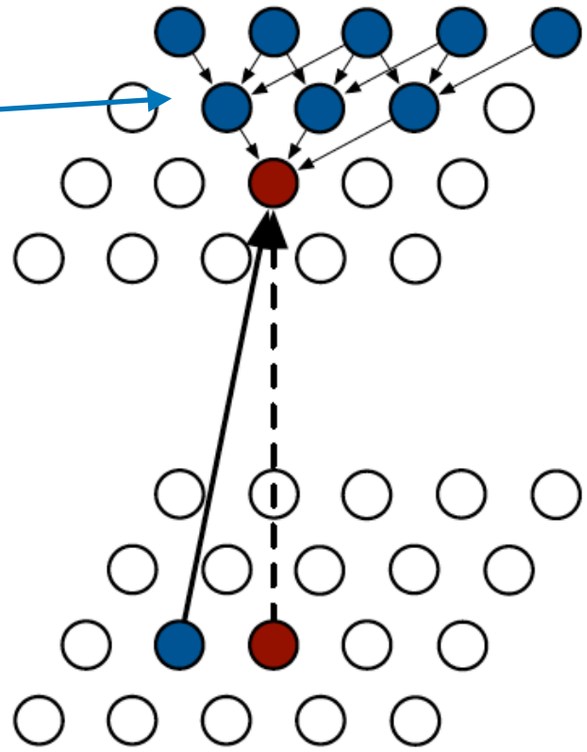


[Van den Oord et al.,ICML2016
"Conditional image generation with PixelCNNdecoders"]

Pixel RNN: pros and cons of row LSTM

- **Compute state for entire row**

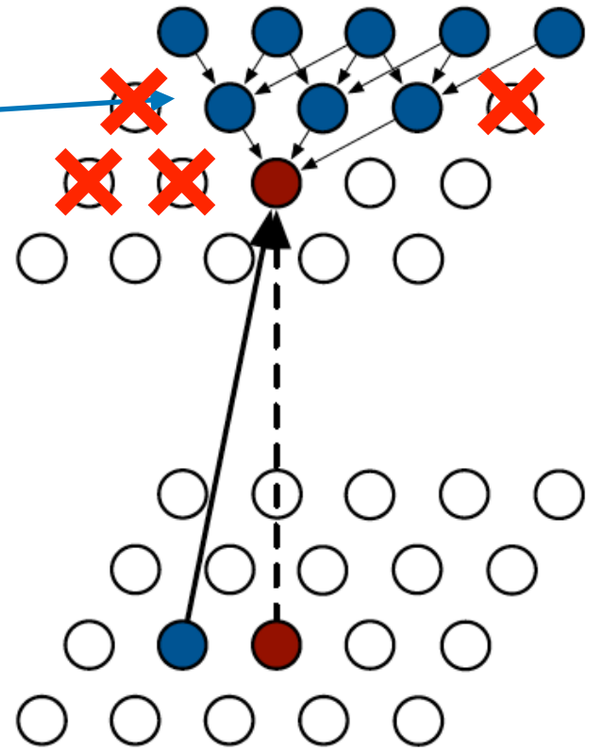
- Triangular receptive field



Van den Oord et al.,IMCL2016
"Conditional image generation with PixelCNNdecoders"

Pixel RNN: pros and cons of row LSTM

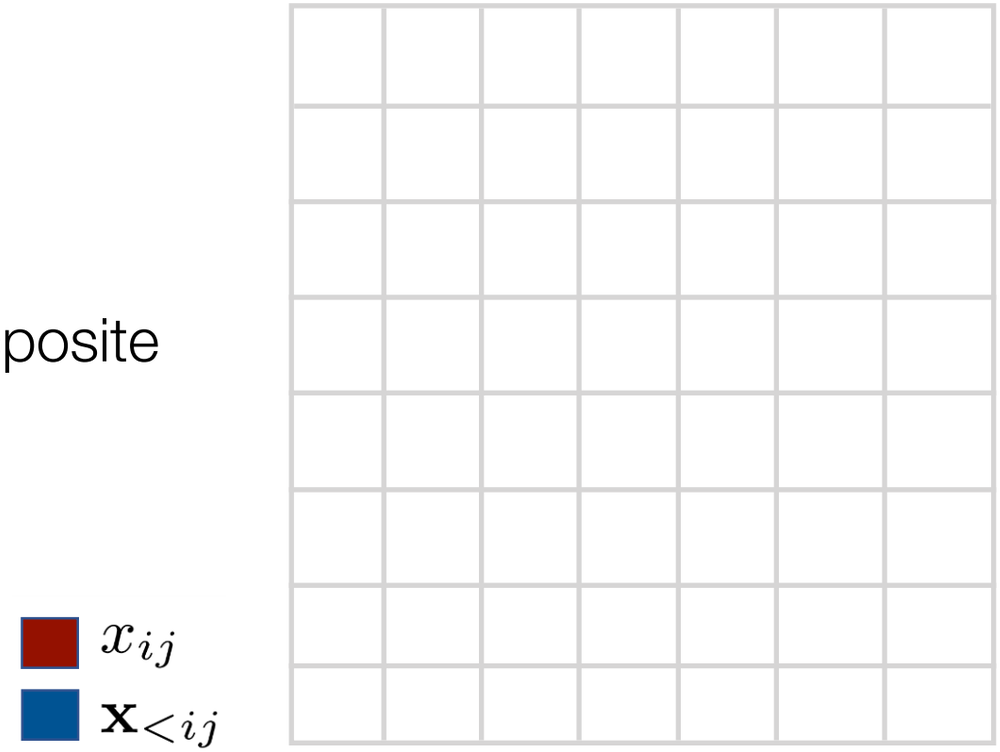
- **Compute state for entire row**
- Triangular receptive field
- **Does not use all available context**



Van den Oord et al.,IMCL2016
"Conditional image generation with PixelCNNdecoders"

Pixel RNN with diagonal biLSTM

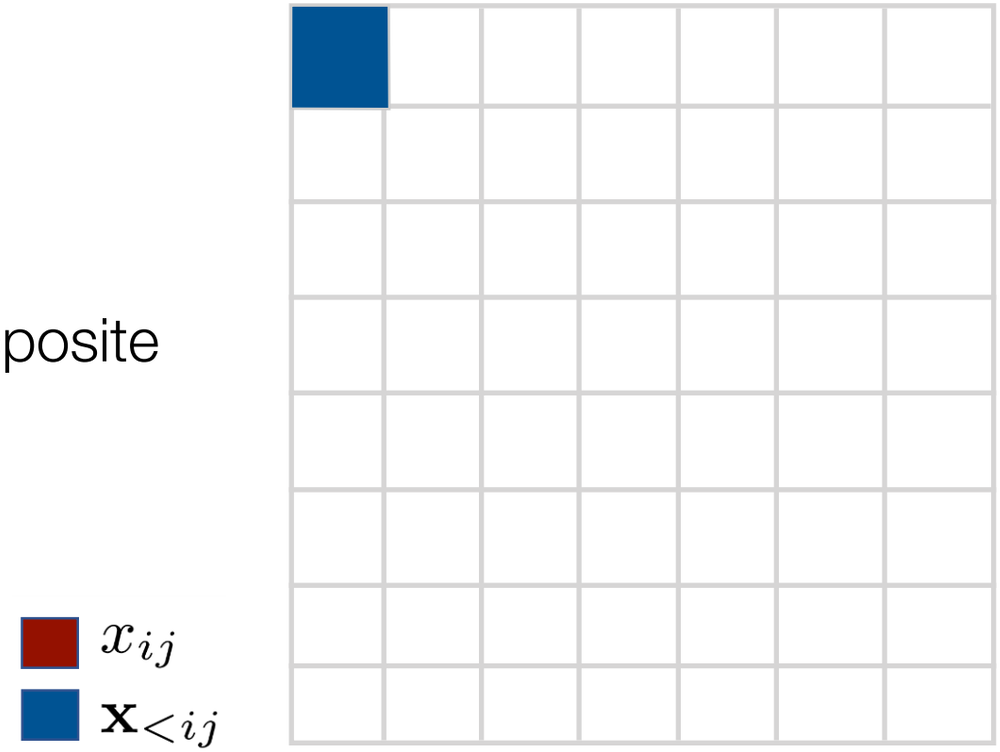
- Capture the entire available context
- Each diagonal scan from one corner to the opposite



Van den Oord et al.,ICML2016
"Conditional image generation with PixelCNNdecoders"

Pixel RNN with diagonal biLSTM

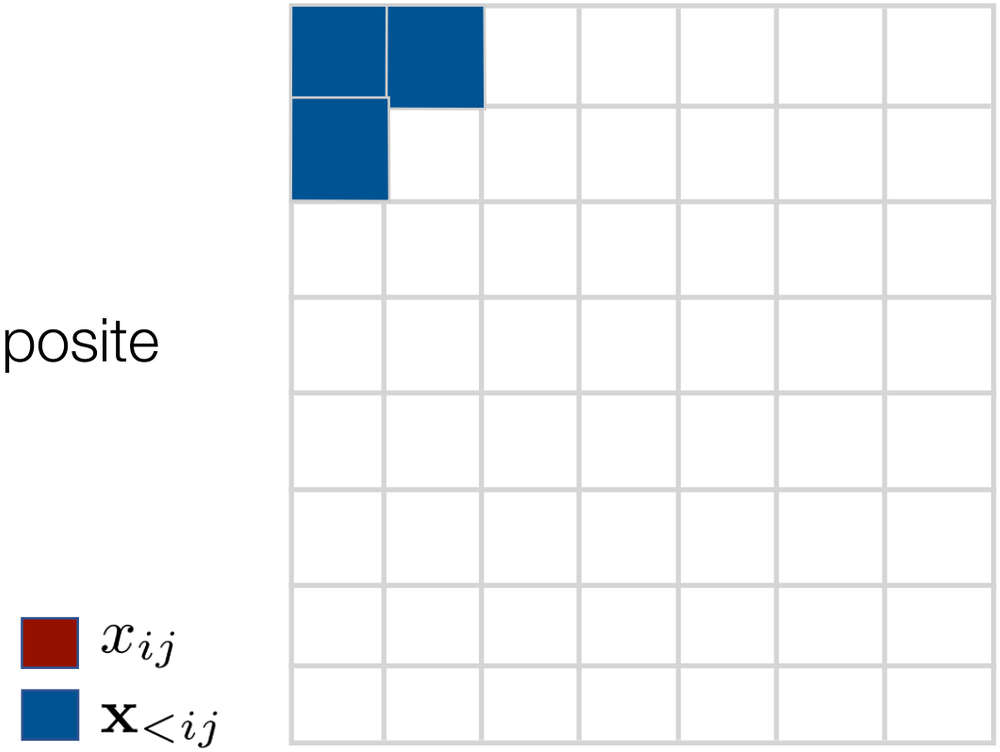
- Capture the entire available context
- Each diagonal scan from one corner to the opposite



Van den Oord et al.,ICML2016
"Conditional image generation with PixelCNNdecoders"

Pixel RNN with diagonal biLSTM

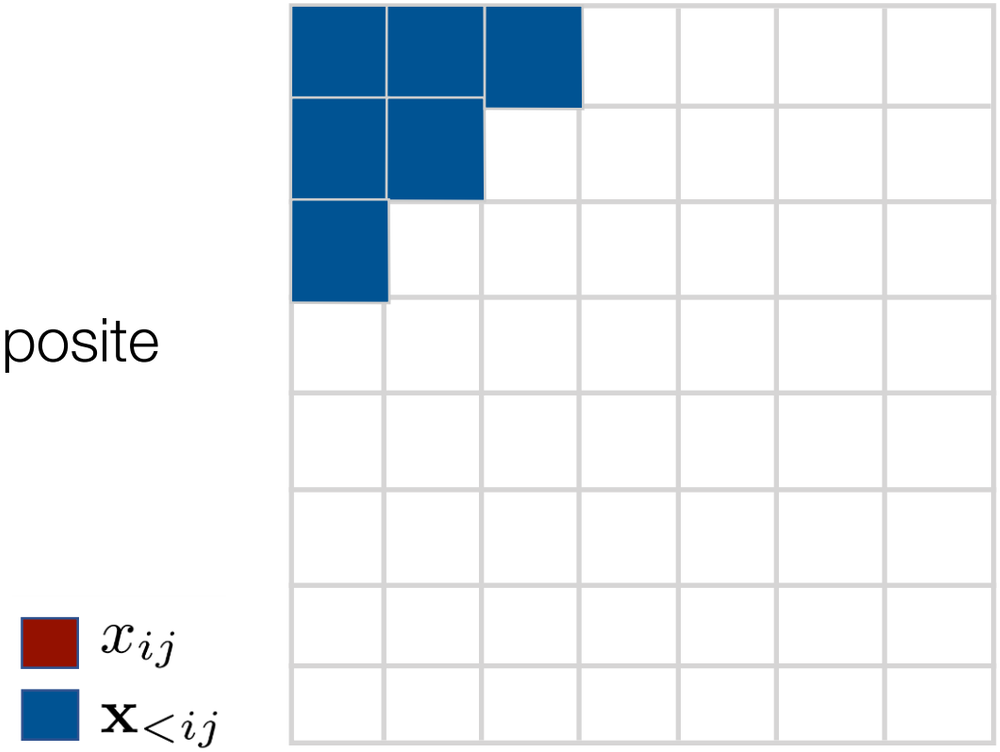
- Capture the entire available context
- Each diagonal scan from one corner to the opposite



Van den Oord et al.,ICML2016
"Conditional image generation with PixelCNNdecoders"

Pixel RNN with diagonal biLSTM

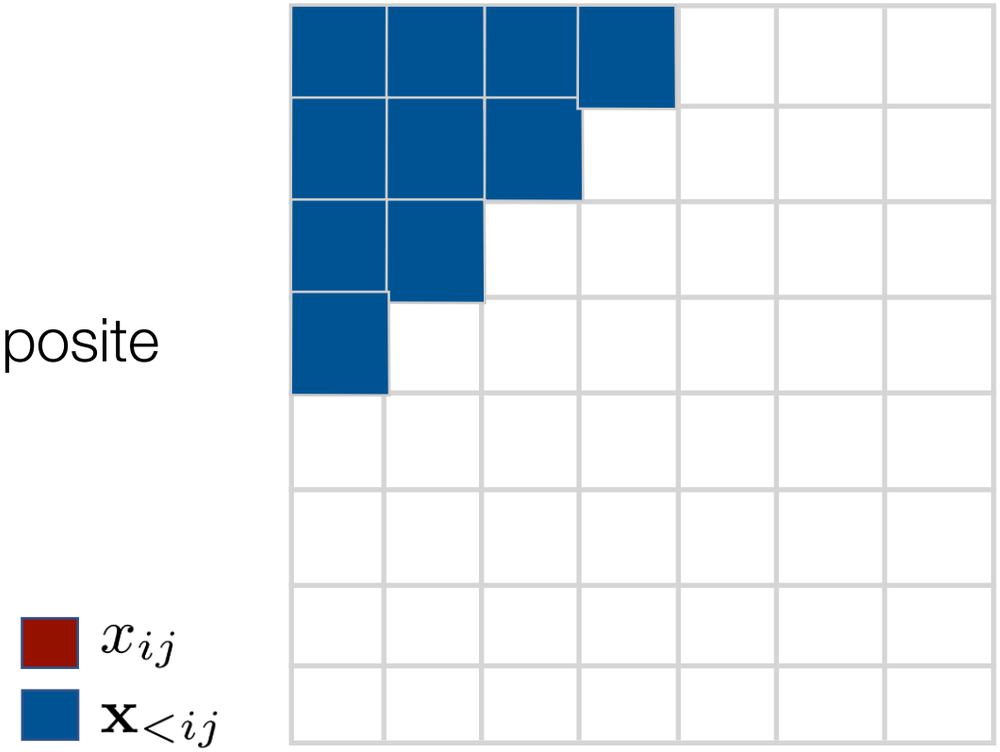
- Capture the entire available context
- Each diagonal scan from one corner to the opposite



Van den Oord et al.,ICML2016
"Conditional image generation with PixelCNNdecoders"

Pixel RNN with diagonal biLSTM

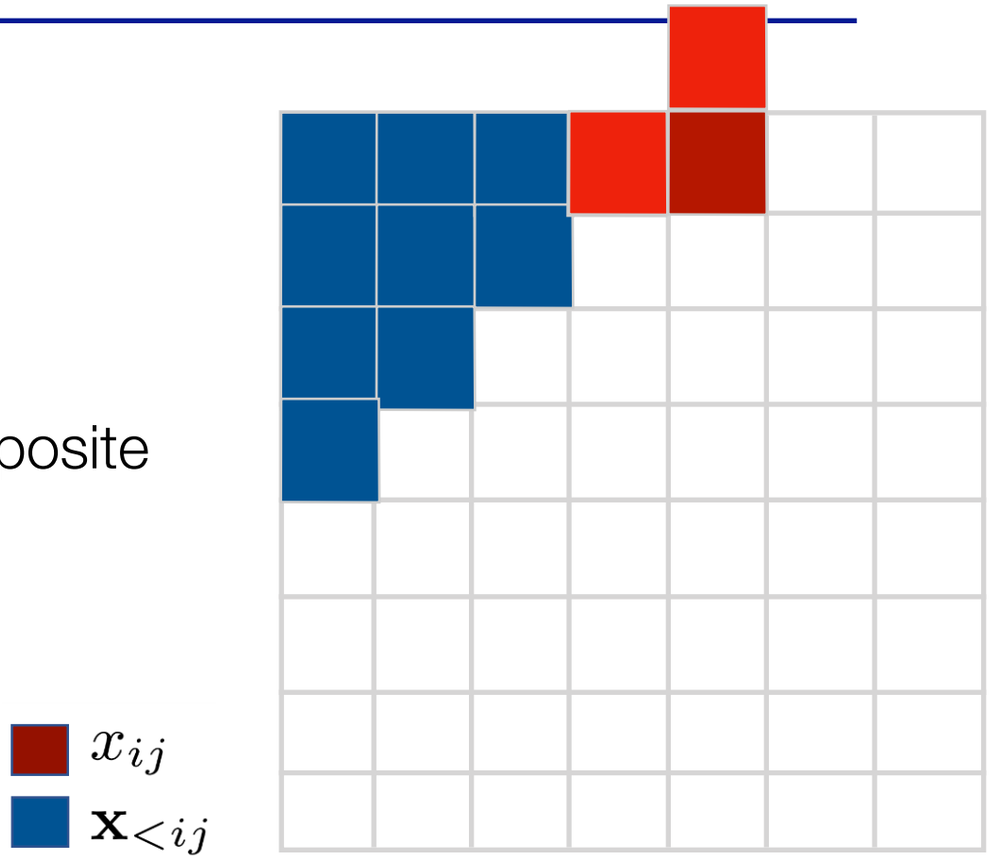
- Capture the entire available context
- Each diagonal scan from one corner to the opposite



Van den Oord et al.,ICML2016
"Conditional image generation with PixelCNNdecoders"

Pixel RNN with diagonal biLSTM

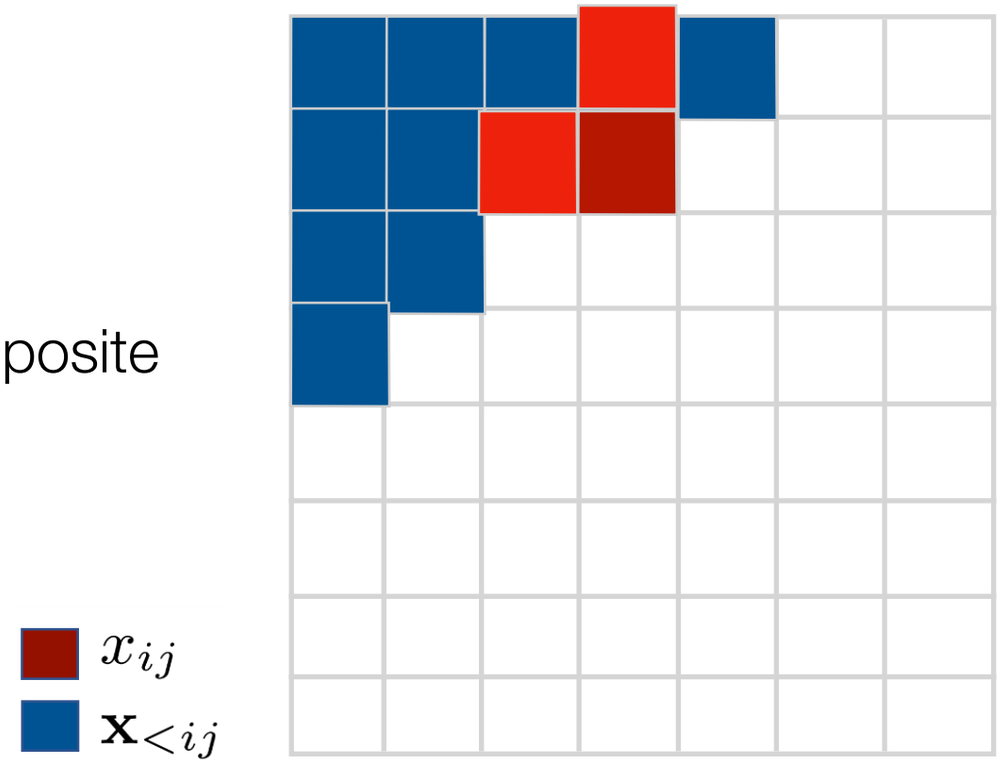
- Capture the entire available context
- Each diagonal scan from one corner to the opposite



Van den Oord et al.,ICML2016
"Conditional image generation with PixelCNNdecoders"

Pixel RNN with diagonal biLSTM

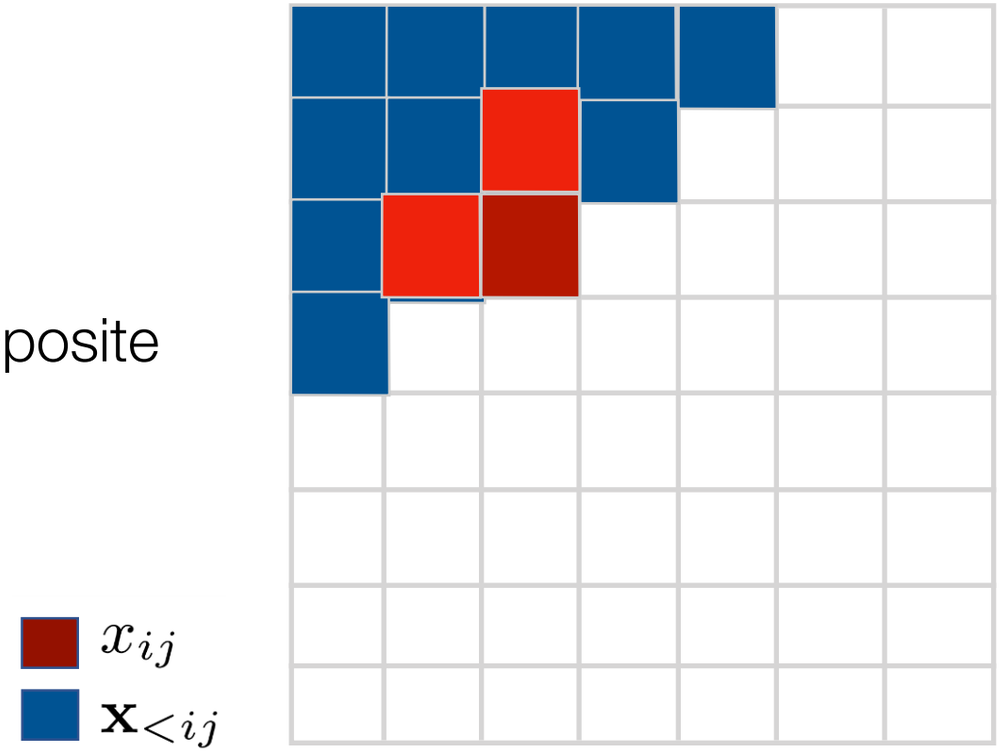
- Capture the entire available context
- Each diagonal scan from one corner to the opposite



Van den Oord et al.,ICML2016
"Conditional image generation with PixelCNNdecoders"

Pixel RNN with diagonal biLSTM

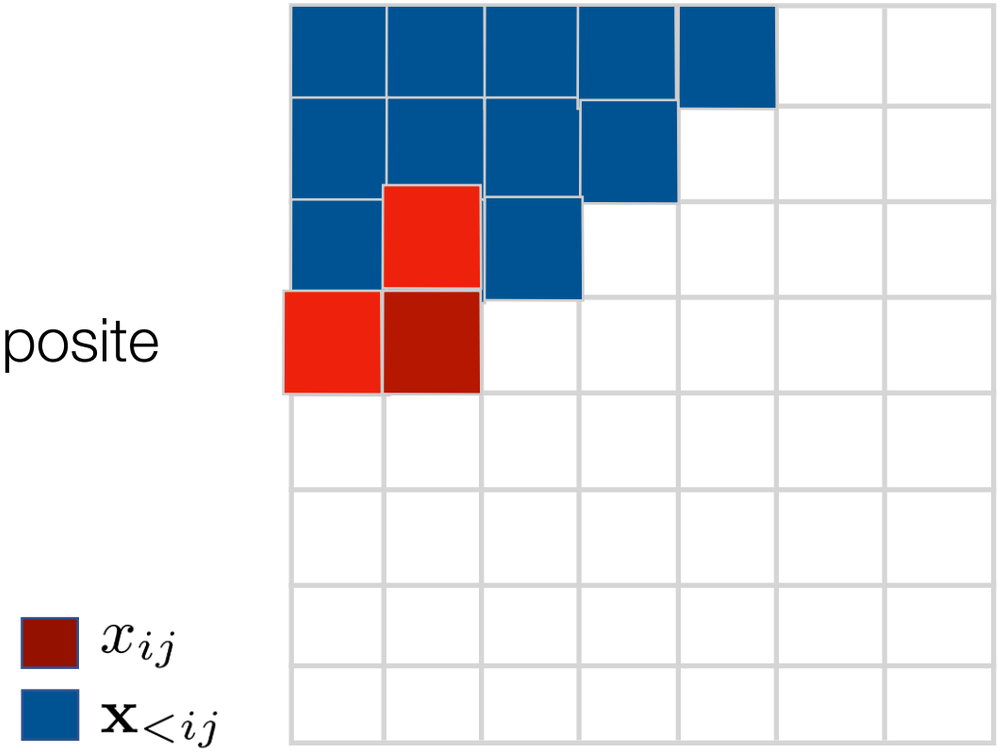
- Capture the entire available context
- Each diagonal scan from one corner to the opposite



Van den Oord et al.,ICML2016
"Conditional image generation with PixelCNNdecoders"

Pixel RNN with diagonal biLSTM

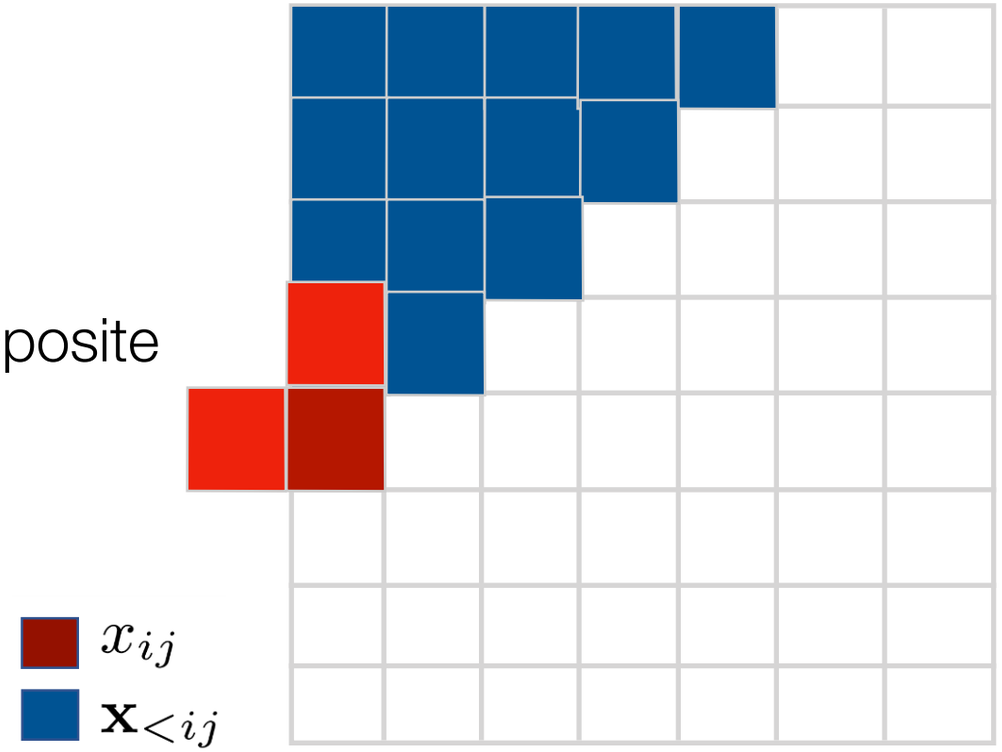
- Capture the entire available context
- Each diagonal scan from one corner to the opposite



Van den Oord et al.,ICML2016
"Conditional image generation with PixelCNNdecoders"

Pixel RNN with diagonal biLSTM

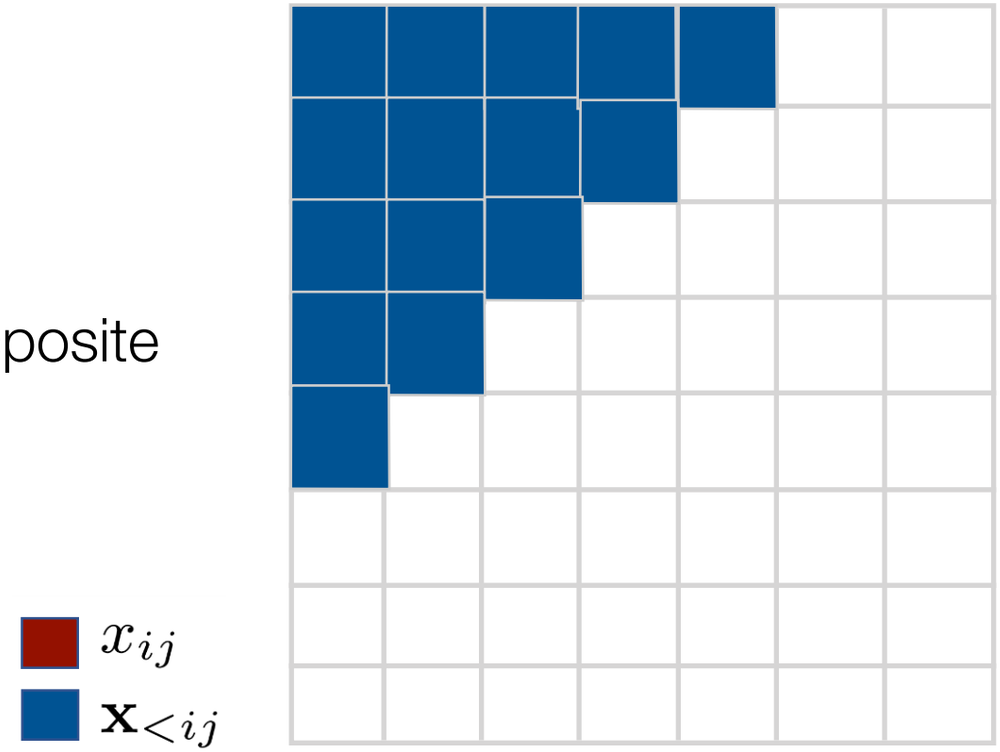
- Capture the entire available context
- Each diagonal scan from one corner to the opposite



Van den Oord et al.,ICML2016
"Conditional image generation with PixelCNNdecoders"

Pixel RNN with diagonal biLSTM

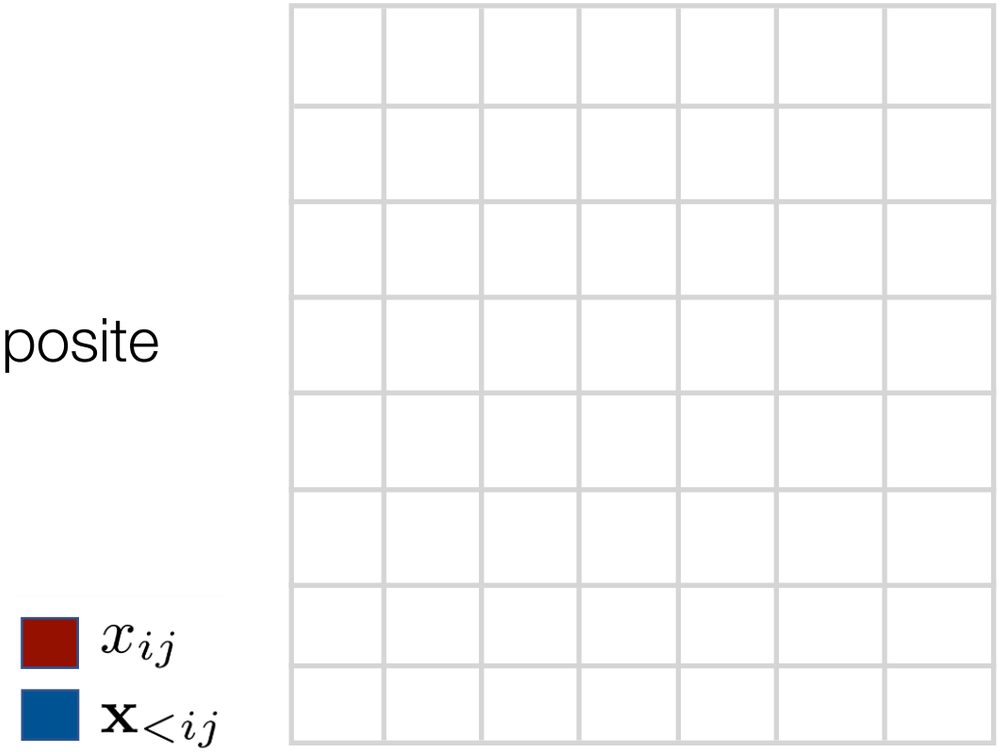
- Capture the entire available context
- Each diagonal scan from one corner to the opposite



Van den Oord et al.,ICML2016
"Conditional image generation with PixelCNNdecoders"

Pixel RNN with diagonal biLSTM

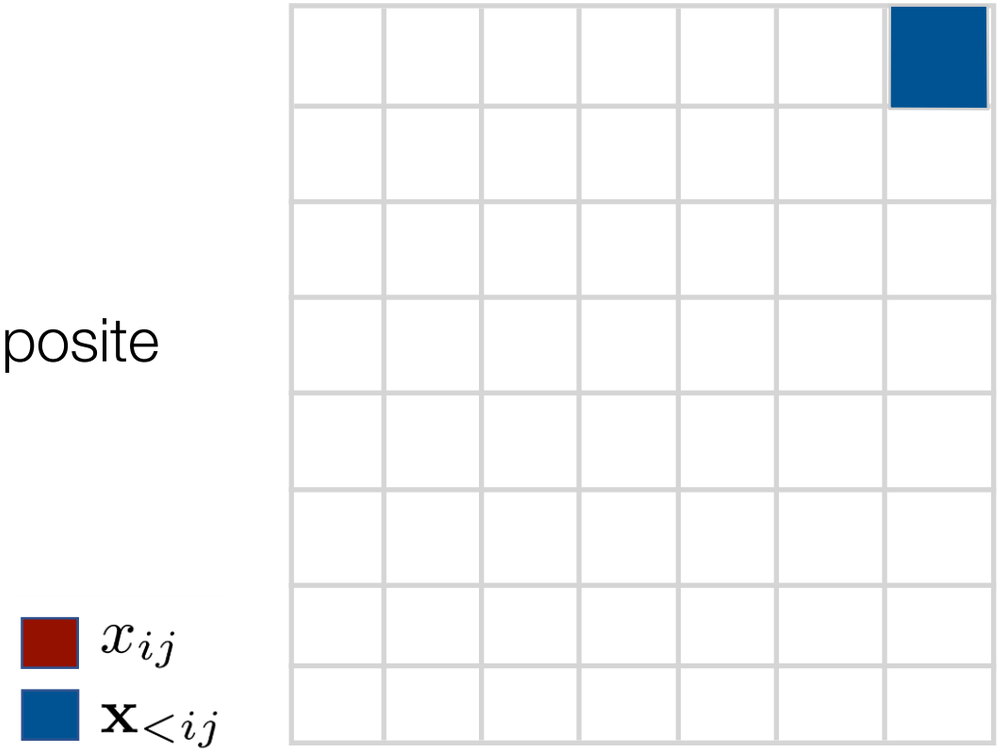
- Capture the entire available context
- Each diagonal scan from one corner to the opposite



Van den Oord et al.,ICML2016
"Conditional image generation with PixelCNNdecoders"

Pixel RNN with diagonal biLSTM

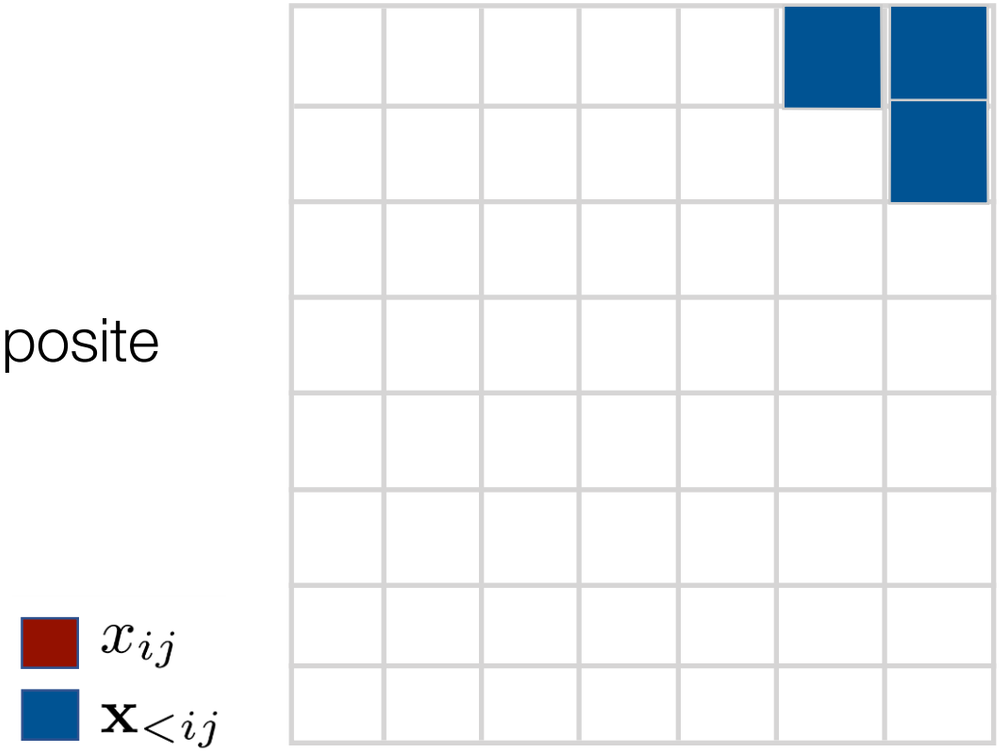
- Capture the entire available context
- Each diagonal scan from one corner to the opposite



Van den Oord et al.,ICML2016
"Conditional image generation with PixelCNNdecoders"

Pixel RNN with diagonal biLSTM

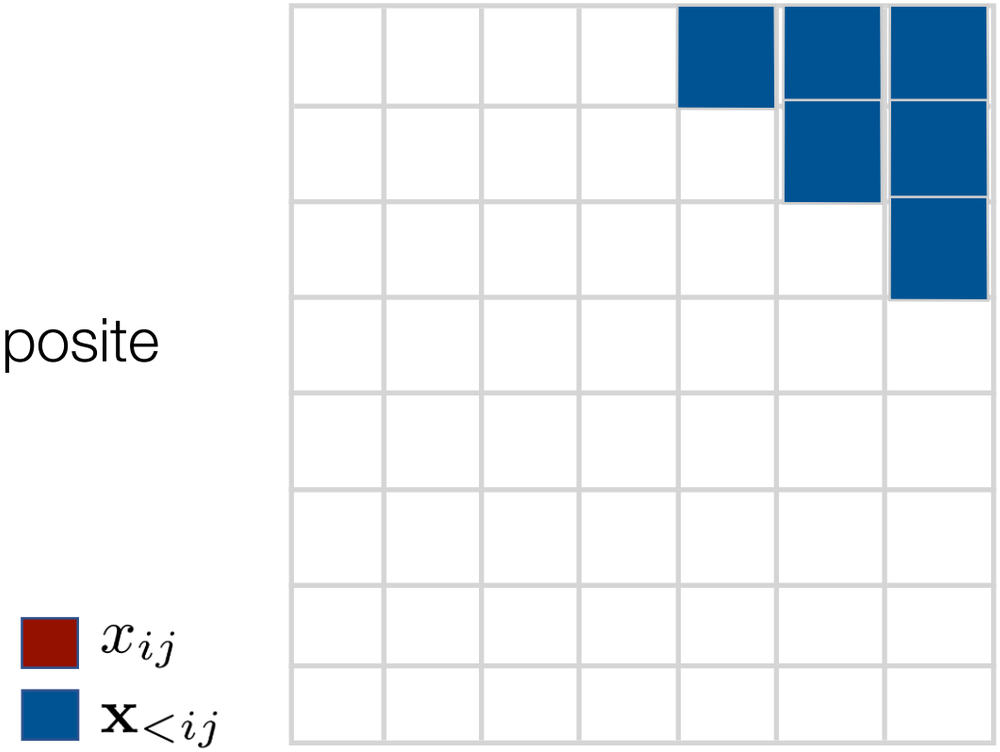
- Capture the entire available context
- Each diagonal scan from one corner to the opposite



Van den Oord et al.,ICML2016
"Conditional image generation with PixelCNNdecoders"

Pixel RNN with diagonal biLSTM

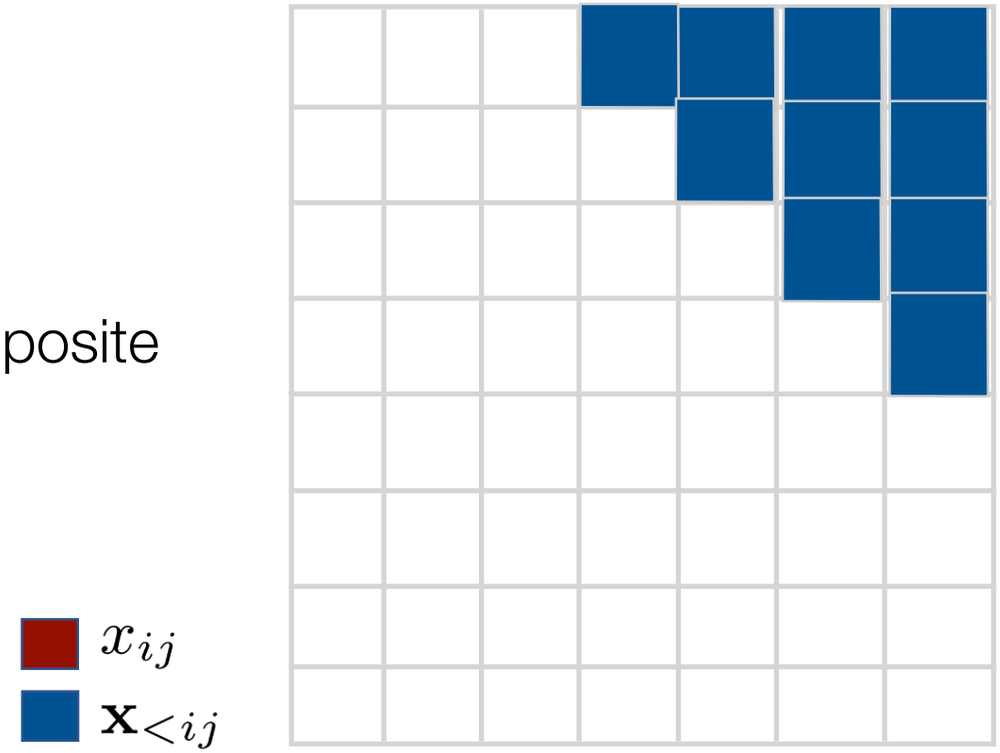
- Capture the entire available context
- Each diagonal scan from one corner to the opposite



Van den Oord et al.,ICML2016
"Conditional image generation with PixelCNNdecoders"

Pixel RNN with diagonal biLSTM

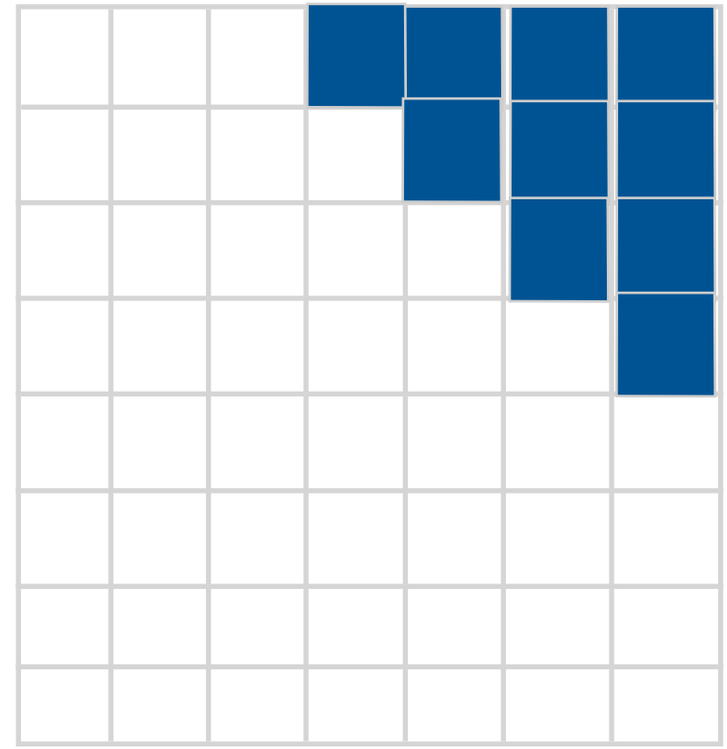
- Capture the entire available context
- Each diagonal scan from one corner to the opposite

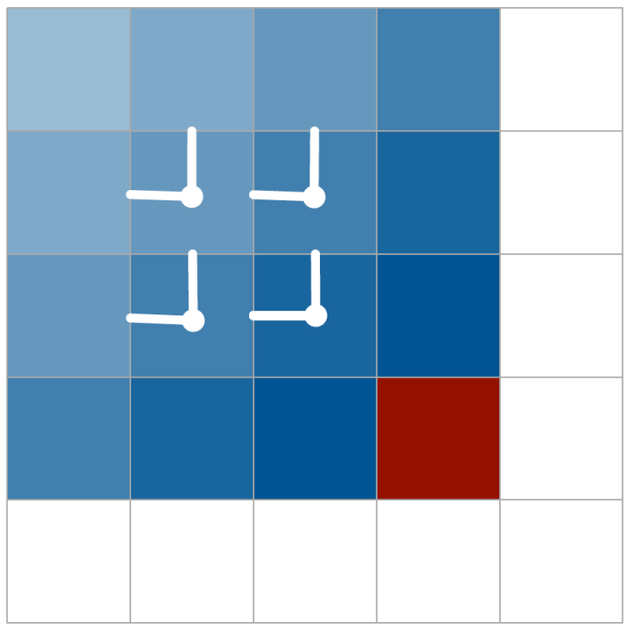


Van den Oord et al.,ICML2016
"Conditional image generation with PixelCNNdecoders"

Pixel RNN with diagonal biLSTM

- Capture the entire available context
- Each diagonal scan from one corner to the opposite

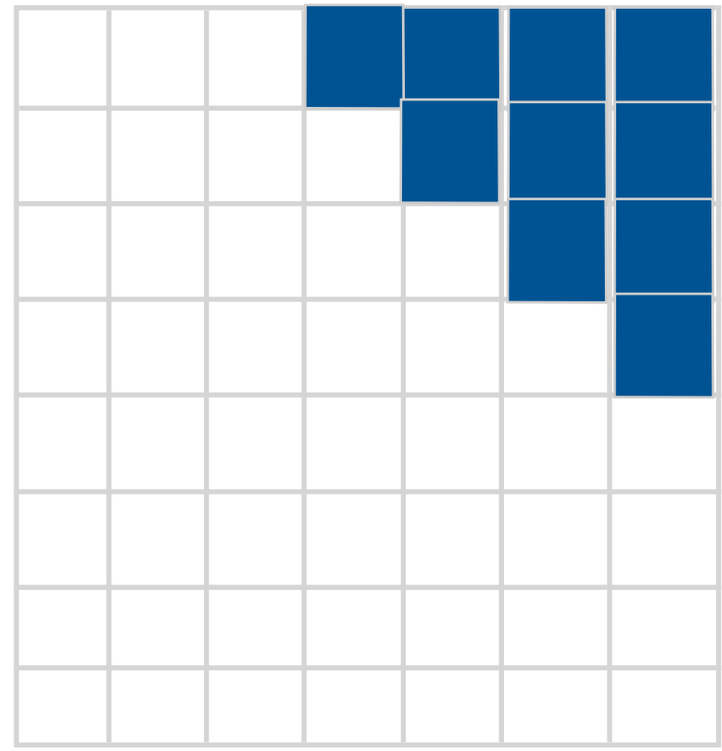


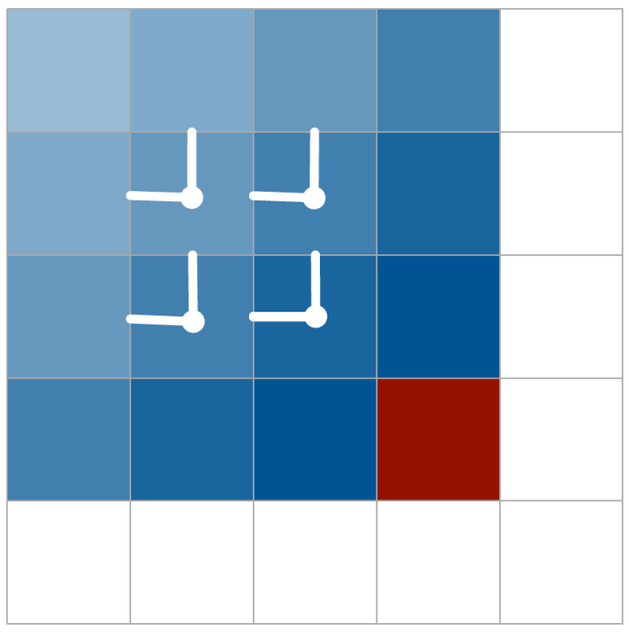


Van den Oord et al.,ICML2016
"Conditional image generation with PixelCNNdecoders"

Pixel RNN with diagonal biLSTM

- Capture the entire available context
- Each diagonal scan from one corner to the opposite





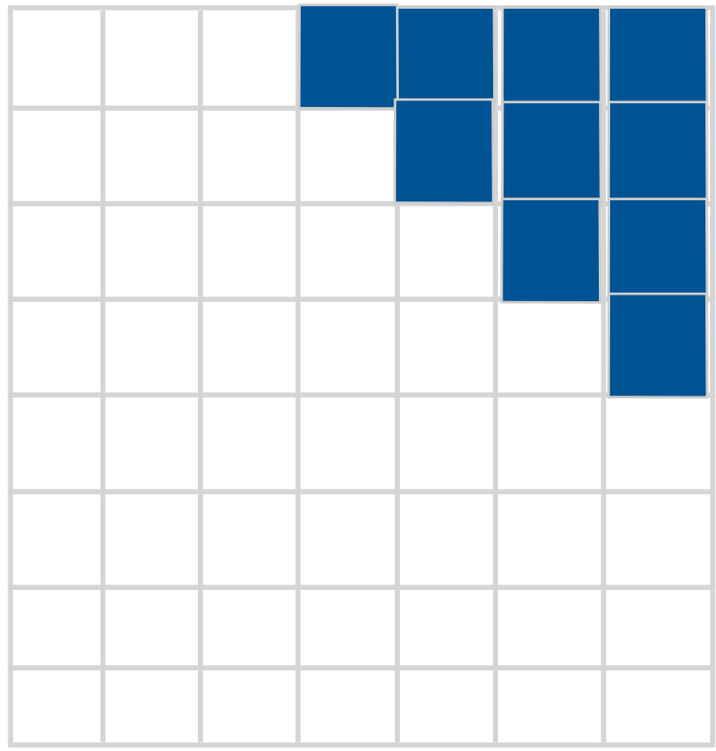
this diagonal convolution
computationally difficult

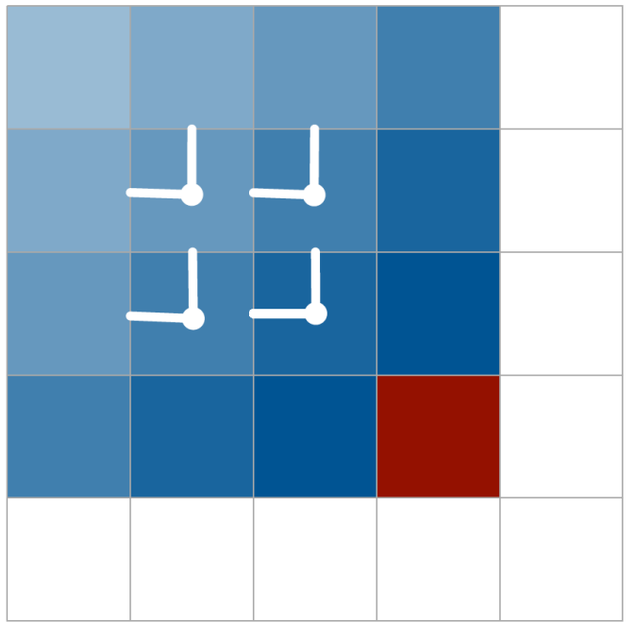
Trick: Skew pixels by one

Van den Oord et al.,ICML2016
"Conditional image generation with PixelCNNdecoders"

Pixel RNN with diagonal biLSTM

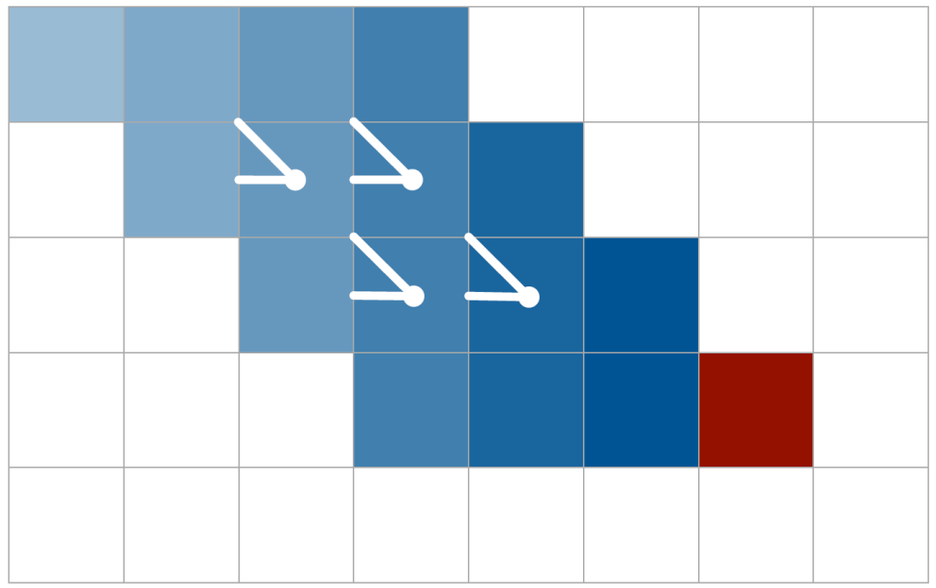
- Capture the entire available context
- Each diagonal scan from one corner to the opposite





this diagonal convolution
computationally difficult

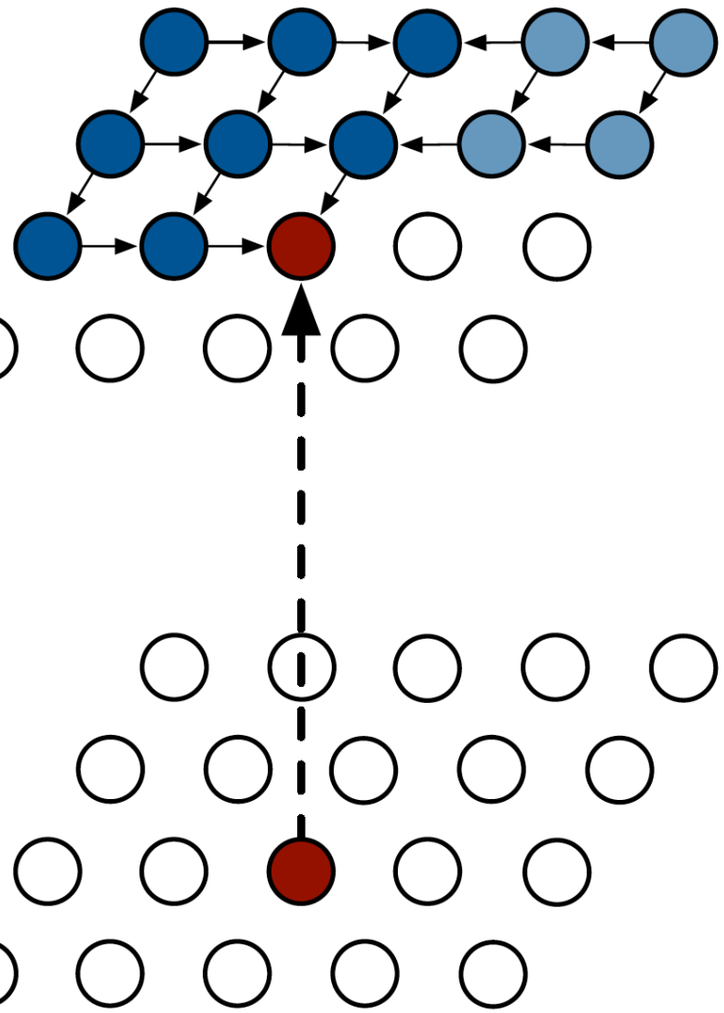
Trick: Skew pixels by one



Van den Oord et al.,ICML2016
"Conditional image generation with PixelCNNdecoders"

Pixel RNN: pros and cons of diagonal biLSTM

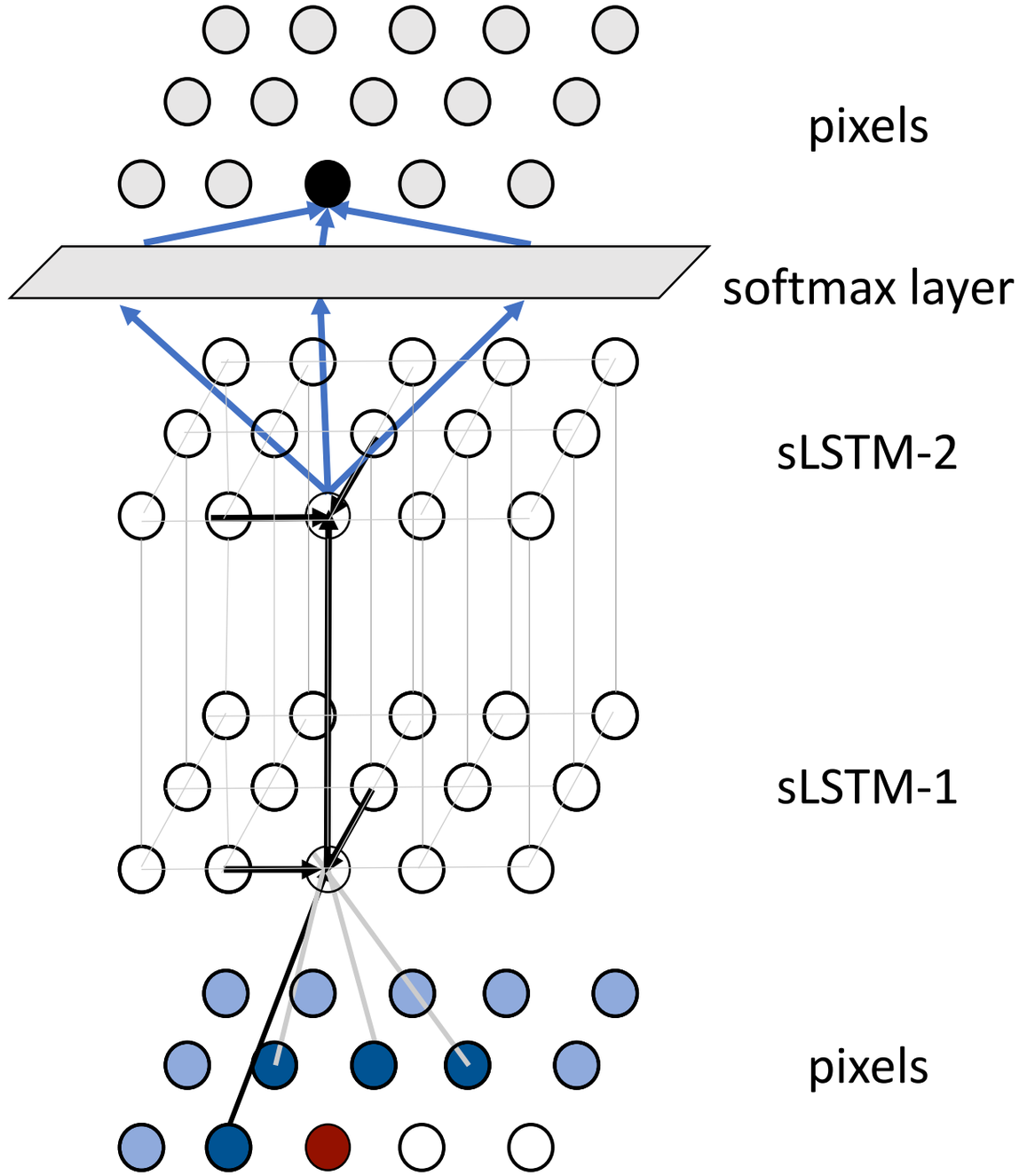
- Compute state for entire row
- Global receptive field => use of all available context



Van den Oord et al.,ICMLS2016
"Conditional image generation with PixelCNNdecoders"

Pixel recurrent neural networks (Pixel-RNN) (ICML 2016)

- Spatial LSTMs
- Treat pixels as discrete variables
- To estimate a pixel value, do classification in every channel (256 classes indicating pixel values 0-255)
- Implemented with a final softmax layer

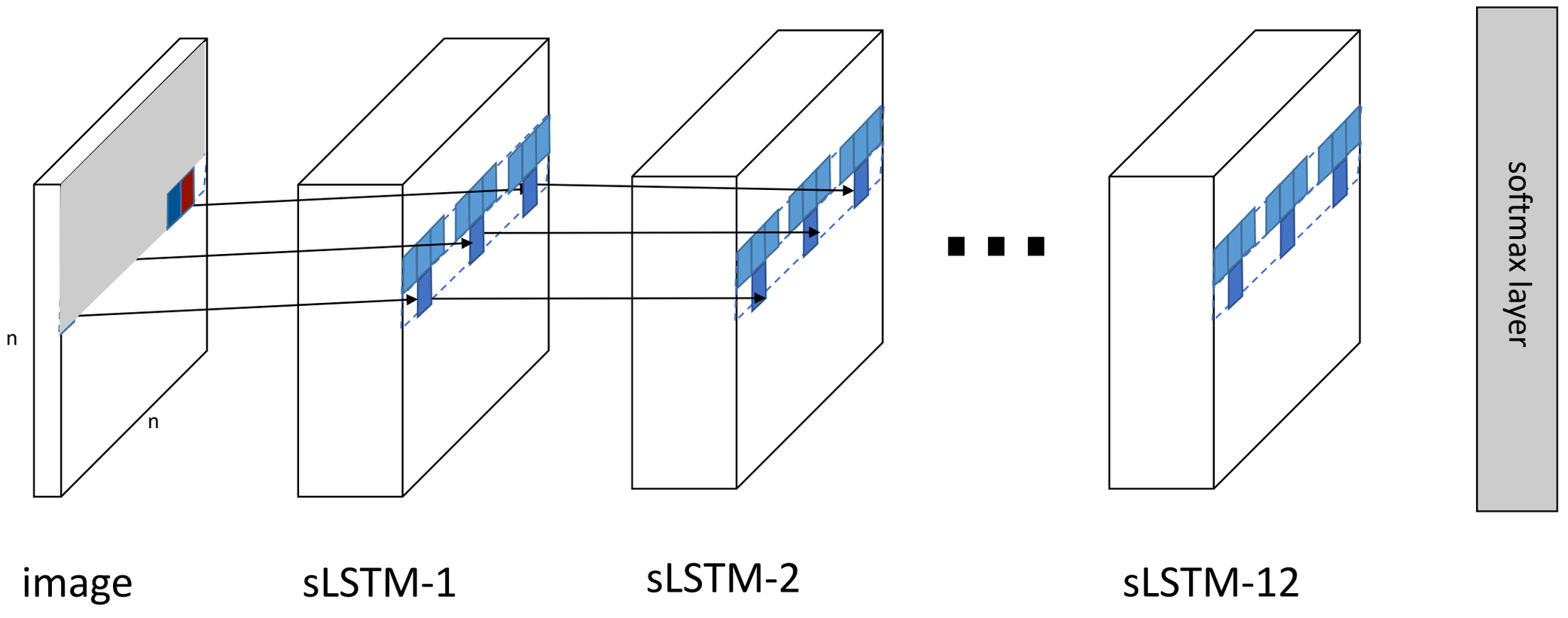


Pixel recurrent neural networks (Pixel-RNN) (ICML 2016)

- Spatial LSTMs
- Treat pixels as discrete variables
- To estimate a pixel value, do classification in every channel (256 classes indicating pixel values 0-255)
- Implemented with a final softmax layer

Pixel recurrent neural networks (Pixel-RNN) (ICML 2016)

- Spatial LSTMs
- Treat pixels as discrete variables
- To estimate a pixel value, do classification in every channel (256 classes indicating pixel values 0-255)
- Implemented with a final softmax layer



Pixel recurrent neural networks (Pixel-RNN) (ICML 2016)

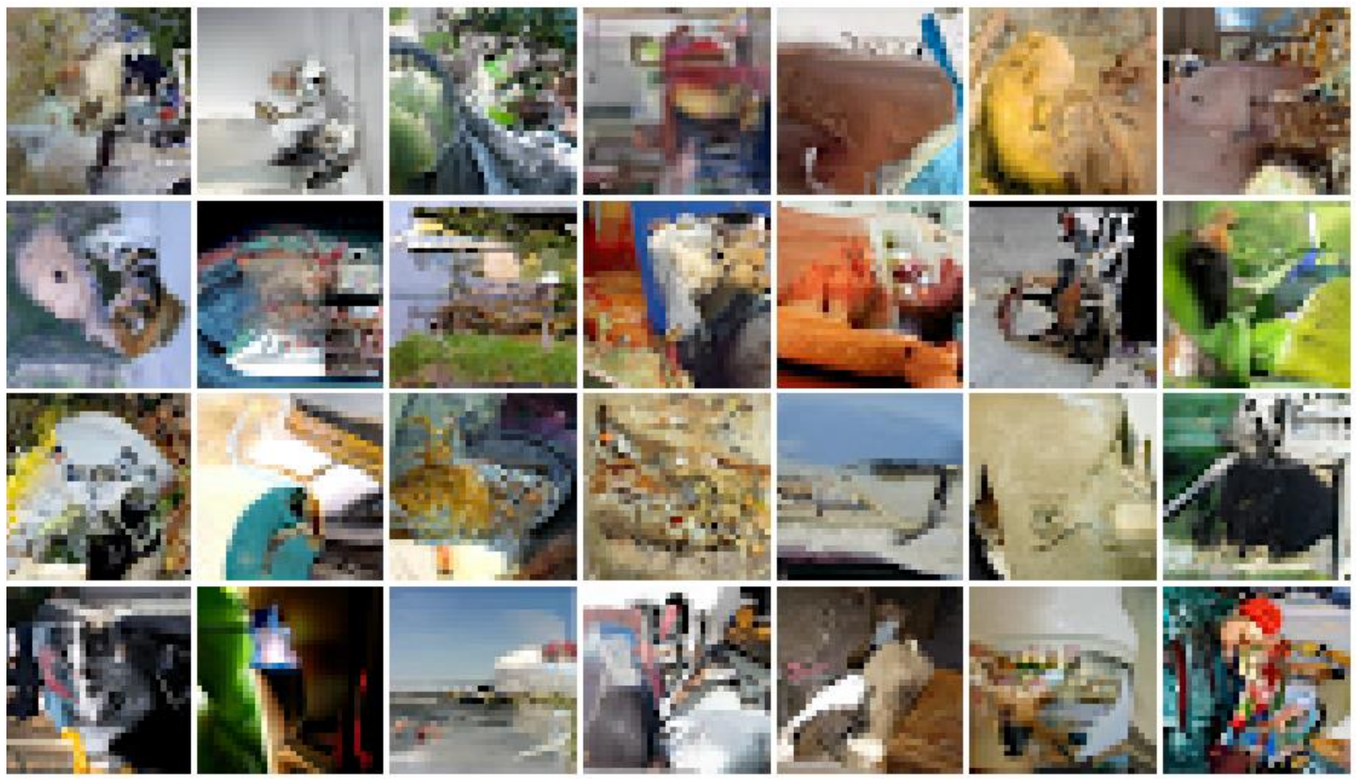
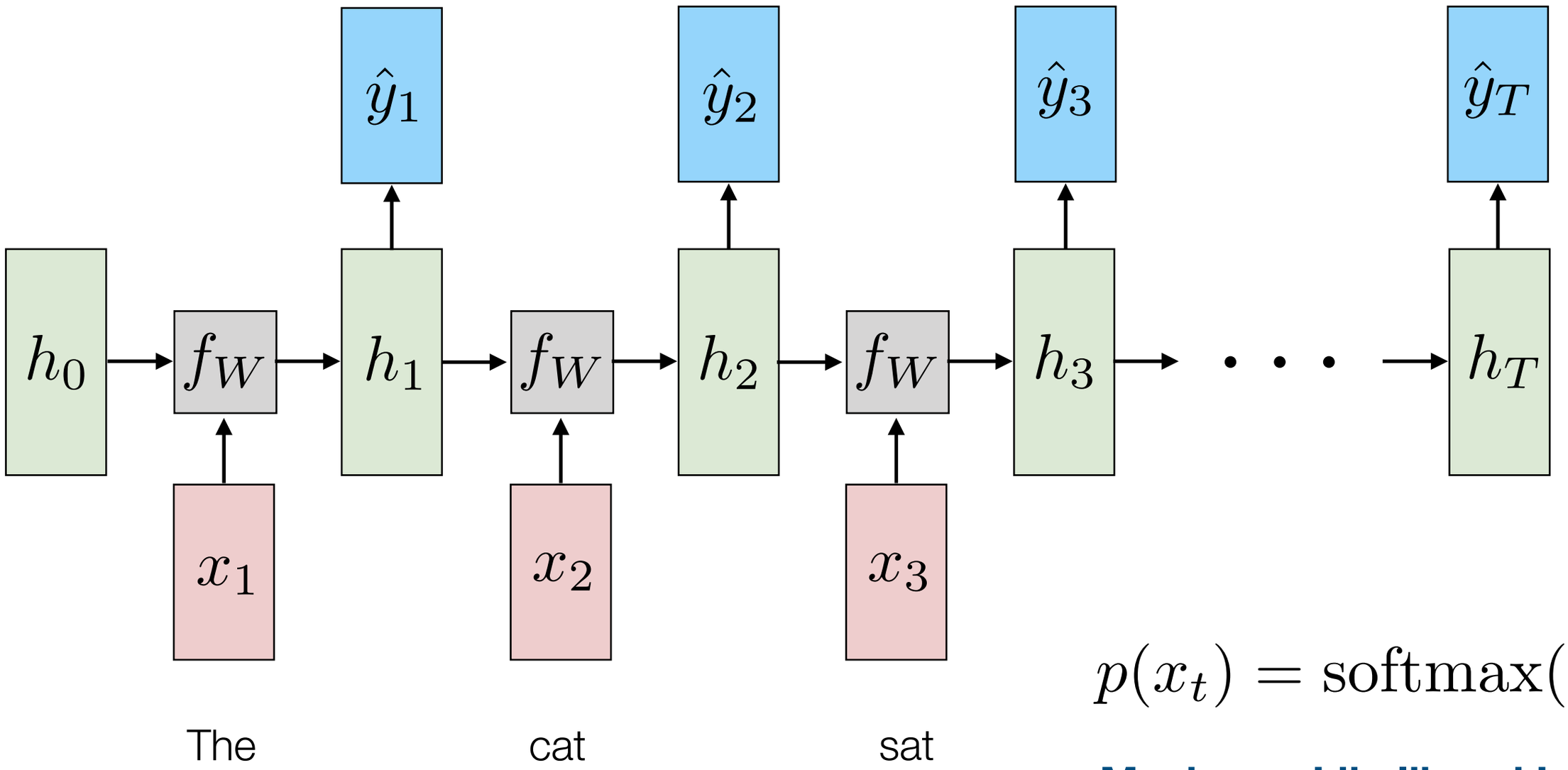


Figure: 32x32 ImageNet results from Diagonal BiLSTM model.

Van den Oord et al.,ICML2016
"Conditional image generation with PixelCNNdecoders"

Recurrent Neural Language Model (Training)

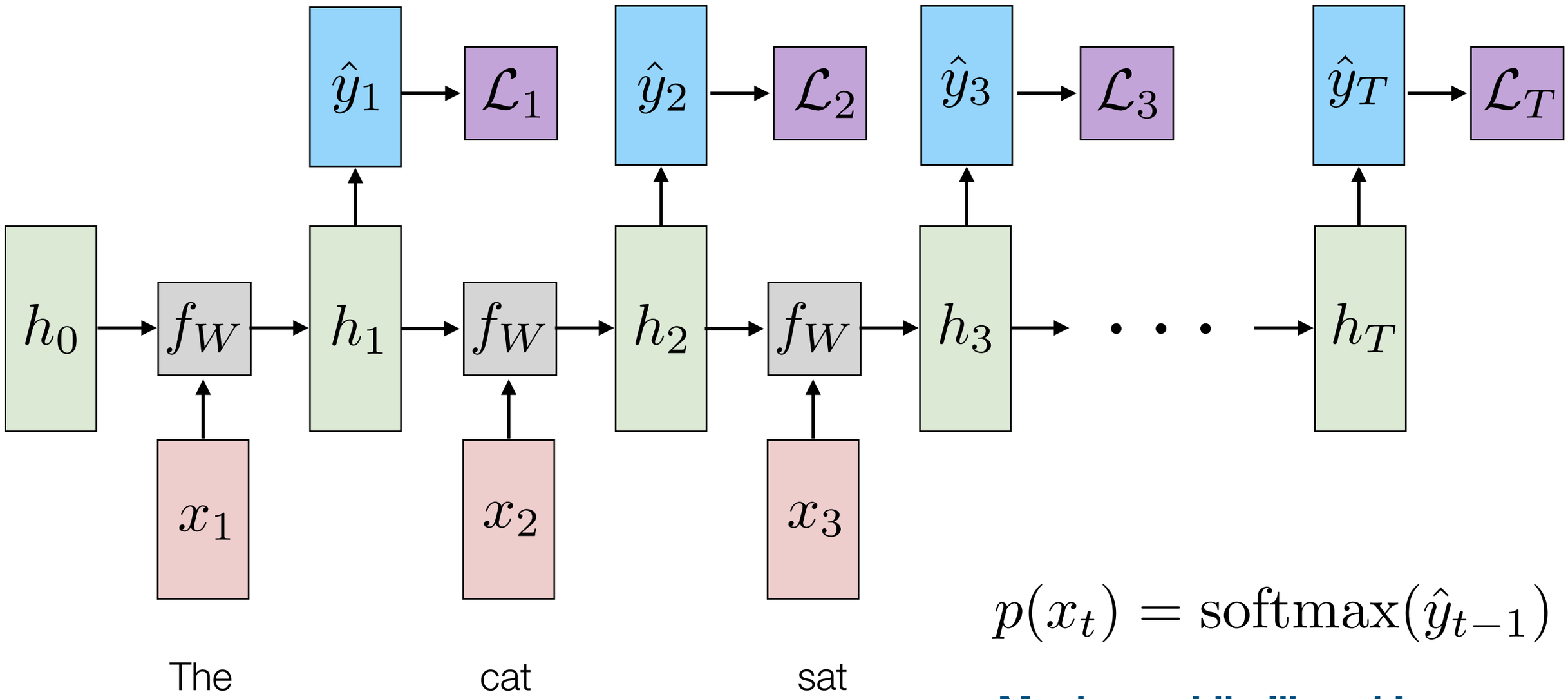


$$p(x_t) = \text{softmax}(\hat{y}_{t-1})$$

Maximum Likelihood Loss

Tomas Mikolov, Martin Karafiat, Lukas Burget, Jan Cernocky, and Sanjeev Khudanpur. 2010. Recurrent Neural Network Based Language Model. InProceedings of INTERSPEECH

Recurrent Neural Language Model (Training)

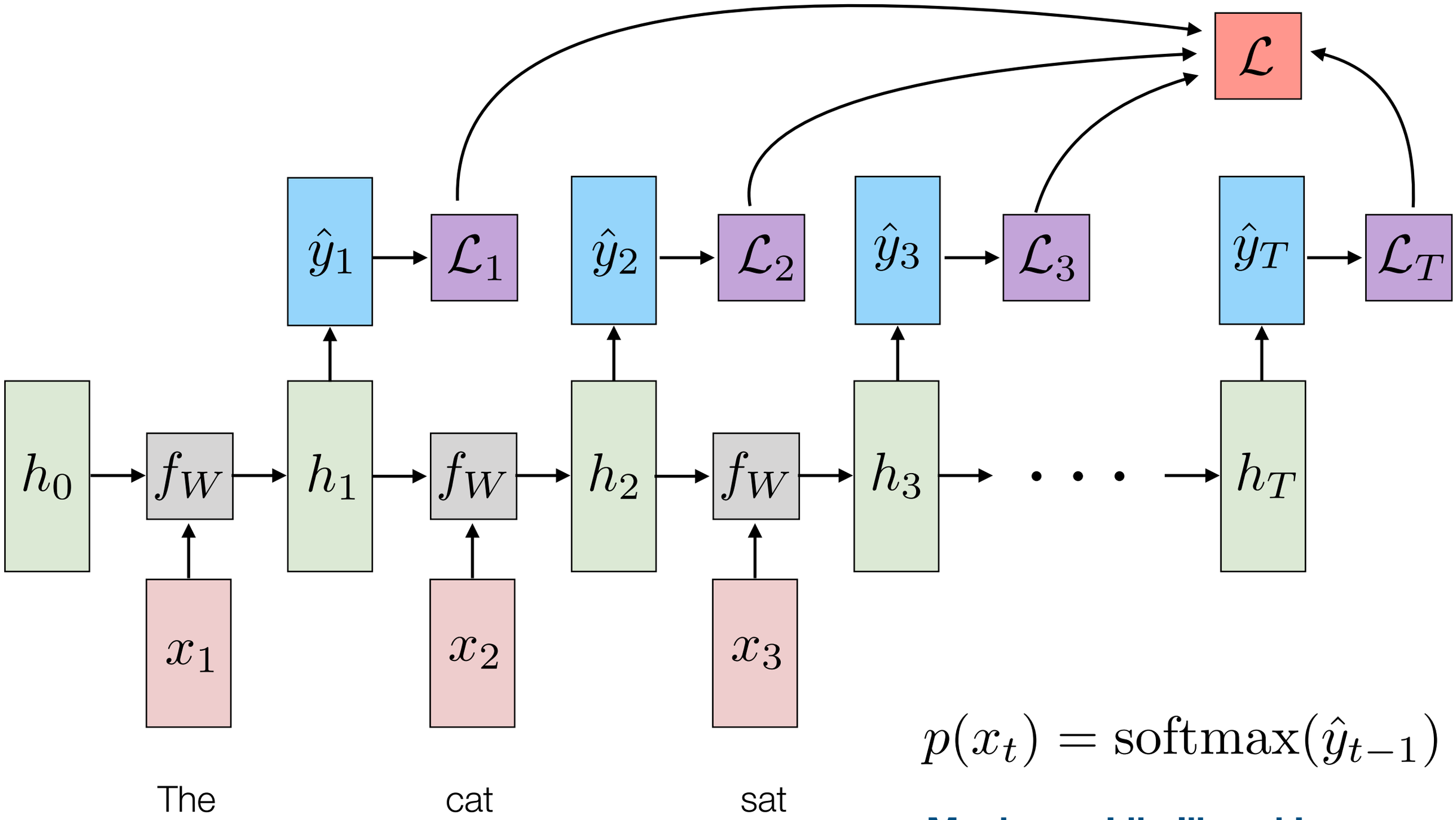


$$p(x_t) = \text{softmax}(\hat{y}_{t-1})$$

Maximum Likelihood Loss

Tomas Mikolov, Martin Karafiat, Lukas Burget, Jan Cernocky, and Sanjeev Khudanpur. 2010. Recurrent Neural Network Based Language Model. InProceedings of INTERSPEECH

Recurrent Neural Language Model (Training)



$$p(x_t) = \text{softmax}(\hat{y}_{t-1})$$

Maximum Likelihood Loss

Tomas Mikolov, Martin Karafiat, Lukas Burget, Jan Cernocky, and Sanjeev Khudanpur. 2010. Recurrent Neural Network Based Language Model. InProceedings of INTERSPEECH

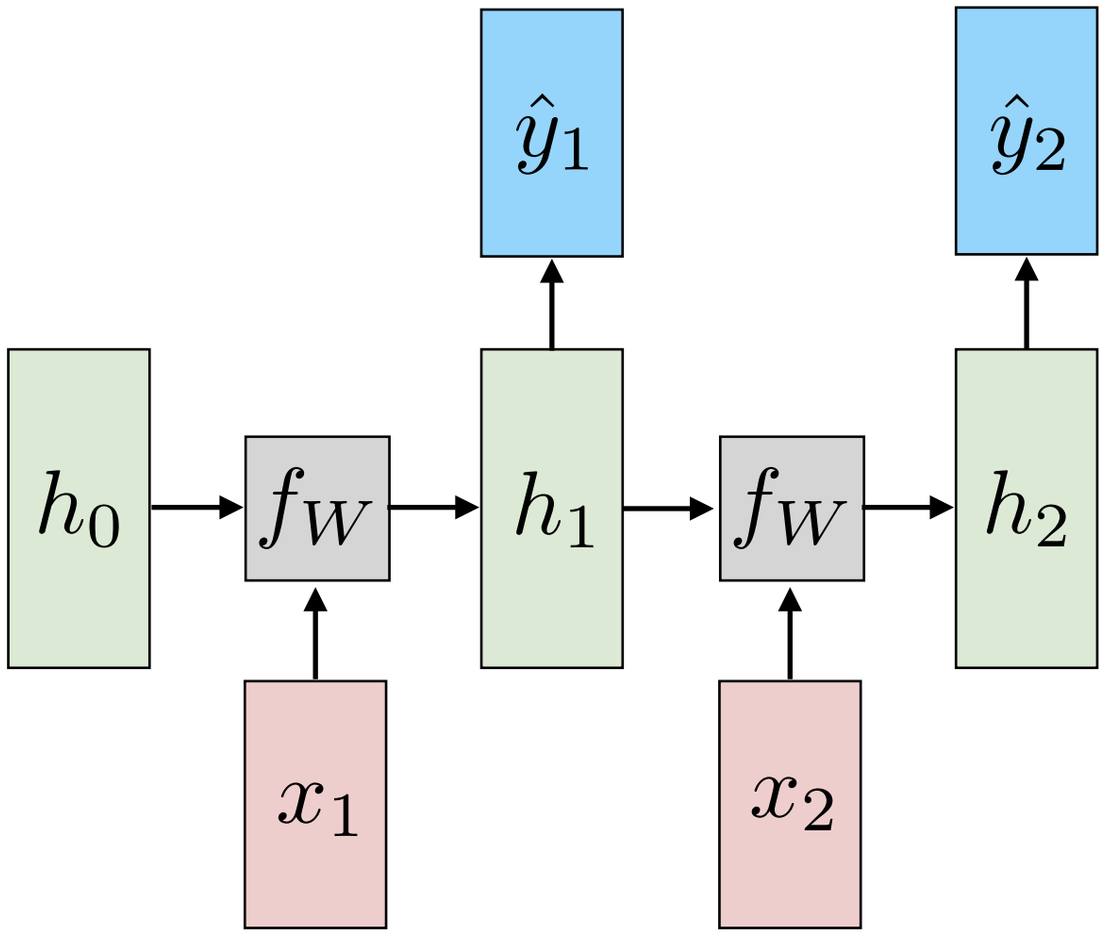
Recurrent Neural Language Model (Test)

Sequentially Sample from Output Distribution

Tomas Mikolov, Martin Karafiat, Lukas Burget, Jan Cernocky, and Sanjeev Khudanpur. 2010. Recurrent Neural Network Based Language Model. InProceedings of INTERSPEECH

Recurrent Neural Language Model (Test)

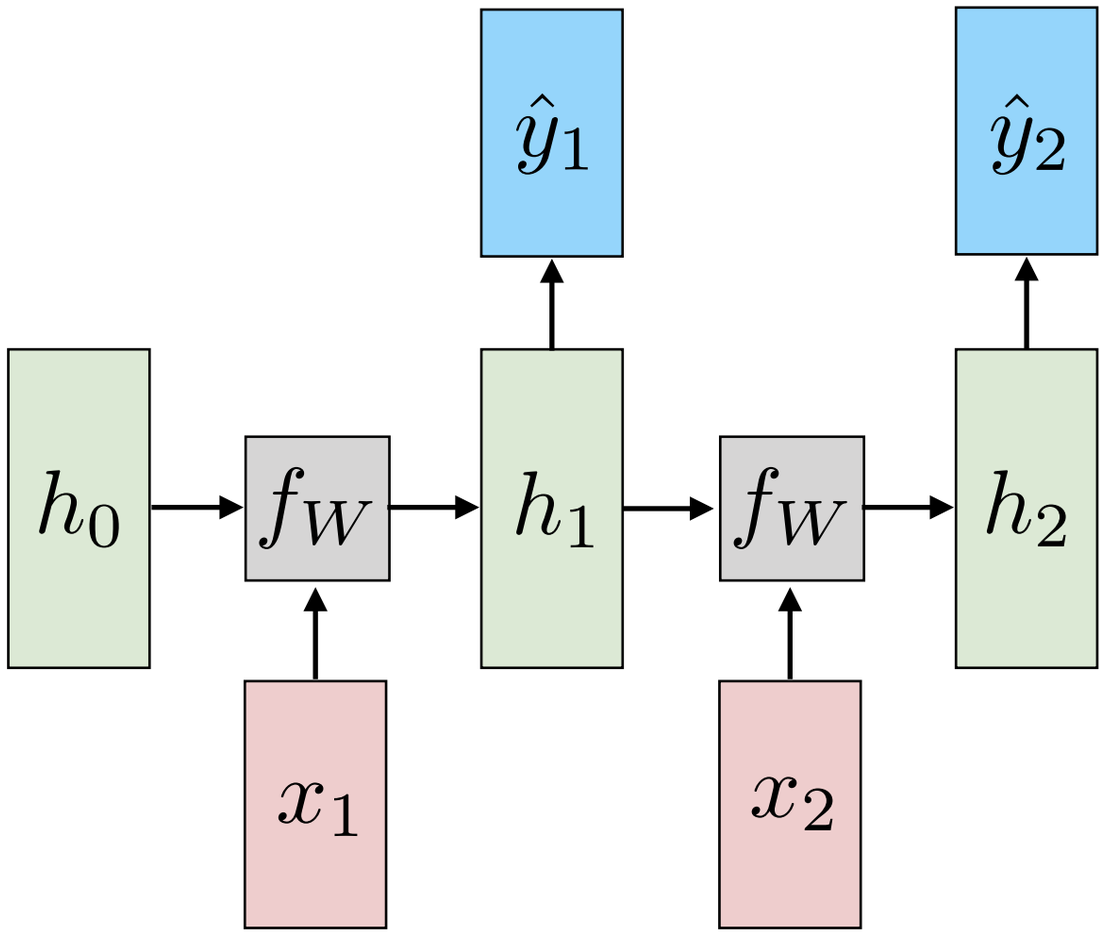
Sequentially Sample from Output Distribution



Tomas Mikolov, Martin Karafiat, Lukas Burget, Jan Cernocky, and Sanjeev Khudanpur. 2010. Recurrent Neural Network Based Language Model. InProceedings of INTERSPEECH

Recurrent Neural Language Model (Test)

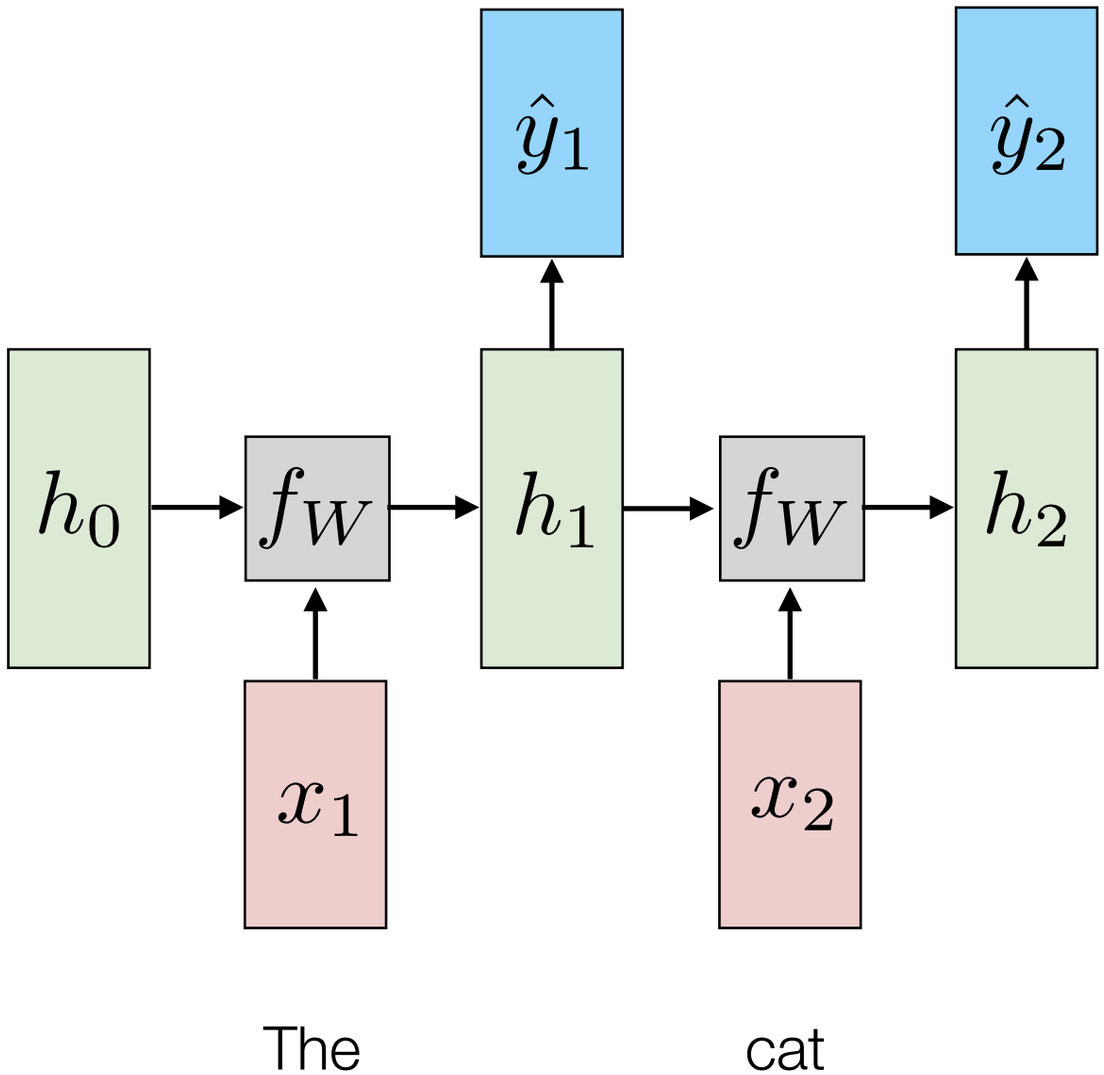
Sequentially Sample from Output Distribution



Tomas Mikolov, Martin Karafiat, Lukas Burget, Jan Cernocky, and Sanjeev Khudanpur. 2010. Recurrent Neural Network Based Language Model. InProceedings of INTERSPEECH

Recurrent Neural Language Model (Test)

Sequentially Sample from Output Distribution

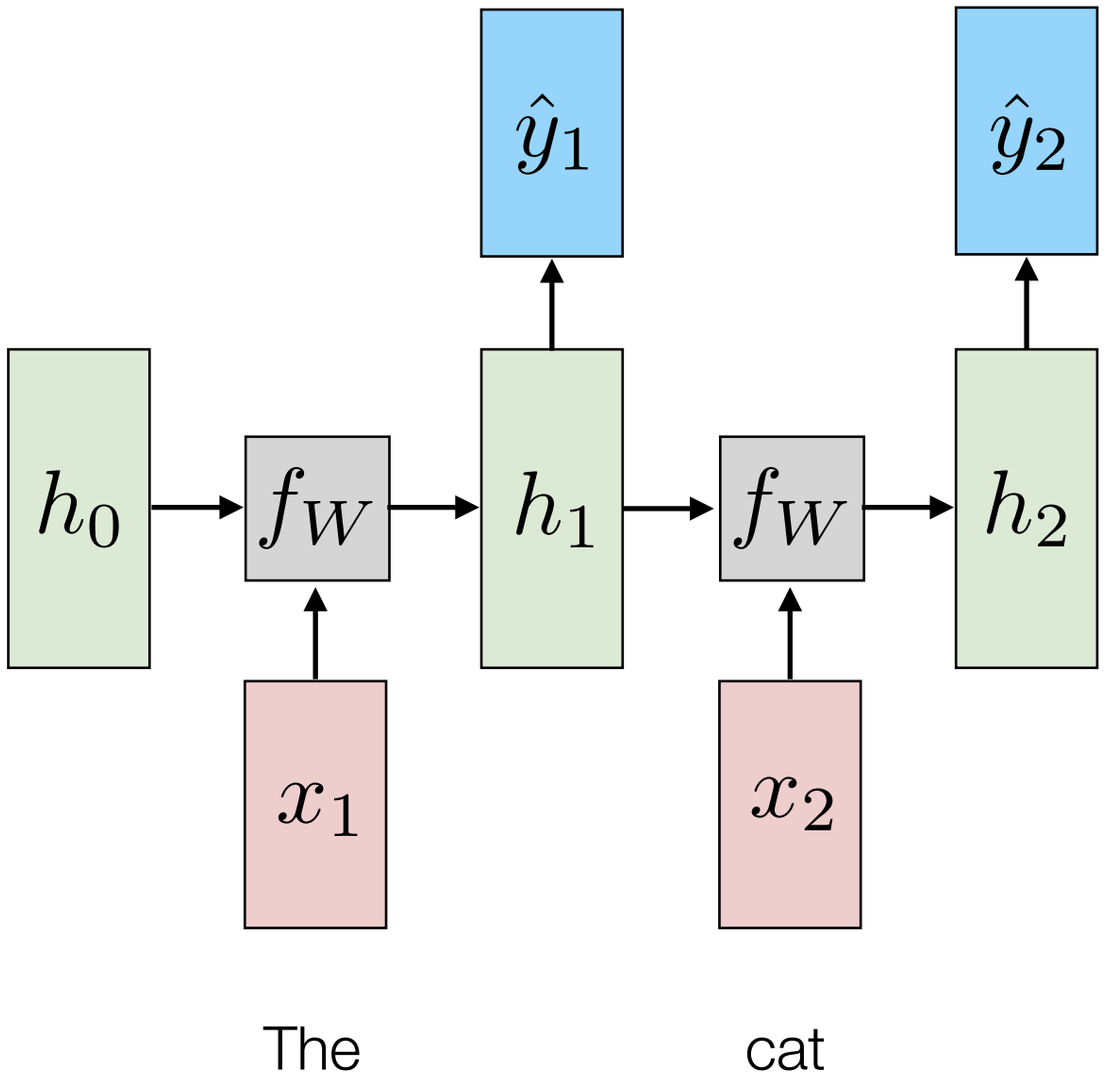


Tomas Mikolov, Martin Karafiat, Lukas Burget, Jan Cernocky, and Sanjeev Khudanpur. 2010. Recurrent Neural Network Based Language Model. InProceedings of INTERSPEECH

Recurrent Neural Language Model (Test)

Sequentially Sample from Output Distribution

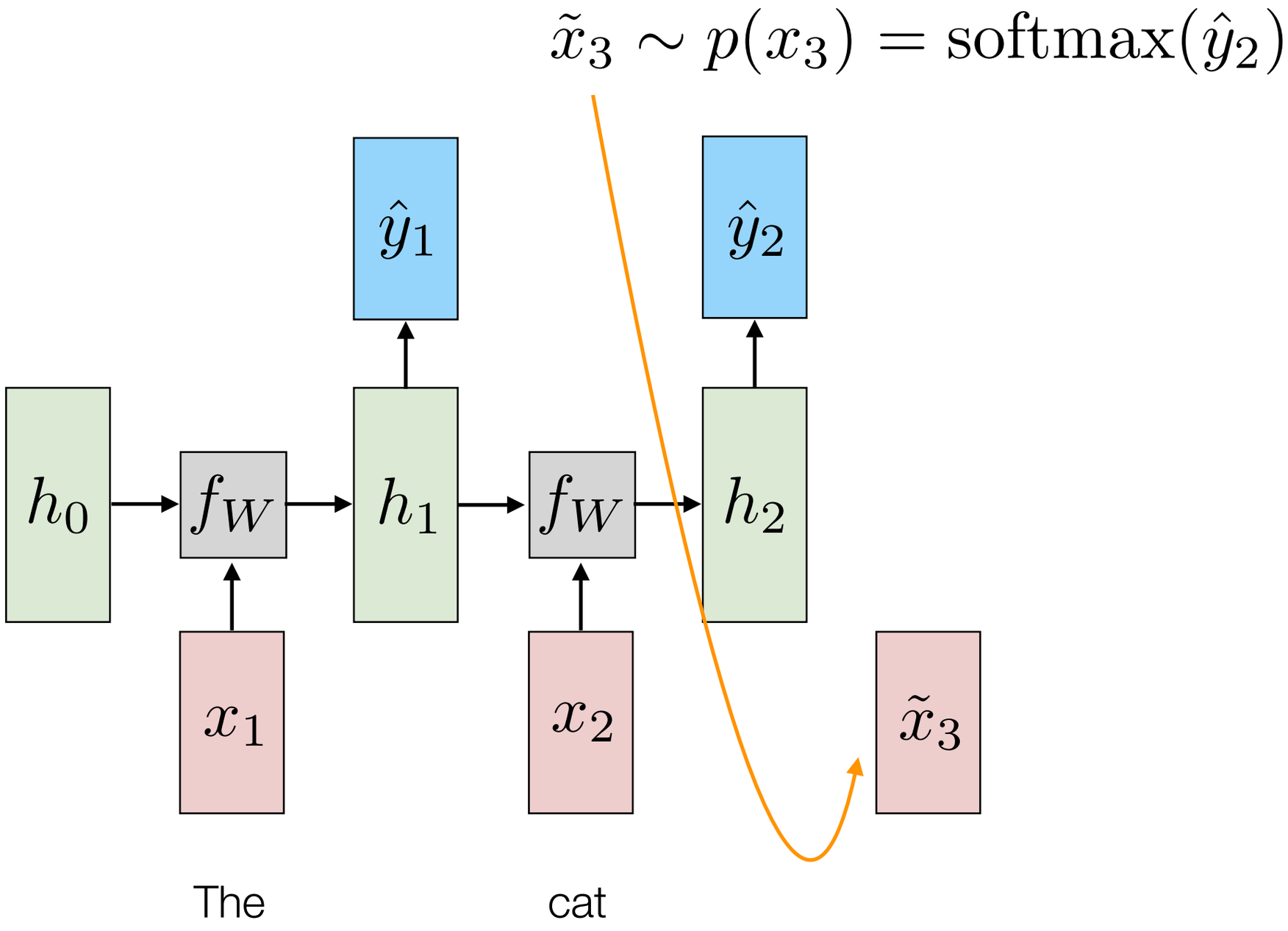
$$\tilde{x}_3 \sim p(x_3) = \text{softmax}(\hat{y}_2)$$



Tomas Mikolov, Martin Karafiat, Lukas Burget, Jan Cernocky, and Sanjeev Khudanpur. 2010. Recurrent Neural Network Based Language Model. InProceedings of INTERSPEECH

Recurrent Neural Language Model (Test)

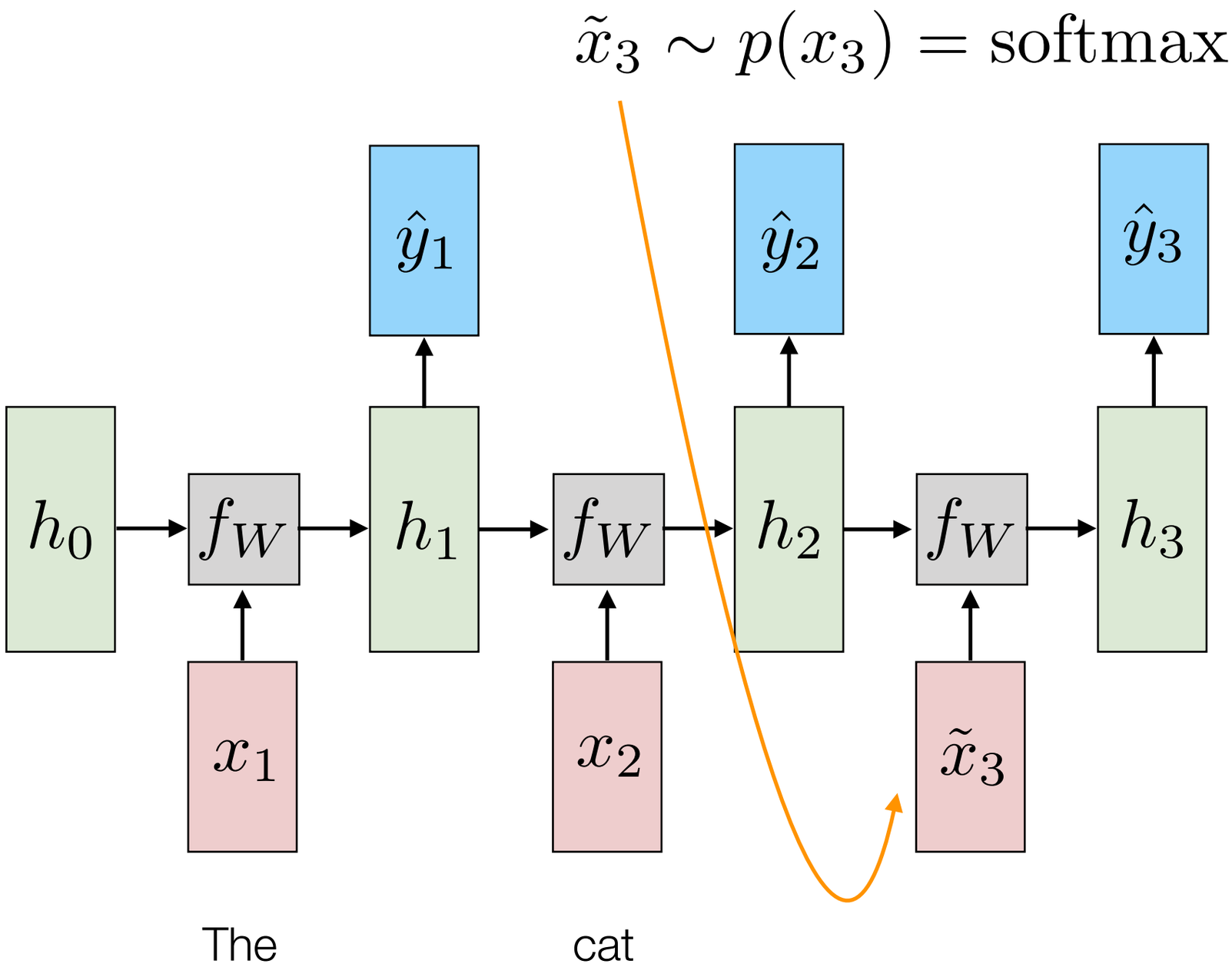
Sequentially Sample from Output Distribution



Tomas Mikolov, Martin Karafiat, Lukas Burget, Jan Cernocky, and Sanjeev Khudanpur. 2010. Recurrent Neural Network Based Language Model. InProceedings of INTERSPEECH

Recurrent Neural Language Model (Test)

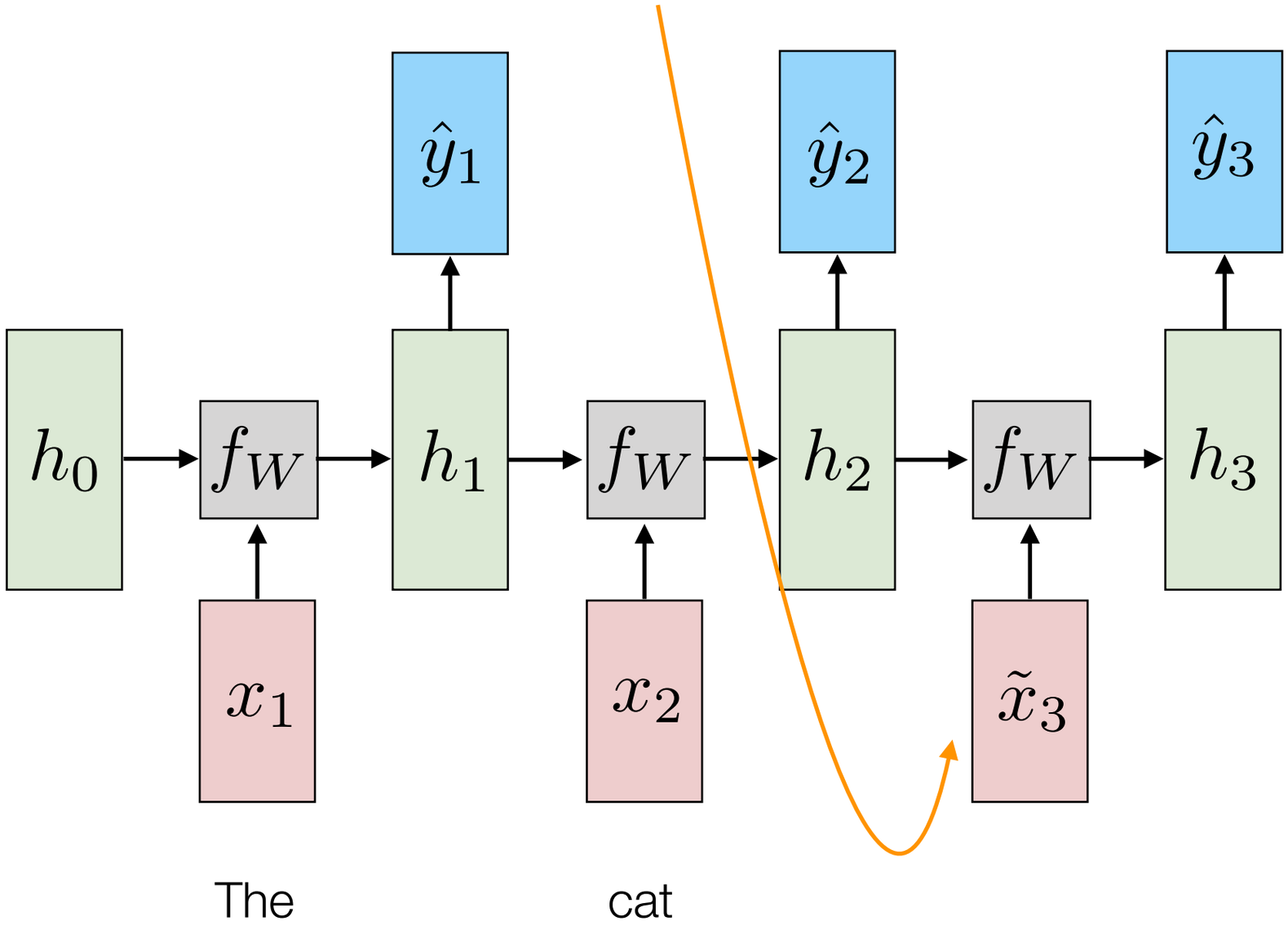
Sequentially Sample from Output Distribution



Tomas Mikolov, Martin Karafiat, Lukas Burget, Jan Cernocky, and Sanjeev Khudanpur. 2010. Recurrent Neural Network Based Language Model. InProceedings of INTERSPEECH

Recurrent Neural Language Model (Test)

Sequentially Sample from Output Distribution

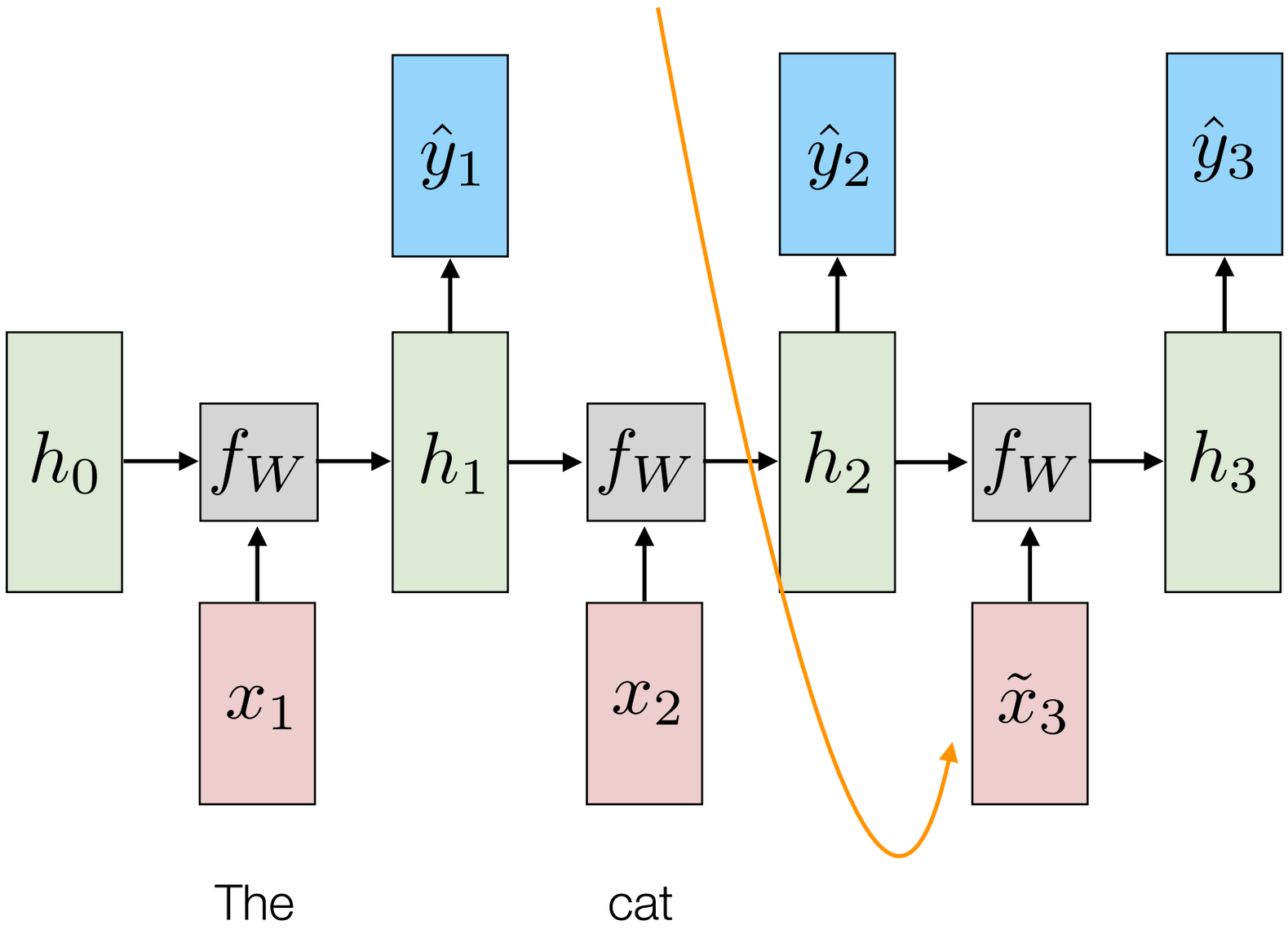


Tomas Mikolov, Martin Karafiat, Lukas Burget, Jan Cernocky, and Sanjeev Khudanpur. 2010. Recurrent Neural Network Based Language Model. InProceedings of INTERSPEECH

Recurrent Neural Language Model (Test)

Sequentially Sample from Output Distribution

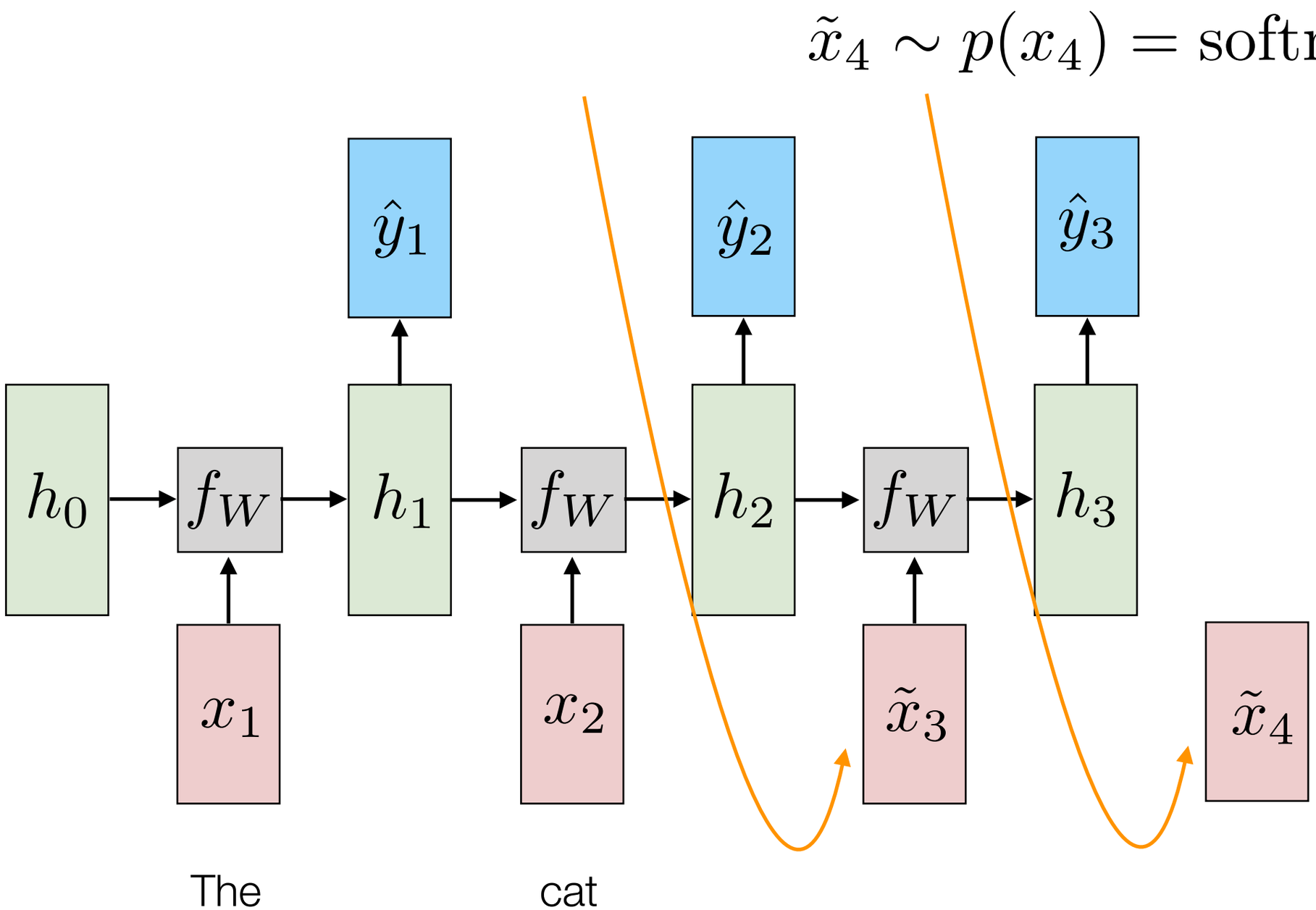
$$\tilde{x}_4 \sim p(x_4) = \text{softmax}(\hat{y}_3)$$



Tomas Mikolov, Martin Karafiat, Lukas Burget, Jan Cernocky, and Sanjeev Khudanpur. 2010. Recurrent Neural Network Based Language Model. InProceedings of INTERSPEECH

Recurrent Neural Language Model (Test)

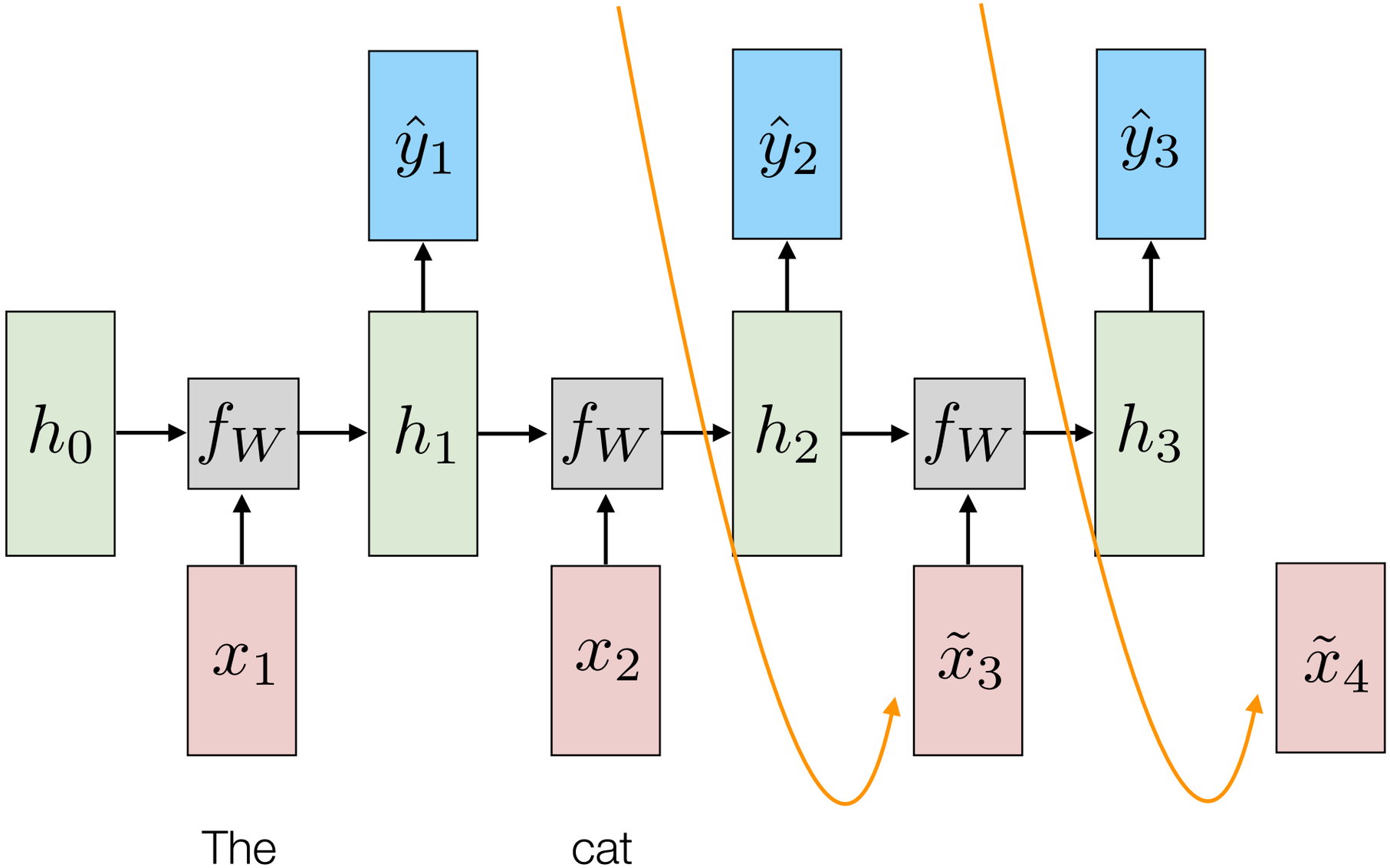
Sequentially Sample from Output Distribution



Tomas Mikolov, Martin Karafiat, Lukas Burget, Jan Cernocky, and Sanjeev Khudanpur. 2010. Recurrent Neural Network Based Language Model. InProceedings of INTERSPEECH

Recurrent Neural Language Model (Test)

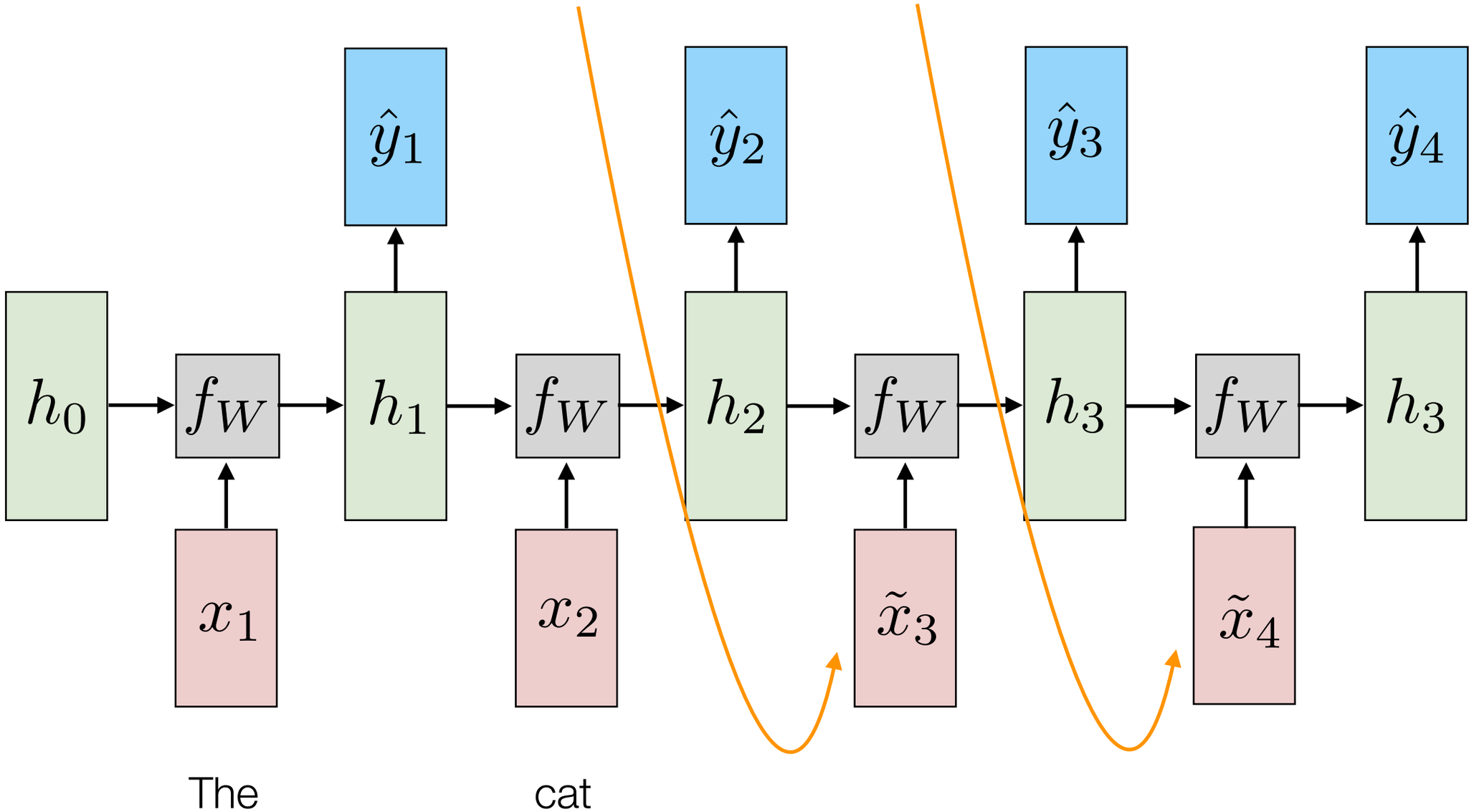
Sequentially Sample from Output Distribution



Tomas Mikolov, Martin Karafiat, Lukas Burget, Jan Cernocky, and Sanjeev Khudanpur. 2010. Recurrent Neural Network Based Language Model. InProceedings of INTERSPEECH

Recurrent Neural Language Model (Test)

Sequentially Sample from Output Distribution

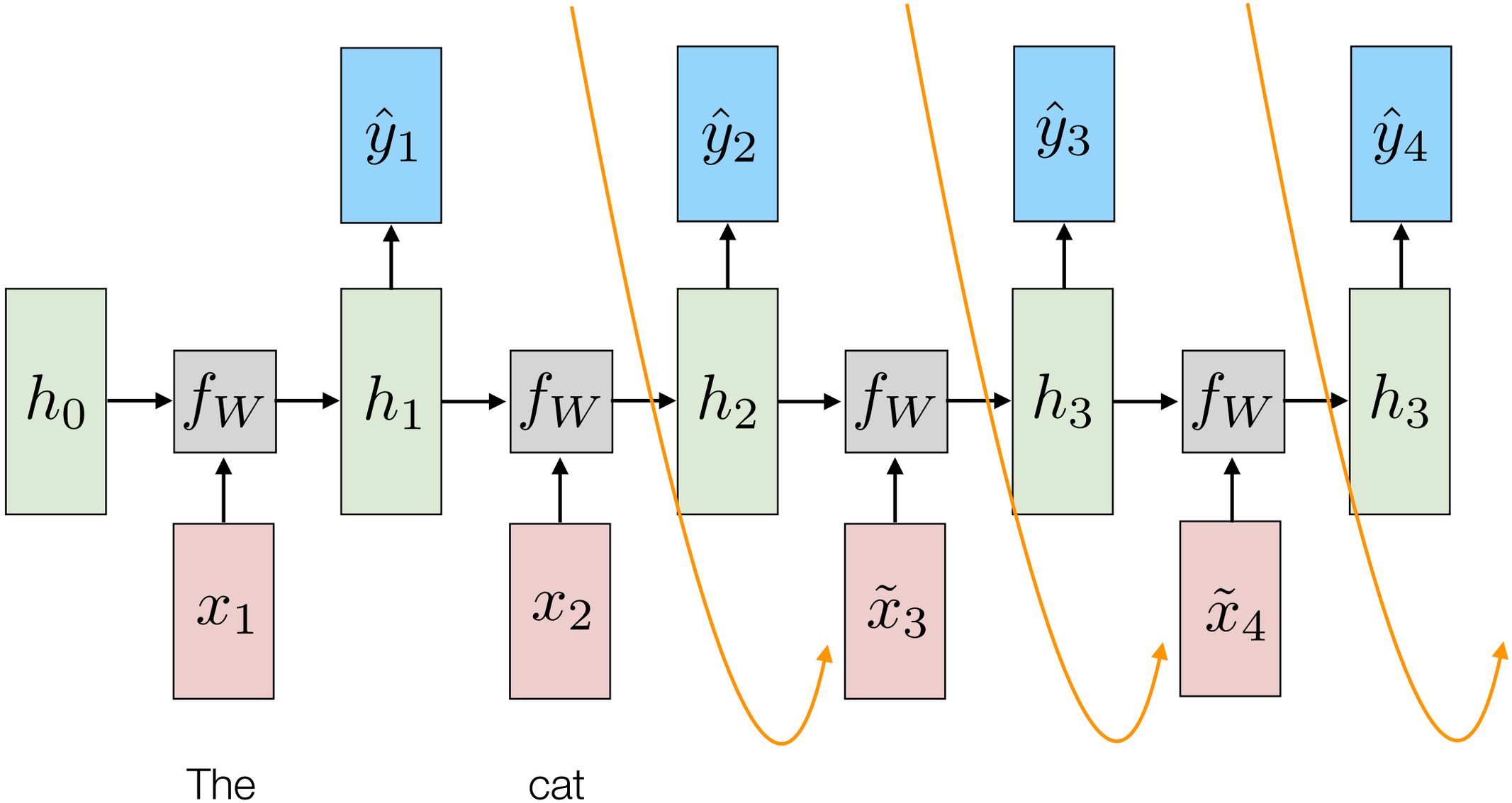


Tomas Mikolov, Martin Karafiat, Lukas Burget, Jan Cernocky, and Sanjeev Khudanpur. 2010. Recurrent Neural Network Based Language Model. InProceedings of INTERSPEECH

Recurrent Neural Language Model (Test)

Sequentially Sample from Output Distribution

$$\tilde{x}_5 \sim p(x_5) = \text{softmax}(\hat{y}_4)$$

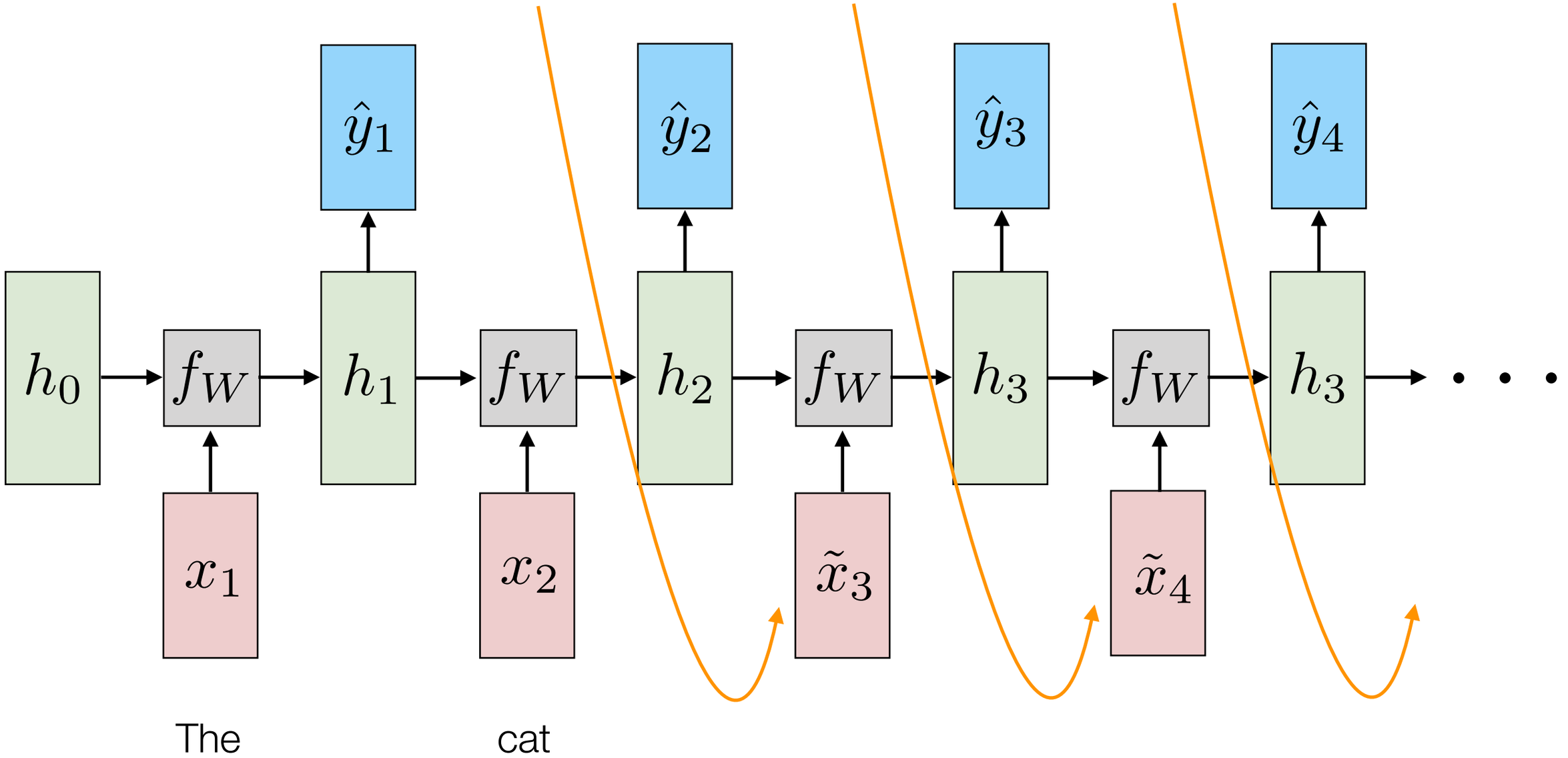


Tomas Mikolov, Martin Karafiat, Lukas Burget, Jan Cernocky, and Sanjeev Khudanpur. 2010. Recurrent Neural Network Based Language Model. InProceedings of INTERSPEECH

Recurrent Neural Language Model (Test)

Sequentially Sample from Output Distribution

$$\tilde{x}_5 \sim p(x_5) = \text{softmax}(\hat{y}_4)$$



Tomas Mikolov, Martin Karafiat, Lukas Burget, Jan Cernocky, and Sanjeev Khudanpur. 2010. Recurrent Neural Network Based Language Model. InProceedings of INTERSPEECH

Lab: RNNs for Time Series Prediction

Pablo M. Olmos pamartin@ing.uc3m.es

In this notebook, you will deploy a simple probabilistic model using a RNN to infer the probability distribution of a time-series. Given a set of signals $\mathcal{D} = \{X^1, X^2, \dots, X^N\}$ where

$$X^i = [X_0^i, X_1^i, \dots, X_T^i]$$

is the i-th signal, we will train a probabilistic model of the form

$$p(X|X_0) = \prod_{t=1}^{T} p(X_t|\mathbf{X}_{0:t-1})$$

where $\mathbf{X}_{0:t-1} = [X_0, X_1, \dots, X_{t-1}]$ and every conditional factor $p(X_t|\mathbf{X}_{0:t-1})$ corresponds to a Gaussian distribution with mean and variance obtained **from the state of a RNN** with input X_{t-1}. The RNN state $\mathbf{h}_{t-1}$ is a projection of the the signal up to time $t-2$, i.e, $\mathbf{X}_{0:t-2}$:

$$p(X_t|X_{t-1}, \mathbf{X}_{0:t-2}) = \mathcal{N}\left(f_{RNN}(X_{t-1}, \mathbf{h}_{t-1}), \sigma^2 + g_{RNN}(X_{t-1}, \mathbf{h}_{t-1})\right)$$

, where σ^2 is a constant basal variance that we treat as an hiper-parameter. During training, we approximate the expected loss at time t

$$\mathcal{L}_t = \mathbb{E}_{\hat{X}_t \sim p(X_t|X_{t-1}, \mathbf{X}_{0:t-2})}[(X_t - \hat{X}_t)^2]$$

using a single sample of $p(X_t|X_{t-1}, \mathbf{X}_{0:t-2})$. The global loss is accumulated during the whole signal length:

$$\mathcal{L} = \frac{1}{T}\sum_{t=1}^{T} \mathcal{L}_t$$